

Módulo 6 - Guía práctica

Curso Spring Boot 2026

Índice

Módulo 6 - Guía práctica: de la seguridad básica a una API JWT stateless	8
Qué vas a construir y comprobar	8
Punto de partida	8
Cómo trabajar con esta guía	9
Práctica 6.1 — Introducción a Spring Security	9
Preparación y contexto de la fuente	9
Paso 6.1.1 — Añadir la dependencia de Spring Security	9
Base de la fuente — preservada	9
Implementación acumulativa M6 — actualizada	9
Paso 6.1.2 — Arrancar y ver el “susto”	9
Base de la fuente — preservada	10
Implementación acumulativa M6 — actualizada	10
Paso 6.1.3 — Autenticarse con la contraseña generada	10
Base de la fuente — preservada	10
Implementación acumulativa M6 — actualizada	10
Paso 6.1.4 — Ver los filtros en acción	11
Base de la fuente — preservada	11
Implementación acumulativa M6 — actualizada	11
Paso 6.1.5 — Crear SecurityConfig	11
Base de la fuente — preservada	11
Implementación acumulativa M6 — actualizada	12
Paso 6.1.6 — Ver el comportamiento sin usuarios definidos por nosotros	12
Base de la fuente — preservada	12
Implementación acumulativa M6 — actualizada	12
Paso 6.1.7 — Permitir todos los endpoints temporalmente	13
Base de la fuente — preservada	13
Implementación acumulativa M6 — actualizada	13
Paso 6.1.8 — Configurar reglas específicas por endpoint	13
Base de la fuente — preservada	13
Implementación acumulativa M6 — actualizada	14
Paso 6.1.9 — Ver la diferencia entre 401 y 403	14
Base de la fuente — preservada	14
Implementación acumulativa M6 — actualizada	15
Paso 6.1.10 — Depurar con logs	15
Base de la fuente — preservada	15
Implementación acumulativa M6 — actualizada	15
Paso 6.1.11 — Errores comunes del ejercicio	15
Base de la fuente — preservada	15
Implementación acumulativa M6 — actualizada	16
Paso 6.1.12 — Reto resuelto: endpoint público sin autenticación	16

Base de la fuente — preservada	16
Implementación acumulativa M6 — actualizada	18
6.2 — Autenticación con usuarios en memoria	18
Práctica 6.2 — Autenticación con usuarios en memoria	18
Preparación y contexto de la fuente	18
Paso 6.2.1 — Repasar el estado actual	18
Base de la fuente — preservada	18
Implementación acumulativa M6 — actualizada	18
Paso 6.2.2 — Añadir un endpoint que devuelva el perfil del usuario	19
Base de la fuente — preservada	19
Implementación acumulativa M6 — actualizada	20
Paso 6.2.3 — Definir el PasswordEncoder	20
Base de la fuente — preservada	21
Implementación acumulativa M6 — actualizada	21
Paso 6.2.4 — Definir el UserDetailsService	21
Base de la fuente — preservada	22
Implementación acumulativa M6 — actualizada	22
Paso 6.2.5 — Cerrar los endpoints	23
Base de la fuente — preservada	23
Implementación acumulativa M6 — actualizada	23
Paso 6.2.6 — Arrancar y probar sin autenticación	24
Base de la fuente — preservada	24
Implementación acumulativa M6 — actualizada	24
Paso 6.2.7 — Autenticarse con el usuario ana	25
Base de la fuente — preservada	25
Implementación acumulativa M6 — actualizada	25
Paso 6.2.8 — Autenticarse con el usuario admin	25
Base de la fuente — preservada	25
Implementación acumulativa M6 — actualizada	26
Paso 6.2.9 — Probar la autorización por rol	26
Base de la fuente — preservada	26
Implementación acumulativa M6 — actualizada	26
Paso 6.2.10 — Probar con credenciales incorrectas	27
Base de la fuente — preservada	27
Implementación acumulativa M6 — actualizada	27
Paso 6.2.11 — Errores comunes del ejercicio	27
Base de la fuente — preservada	27
Implementación acumulativa M6 — actualizada	28
Paso 6.2.12 — Reto resuelto: tercer usuario con roles combinados	28
Base de la fuente — preservada	28
Implementación acumulativa M6 — actualizada	30
1. Evolucionar InMemoryUserConfig	30
2. Crear el endpoint	30
3. Autorizar cualquiera de los dos roles	30
4. Probar	31
Resultado esperado global de 6.2	31
Restauraciones/evoluciones registradas	31
6.3 — Usuarios en base de datos	31
Práctica 6.3 — Usuarios en base de datos	31
Preparación y contexto de la fuente	31
Paso 6.3.1 — Crear el paquete auth	32
Base de la fuente — preservada	32

Implementación acumulativa M6 — actualizada	32
Paso 6.3.2 — Crear la entidad Rol	32
Base de la fuente — preservada	32
Implementación acumulativa M6 — actualizada	33
Paso 6.3.3 — Crear la entidad Usuario	34
Base de la fuente — preservada	34
Implementación acumulativa M6 — actualizada	35
Paso 6.3.4 — Crear los repositorios	35
Base de la fuente — preservada	35
Implementación acumulativa M6 — actualizada	36
Paso 6.3.5 — Crear el UserDetailsService personalizado	36
Base de la fuente — preservada	36
Implementación acumulativa M6 — actualizada	37
Paso 6.3.6 — Eliminar los usuarios en memoria	38
Base de la fuente — preservada	38
Implementación acumulativa M6 — actualizada	38
Paso 6.3.7 — Crear el inicializador de usuarios	39
Base de la fuente — preservada	39
Implementación acumulativa M6 — actualizada	40
Paso 6.3.8 — Arrancar y verificar	40
Base de la fuente — preservada	40
Implementación acumulativa M6 — actualizada	40
Paso 6.3.9 — Probar la autenticación	41
Base de la fuente — preservada	41
Implementación acumulativa M6 — actualizada	41
Paso 6.3.10 — Crear los DTOs de registro	41
Base de la fuente — preservada	41
Implementación acumulativa M6 — actualizada	42
Paso 6.3.11 — Crear AuthService y AuthController	43
Base de la fuente — preservada	43
Implementación acumulativa M6 — actualizada	44
Paso 6.3.12 — Configurar el endpoint público y probar	45
Base de la fuente — preservada	45
Implementación acumulativa M6 — actualizada	45
Paso 6.3.13 — Errores comunes del ejercicio	46
Base de la fuente — preservada	46
Implementación acumulativa M6 — actualizada	46
Paso 6.3.14 — Reto resuelto: cambiar la contraseña	47
Base de la fuente — preservada	47
Implementación acumulativa M6 — actualizada	48
DTO	49
Servicio	49
Controlador	49
Prueba	49
Corrección explícita de la fuente	50
Resultado esperado global 6.3	50

6.4 — Autorización por roles 50

Práctica 6.4 — Autorización por roles 50

Preparación y contexto de la fuente	50
Paso 6.4.1 — Repasar el estado actual	50
Base de la fuente — preservada	51
Implementación acumulativa M6 — actualizada	51
Paso 6.4.2 — Activar la seguridad por método	51
Base de la fuente — preservada	51

Implementación acumulativa M6 — actualizada	51
Paso 6.4.3 — Crear un UsuarioController de administración	52
Base de la fuente — preservada	52
Implementación acumulativa M6 — actualizada	53
Paso 6.4.4 — Crear el UsuarioService	53
Base de la fuente — preservada	53
Implementación acumulativa M6 — actualizada	54
Paso 6.4.5 — Añadir reglas de URL y de método combinadas	55
Base de la fuente — preservada	55
Implementación acumulativa M6 — actualizada	55
Paso 6.4.6 — Probar la autorización con distintos usuarios	55
Base de la fuente — preservada	56
Implementación acumulativa M6 — actualizada	56
Paso 6.4.7 — Crear un endpoint para consultar el propio perfil	56
Base de la fuente — preservada	56
Implementación acumulativa M6 — actualizada	57
Paso 6.4.8 — Crear UsuarioPrincipal para 6.7	57
Base de la fuente — preservada	57
Implementación acumulativa M6 — actualizada	59
Paso 6.4.9 — Probar la expresión de autorización	59
Base de la fuente — preservada	59
Implementación acumulativa M6 — actualizada	59
Paso 6.4.10 — Escribir tests de autorización	60
Base de la fuente — preservada	60
Implementación acumulativa M6 — actualizada	61
Paso 6.4.11 — Errores comunes del ejercicio	62
Base de la fuente — preservada	62
Implementación acumulativa M6 — actualizada	62
Paso 6.4.12 — Reto resuelto: endpoint de cambio de rol	62
Base de la fuente — preservada	63
Implementación acumulativa M6 — actualizada	64
DTO	65
Servicio	65
Controlador	65
Prueba	65
Resultado esperado global 6.4	66

6.5 — Introducción a JWT 66

Práctica 6.5 — Introducción a JWT 66

Preparación y contexto de la fuente	66
Paso 6.5.1 — Entender la estructura con un ejemplo	66
Base de la fuente — preservada	66
Implementación acumulativa M6 — actualizada	67
Paso 6.5.2 — Crear una clase para generar un JWT manualmente	67
Base de la fuente — preservada	67
Implementación acumulativa M6 — actualizada	69
Paso 6.5.3 — Decodificar el token generado	69
Base de la fuente — preservada	69
Implementación acumulativa M6 — actualizada	69
Paso 6.5.4 — Modificar el payload sin actualizar la firma	70
Base de la fuente — preservada	70
Implementación acumulativa M6 — actualizada	70
Paso 6.5.5 — Verificar el token con Java	70
Base de la fuente — preservada	70
Implementación acumulativa M6 — actualizada	71

Paso 6.5.6 — Probar con distintos payloads	71
Base de la fuente — preservada	71
Implementación acumulativa M6 — actualizada	72
Paso 6.5.7 — Reflexionar sobre la seguridad	72
Base de la fuente — preservada	72
Implementación acumulativa M6 — actualizada	72
Paso 6.5.8 — Comparar con un token real	73
Base de la fuente — preservada	73
Implementación acumulativa M6 — actualizada	73
Paso 6.5.9 — Errores comunes del ejercicio	73
Base de la fuente — preservada	73
Implementación acumulativa M6 — actualizada	74
Paso 6.5.10 — Experimentar con la expiración	74
Base de la fuente — preservada	74
Implementación acumulativa M6 — actualizada	74
Paso 6.5.11 — Verificar el token con una clave incorrecta	74
Base de la fuente — preservada	75
Implementación acumulativa M6 — actualizada	75
Paso 6.5.12 — Reto resuelto: decodificar un token sin verificar la firma	75
Base de la fuente — preservada	75
Implementación acumulativa M6 — actualizada	76
Resultado esperado global 6.5	77
6.6 — Generación de tokens JWT	77
Práctica 6.6 — Generación de tokens JWT	77
Preparación y contexto de la fuente	77
Paso 6.6.1 — Añadir las dependencias de JWT	77
Base de la fuente — preservada	77
Implementación acumulativa M6 — actualizada	78
Paso 6.6.2 — Configurar las propiedades de JWT	78
Base de la fuente — preservada	78
Implementación acumulativa M6 — actualizada	78
Paso 6.6.3 — Crear la configuración de la clave	79
Base de la fuente — preservada	79
Implementación acumulativa M6 — actualizada	79
Paso 6.6.4 — Crear el JwtService	80
Base de la fuente — preservada	80
Implementación acumulativa M6 — actualizada	82
Paso 6.6.5 — Crear los DTOs de login	83
Base de la fuente — preservada	83
Implementación acumulativa M6 — actualizada	84
Paso 6.6.6 — Ampliar AuthService con login y refresh	84
Base de la fuente — preservada	84
Implementación acumulativa M6 — actualizada	86
Paso 6.6.7 — Ampliar AuthController	86
Base de la fuente — preservada	86
Implementación acumulativa M6 — actualizada	86
Paso 6.6.8 — Configurar la seguridad para permitir login	87
Base de la fuente — preservada	87
Implementación acumulativa M6 — actualizada	87
Paso 6.6.9 — Arrancar y probar el login	87
Base de la fuente — preservada	87
Implementación acumulativa M6 — actualizada	87
Paso 6.6.10 — Decodificar el token	88
Base de la fuente — preservada	88

Implementación acumulativa M6 — actualizada	88
Paso 6.6.11 — Probar el refresh	88
Base de la fuente — preservada	88
Implementación acumulativa M6 — actualizada	89
Paso 6.6.12 — Errores comunes del ejercicio	89
Base de la fuente — preservada	89
Implementación acumulativa M6 — actualizada	89
Materialización exacta del contrato de error en el proyecto final	89
Paso 6.6.13 — Reto resuelto: endpoint para logout	90
Base de la fuente — preservada	90
Implementación acumulativa M6 — actualizada	92
Cierre 6.6	92
6.7 — Filtro JWT	92
Práctica 6.7 — Filtro JWT	92
Preparación y contexto de la fuente	92
Paso 6.7.1 — Verificar JwtService y UsuarioPrincipal	93
Base de la fuente — preservada	93
Implementación acumulativa M6 — actualizada	93
Paso 6.7.2 — Crear JwtAuthenticationFilter	93
Base de la fuente — preservada	93
Implementación acumulativa M6 — actualizada	95
Paso 6.7.3 — Registrar el filtro en la cadena	96
Base de la fuente — preservada	96
Implementación acumulativa M6 — actualizada	96
Paso 6.7.4 — Desactivar HTTP Basic	97
Base de la fuente — preservada	97
Implementación acumulativa M6 — actualizada	97
Paso 6.7.5 — Arrancar y probar	97
Base de la fuente — preservada	97
Implementación acumulativa M6 — actualizada	98
Paso 6.7.6 — Verificar UsuarioPrincipal	98
Base de la fuente — preservada	98
Implementación acumulativa M6 — actualizada	99
Paso 6.7.7 — Probar con rol ADMIN	99
Base de la fuente — preservada	99
Implementación acumulativa M6 — actualizada	99
Paso 6.7.8 — Probar con un token inválido	99
Base de la fuente — preservada	99
Implementación acumulativa M6 — actualizada	100
Paso 6.7.9 — Probar con cabecera mal formada	100
Base de la fuente — preservada	100
Implementación acumulativa M6 — actualizada	100
Paso 6.7.10 — Depurar el filtro	100
Base de la fuente — preservada	100
Implementación acumulativa M6 — actualizada	100
Paso 6.7.11 — Errores comunes del ejercicio	101
Base de la fuente — preservada	101
Implementación acumulativa M6 — actualizada	101
Paso 6.7.12 — Reto resuelto: SpEL con ID del usuario	101
Base de la fuente — preservada	101
Implementación acumulativa M6 — actualizada	103
Cierre 6.7	104
6.8 — Protección de endpoints con JWT	104

Práctica 6.8 — Protección de endpoints con JWT	104
Preparación y contexto de la fuente	104
Paso 6.8.1 — Crear JwtAuthenticationEntryPoint	104
Base de la fuente — preservada	104
Implementación acumulativa M6 — actualizada	105
Paso 6.8.2 — Crear JwtAccessDeniedHandler	105
Base de la fuente — preservada	105
Implementación acumulativa M6 — actualizada	106
Paso 6.8.3 — Actualizar SecurityConfig	106
Base de la fuente — preservada	106
Implementación acumulativa M6 — actualizada	107
Paso 6.8.4 — Añadir endpoint de gestor	108
Base de la fuente — preservada	108
Implementación acumulativa M6 — actualizada	108
Paso 6.8.5 — Añadir rol GESTOR al inicializador	108
Base de la fuente — preservada	108
Implementación acumulativa M6 — actualizada	109
Paso 6.8.6 — Probar el flujo completo	109
Base de la fuente — preservada	109
Implementación acumulativa M6 — actualizada	109
Paso 6.8.7 — Probar sin token	110
Base de la fuente — preservada	110
Implementación acumulativa M6 — actualizada	110
Paso 6.8.8 — Probar refresh token	110
Base de la fuente — preservada	110
Implementación acumulativa M6 — actualizada	110
Paso 6.8.9 — Probar logout	111
Base de la fuente — preservada	111
Implementación acumulativa M6 — actualizada	111
Paso 6.8.10 — Escribir tests de integración	111
Base de la fuente — preservada	111
Implementación acumulativa M6 — actualizada	112
Paso 6.8.11 — Errores comunes del ejercicio	112
Base de la fuente — preservada	112
Implementación acumulativa M6 — actualizada	113
Paso 6.8.12 — Reto resuelto: consultar el propio usuario	113
Base de la fuente — preservada	113
Implementación acumulativa M6 — actualizada	114
Cierre 6.8	115
6.9 — Testing de seguridad	115
Práctica 6.9 — Testing de seguridad	115
Preparación y contexto de la fuente	115
Paso 6.9.1 — Añadir spring-security-test	115
Base de la fuente — preservada	115
Implementación acumulativa M6 — actualizada	115
Paso 6.9.2 — Crear test del UsuarioController	115
Base de la fuente — preservada	116
Implementación acumulativa M6 — actualizada	117
Paso 6.9.3 — Ejecutar los tests	117
Base de la fuente — preservada	117
Implementación acumulativa M6 — actualizada	117
Paso 6.9.4 — Testear endpoint de gestor	118
Base de la fuente — preservada	118
Implementación acumulativa M6 — actualizada	118

Paso 6.9.5 — Testear endpoint público	119
Base de la fuente — preservada	119
Implementación acumulativa M6 — actualizada	119
Paso 6.9.6 — Testear login	119
Base de la fuente — preservada	119
Implementación acumulativa M6 — actualizada	121
Paso 6.9.7 — Testear refresh	121
Base de la fuente — preservada	121
Implementación acumulativa M6 — actualizada	121
Paso 6.9.8 — Testear logout	122
Base de la fuente — preservada	122
Implementación acumulativa M6 — actualizada	122
Paso 6.9.9 — Testear con post-processors	122
Base de la fuente — preservada	122
Implementación acumulativa M6 — actualizada	122
Paso 6.9.10 — Testear con @WithUserDetails	123
Base de la fuente — preservada	123
Implementación acumulativa M6 — actualizada	124
Paso 6.9.11 — Testear el filtro JWT	124
Base de la fuente — preservada	124
Implementación acumulativa M6 — actualizada	125
Paso 6.9.12 — Errores comunes del ejercicio	125
Base de la fuente — preservada	125
Implementación acumulativa M6 — actualizada	125
Paso 6.9.13 — Reto resuelto: test completo de autorización	126
Base de la fuente — preservada	126
Implementación acumulativa M6 — actualizada	128
Resultado esperado global del módulo	128

Módulo 6 - Guía práctica: de la seguridad básica a una API JWT stateless

Qué vas a construir y comprobar

En esta guía transformarás el proyecto final del Módulo 5 en una aplicación protegida con Spring Security y JWT. El trabajo es acumulativo: cada práctica parte del estado conseguido en la anterior y el snapshot final de M6/proyecto/ representa el resultado de completar todos los pasos en orden.

Empezaremos viendo el comportamiento seguro por defecto y usando HTTP Basic como herramienta temporal de aprendizaje. Después crearemos usuarios y roles, moveremos las identidades a base de datos, aplicaremos autorización por URL y por método, estudiaremos la estructura de JWT y construiremos login, refresh, filtro JWT, logout/revocación y configuración stateless. El módulo termina con tests de seguridad que cubren los caminos de éxito y de rechazo.

Durante el recorrido aparecen estados deliberadamente temporales -por ejemplo permitAll, usuarios en memoria, HTTP Basic o experimentos manuales de JWT-. Cada uno se cierra, elimina o sustituye explícitamente antes del estado final. Esto permite experimentar sin dejar configuraciones provisionales escondidas en el proyecto publicado.

Punto de partida

Necesitas el proyecto final de M5, Java 17 y el Maven Wrapper del curso. No hace falta instalar Maven globalmente. Los comandos se muestran con `./mvnw`; en Windows puedes ejecutar el equivalente `mvnw.cmd`.

Cómo trabajar con esta guía

1. Ejecuta los pasos 6.1.1-6.9.13 en orden.
2. Antes de cambiar código, identifica el fichero y el estado que indica el paso.
3. Ejecuta el comando de comprobación propuesto y observa el resultado antes de continuar.
4. Cuando un ejercicio sea temporal, completa también su restauración o eliminación.
5. Al finalizar, compara tu árbol con M6/proyecto/: debe representar exactamente el mismo estado funcional.

Práctica 6.1 — Introducción a Spring Security

Preparación y contexto de la fuente

Contexto del ejercicio: Vamos a añadir Spring Security al proyecto, ver el comportamiento por defecto, y configurar un SecurityFilterChain para controlar qué endpoints están protegidos.

Requisitos previos: Tener el proyecto mi-proyecto con los endpoints del Módulo 4 y la configuración del Módulo 5.

Paso 6.1.1 — Añadir la dependencia de Spring Security

Fuente: p. 1220.

Base de la fuente — preservada

Abre el pom.xml y añade la dependencia:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

Guarda y recarga Maven.

Pregunta: ¿Qué crees que pasará con los endpoints actuales al arrancar la aplicación?

Implementación acumulativa M6 — actualizada

Qué hacemos. Partimos del pom.xml real de M5 y añadimos spring-boot-starter-security sin retirar ninguna dependencia heredada.

Por qué. Queremos observar el comportamiento seguro por defecto antes de configurarlo.

Archivo. M6/proyecto/pom.xml.

Comando de verificación cuando el árbol M5 esté completamente materializado:

```
./mvnw -B -DskipTests compile
```

Observable. Maven resuelve Spring Security y el proyecto compila sin perder web, validation, JPA, H2, PostgreSQL ni springdoc.

Error posible. Sustituir el POM de M5 por un POM mínimo. **Corrección:** comparar dependencias/plugins con el baseline; M6 sólo añade/evoluciona.

Paso 6.1.2 — Arrancar y ver el “susto”

Fuente: p. 1220.

Base de la fuente — preservada

Arranca la aplicación. En los logs verás algo como:

```
Using generated security password: 3fa2b9e1-8c5d-4a7e-9f2b-1d4e5c6a7b8d
```

```
This generated password is for development use only. Your security configuration  
↪ must be updated before running your application in production.
```

Spring Security ha generado una contraseña aleatoria para el usuario user. Ahora prueba a hacer una petición:

```
curl -i http://localhost:8080/api/v1/alumnos
```

Verás un 401 Unauthorized con una cabecera WWW-Authenticate:

```
HTTP/1.1 401  
WWW-Authenticate: Basic realm="Realm"  
...
```

Todos los endpoints están protegidos. Es el comportamiento por defecto de Spring Security.

Pregunta: ¿Por qué Spring Security protege todo por defecto en lugar de dejar todo abierto?

Implementación acumulativa M6 — actualizada

```
./mvnw spring-boot:run
```

Observable. Spring Security protege las rutas por defecto y Spring Boot registra una contraseña de desarrollo para el usuario generado. Una petición anónima a un endpoint antes público deja de responder como en M5.

Qué aprendemos. Añadir el starter cambia el comportamiento runtime incluso antes de escribir SecurityConfig.

Paso 6.1.3 — Autenticarse con la contraseña generada

Fuente: p. 1221.

Base de la fuente — preservada

Prueba a autenticarte con el usuario user y la contraseña que aparece en los logs:

```
curl -i -u user:3fa2b9e1-8c5d-4a7e-9f2b-1d4e5c6a7b8d \  
http://localhost:8080/api/v1/alumnos
```

Verás la lista de alumnos. El flag -u de curl envía la cabecera Authorization: Basic base64(user:password).

Prueba también con el navegador: abre <http://localhost:8080/api/v1/alumnos>. Aparecerá un diálogo pidiendo usuario y contraseña. Introduce user y la contraseña generada. Verás la lista de alumnos.

Pregunta: ¿Qué diferencia hay entre usar HTTP Basic y usar un formulario de login?

Implementación acumulativa M6 — actualizada

Toma la contraseña generada del log de **ese arranque** y prueba, por ejemplo:

```
curl -i -u user:<PASSWORD_DEL_LOG> http://localhost:8080/hola
```

Observable. Con credenciales válidas el endpoint puede alcanzarse; sin ellas aparece el rechazo de autenticación. No se copia esa contraseña a properties ni se convierte en configuración permanente.

Paso 6.1.4 — Ver los filtros en acción

Fuente: p. 1222.

Base de la fuente — preservada

Activa el log DEBUG para Spring Security en application.properties:

```
logging.level.org.springframework.security=DEBUG
```

Reinicia la aplicación y haz una petición autenticada. Verás en los logs algo como:

```
Securing GET /api/v1/alumnos
Secured GET /api/v1/alumnos
Set SecurityContextHolder to AnonymousAuthenticationToken
...
```

Los logs muestran qué filtros han procesado la petición y qué decisiones han tomado.

Pregunta: ¿Qué información muestran los logs sobre el procesamiento de la seguridad?

Implementación acumulativa M6 — actualizada

Activa temporalmente:

```
logging.level.org.springframework.security=DEBUG
```

Repite una petición anónima y otra autenticada.

Observable. El log permite ver la cadena de seguridad y el tratamiento de la identidad.

Restauración. El nivel DEBUG es una ayuda diagnóstica temporal; no queda forzado como decisión final de producción.

Paso 6.1.5 — Crear SecurityConfig

Fuente: p. 1223.

Base de la fuente — preservada

Vamos a configurar Spring Security de forma explícita. Crea la clase SecurityConfig en el paquete config:

```
package es.mecd.demo.miproyecto.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.
    ↪ EnableWebSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableWebSecurity
public class SecurityConfig {
```

```

@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .authorizeHttpRequests(auth -> auth
            .anyRequest().authenticated()
        )
        .httpBasic(Customizer.withDefaults());
    return http.build();
}

```

Esta configuración reemplaza la configuración por defecto. Hace lo mismo (todo autenticado, HTTP Basic), pero ahora es explícita y podemos modificarla.

Reinicia la aplicación. Ahora la contraseña generada ya no aparece en los logs, porque hemos tomado el control de la configuración. Pero los endpoints siguen protegidos.

Pregunta: ¿Qué diferencia hay entre la configuración por defecto y esta configuración explícita?

Implementación acumulativa M6 — actualizada

El proyecto acumulativo incluye `src/main/java/es/meecd/demo/miproyecto/config/SecurityConfig.java` con configuración moderna basada en bean `SecurityFilterChain`.

Puntos obligatorios del estado 6.1:

```

.csrf(AbstractHttpConfigurer::disable)
.cors(Customizer.withDefaults())
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/api/v1/public/**").permitAll()
    .anyRequest().authenticated()
)
.httpBasic(Customizer.withDefaults())

```

Observable. Existe una política explícita; no se usa `WebSecurityConfigurerAdapter`.

Paso 6.1.6 — Ver el comportamiento sin usuarios definidos por nosotros

Fuente: p. 1224.

Base de la fuente — preservada

Si has seguido los pasos, verás que ahora no hay usuarios definidos. Al autenticarte, Spring Security responde con un 401 porque no encuentra ningún `UserDetailsService`.

Prueba:

```
curl -i -u user:password http://localhost:8080/api/v1/alumnos
```

Verás un 401. Aún no hemos definido usuarios. Eso lo haremos en el siguiente punto (6.2).

Pregunta: ¿Qué pasa si no hay usuarios definidos? ¿Dónde buscaría Spring Security las credenciales?

Implementación acumulativa M6 — actualizada

No declares todavía un `UserDetailsService` propio. Reinicia y comprueba que el usuario de desarrollo autoconfigurado sigue permitiendo observar HTTP Basic.

Objetivo pedagógico. Separar “cadena de seguridad” de “fuente de usuarios”. Los usuarios controlados llegan en 6.2.

Paso 6.1.7 — Permitir todos los endpoints temporalmente

Fuente: p. 1225.

Base de la fuente — preservada

Para poder seguir trabajando mientras configuramos la seguridad, vamos a permitir temporalmente todos los endpoints. Modifica el SecurityConfig:

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .authorizeHttpRequests(auth -> auth
            .anyRequest().permitAll()
        )
        .csrf(csrf -> csrf.disable())
        .httpBasic(Customizer.withDefaults());
    return http.build();
}
```

`anyRequest().permitAll()` permite el acceso sin autenticación a todos los endpoints.

`csrf(csrf -> csrf.disable())` desactiva la protección CSRF. Para APIs REST sin sesión, no es necesaria. La veremos más adelante.

Reinicia y prueba:

```
curl -i http://localhost:8080/api/v1/alumnos
```

Ahora devuelve 200. Los endpoints están abiertos.

Pregunta: ¿Por qué se desactiva CSRF en APIs REST? ¿Qué protección aporta CSRF en aplicaciones web tradicionales?

Implementación acumulativa M6 — actualizada

Estado temporal T6.1-A — INTRODUCER. Sustituye momentáneamente las reglas por:

```
.anyRequest().permitAll()
```

Arranca y comprueba que `/hola` y `/api/v1/alumnos` vuelven a ser accesibles sin autenticación.

No avanzar dejando este estado activo. Este paso existe para demostrar que la política declarativa controla el acceso.

Paso 6.1.8 — Configurar reglas específicas por endpoint

Fuente: p. 1226.

Base de la fuente — preservada

Vamos a configurar reglas más finas. Modifica el SecurityConfig:

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/v1/public/**").permitAll()
        )
}
```

```

        .requestMatchers("/h2-console/**").permitAll()
        .requestMatchers("/swagger-ui/**", "/v3/api-docs/**").permitAll()
        .anyRequest().authenticated()
    )
    .csrf(csrf -> csrf.disable())

    .httpBasic(Customizer.withDefaults());
    return http.build();
}

```

Reglas:

- /api/v1/public/**: acceso libre (por ejemplo, endpoints de consulta pública).
- /h2-console/**: acceso libre (consola de H2 en desarrollo).
- /swagger-ui/** y /v3/api-docs/**: acceso libre (documentación de la API).
- anyRequest().authenticated(): el resto requiere autenticación.

Prueba:

Endpoint público

```
curl -i http://localhost:8080/api/v1/public/saludo
```

Endpoint protegido

```
curl -i http://localhost:8080/api/v1/alumnos
```

Consola H2

```
curl -i http://localhost:8080/h2-console
```

Los dos primeros devolverán 401 para los endpoints protegidos, 200 para los públicos. La consola de H2 devolverá 200.

Pregunta: ¿Por qué el orden de las reglas importa? ¿Qué pasaría si anyRequest() estuviera primero?

Implementación acumulativa M6 — actualizada

Estado temporal T6.1-A — RESTORE/EVOLVE. Elimina el permitAll() global y deja únicamente rutas públicas explícitas; el resto vuelve a authenticated().

```

.requestMatchers("/api/v1/public/**").permitAll()
.anyRequest().authenticated()

```

Verificación negativa:

```
curl -i http://localhost:8080/api/v1/alumnos
```

Debe volver a requerir autenticación.

Paso 6.1.9 — Ver la diferencia entre 401 y 403

Fuente: p. 1228.

Base de la fuente — preservada

Añade una regla que requiera un rol concreto:

```

.requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
.anyRequest().authenticated()

```

Prueba a acceder a un endpoint de admin sin autenticarte:

```
curl -i http://localhost:8080/api/v1/admin/usuarios
```

Verás un 401 Unauthorized porque no estás autenticado.

Ahora auténticate con el usuario que hayas definido (todavía no hay ninguno, pero en el siguiente punto lo configuramos). Cuando lo tengas, si el usuario no tiene el rol ADMIN, verás un 403 Forbidden.

Pregunta: ¿Qué diferencia hay entre 401 y 403 en este ejemplo?

Implementación acumulativa M6 — actualizada

Para obtener ambos observables de forma controlada, usa temporalmente una regla de rol sobre un endpoint existente:

```
.requestMatchers("/api/v1/alumnos/**").hasRole("ADMIN")
```

- Sin credenciales: se observa el flujo de **401**.
- Con el usuario generado, que no posee ADMIN: se observa **403**.

Restauración del experimento. Después de observar ambos códigos, devuelve `/api/v1/alumnos/**` al nivel previsto para el cierre 6.1 (`authenticated()`); los roles controlados se introducen en 6.2/6.4.

Paso 6.1.10 — Depurar con logs

Fuente: p. 1228.

Base de la fuente — preservada

Con el log DEBUG activado, haz peticiones y observa los logs:

```
Securing GET /api/v1/alumnos
Secured GET /api/v1/alumnos
Set SecurityContextHolder to AnonymousAuthenticationToken
...
Pre-authenticated entry point called. Rejecting access
```

Los logs muestran cada paso. Si algo no funciona como esperas, los logs te dicen por qué.

Pregunta: ¿Qué información adicional te dan los logs DEBUG de Spring Security?

Implementación acumulativa M6 — actualizada

Repite peticiones con DEBUG de Spring Security y relaciona cada código con el punto de la cadena que tomó la decisión. Después restaura el logging ordinario.

Paso 6.1.11 — Errores comunes del ejercicio

Fuente: p. 1229.

Base de la fuente — preservada

Error Causa Solución

Spring Security protege por Configurar `permitAll` para 401 en todos los endpoints defecto endpoints públicos

Se ha configurado La contraseña generada no aparece Definir usuarios explícitamente un `SecurityFilterChain`

Error Causa Solución

El usuario está autenticado pero 403 en lugar de 401 Revisar los roles sin permisos

Falta @EnableWebSecurity o el La configuración no se aplica Añadirlos bean no se crea

Los logs no muestran nada El nivel de log no está en DEBUG Activarlo

WebSecurityConfigurerAdapter no Está eliminado en Spring Boot Usar SecurityFilterChain compila 3.x

Spring Security bloquea CORS bloquea el preflight Configurar CORS en la cadena OPTIONS

Implementación acumulativa M6 — actualizada

Comprueba explícitamente estos fallos antes de cerrar el punto:

- dejar `anyRequest().permitAll()` activo;
- abrir Swagger u otras rutas por comodidad sin justificarlo;
- desactivar CORS al introducir Security;
- confundir 401 y 403;
- introducir `WebSecurityConfigurerAdapter`;
- esconder una contraseña fija en el repositorio;
- borrar tests heredados porque ahora reciben 401.

La corrección de los tests heredados debe consistir en adaptarlos al nuevo contrato de seguridad, no en eliminarlos.

Paso 6.1.12 — Reto resuelto: endpoint público sin autenticación

Fuente: p. 1230.

Base de la fuente — preservada

Reto: Añadir un endpoint `/api/v1/public/info` que devuelva información de la aplicación sin requerir autenticación.

Solución paso a paso:

Paso 1: Crear el controlador:

```

@RestController
@RequestMapping("/api/v1/public")
public class PublicController {

    @GetMapping("/info")
    public Map<String, String> info() {
        return Map.of(
            "aplicacion", "mi-proyecto",
            "version", "1.0.0",
            "estado", "activo"
        );
    }
}

```

Paso 2: Verificar que la regla `/api/v1/public/` ya está en el `SecurityConfig` con `permitAll()`.

Paso 3: Reiniciar y probar:

```
curl -i http://localhost:8080/api/v1/public/info
```

Verás un 200 con el JSON de información, sin autenticación.

Paso 4: Probar un endpoint que no sea público:

```
curl -i http://localhost:8080/api/v1/alumnos
```

Verás un 401.

Pregunta: ¿Qué habría que cambiar para que solo el endpoint `/api/v1/public/info` fuera público, y no todo `/api/v1/public/**`?

Resultado esperado global

Al final del ejercicio, deberías tener:

- Spring Security añadido al proyecto.
- Un SecurityConfig con SecurityFilterChain configurado.
- Reglas de autorización por endpoint.
- CSRF desactivado.
- La capacidad de probar distintos escenarios con curl.

Resumen técnico del ejercicio El ejercicio ha demostrado:

- spring-boot-starter-security: añade seguridad al proyecto.
- Comportamiento por defecto: todo protegido, contraseña aleatoria, HTTP Basic.
- SecurityFilterChain como bean: configuración moderna.
- authorizeHttpRequests: reglas de autorización.
- permitAll vs authenticated: endpoints públicos vs privados.
- csrf().disable(): desactivar CSRF para APIs REST.
- Logs DEBUG: diagnóstico de seguridad.
- Diferencia 401 vs 403.

Conclusión y enlace al siguiente punto En este punto 6.1 hemos introducido Spring Security:

- Qué es y qué problema resuelve.
- El “susto” inicial: todo se protege por defecto.
- Autenticación vs autorización: 401 vs 403.
- Cadena de filtros: cómo Spring Security intercepta peticiones.
- WebSecurityConfigurerAdapter deprecado: usar SecurityFilterChain como bean.
- @EnableWebSecurity: activa la configuración de seguridad.
- AuthenticationManager, UserDetailsService, PasswordEncoder.
- Ecosistema: en memoria, base de datos, OAuth2, JWT, SAML.
- Configuración básica con SecurityFilterChain.

En la práctica, hemos añadido Spring Security, hemos visto el comportamiento por defecto, hemos configurado un SecurityFilterChain, y hemos probado reglas de autorización con curl.

La idea clave: Spring Security protege todo por defecto. Tú decides qué abrir y a quién. La configuración moderna se hace con SecurityFilterChain como bean, no con herencia.

En el siguiente punto, 6.2 — Autenticación con usuarios en memoria, configuraremos usuarios con InMemoryUserDetailsManager, aprenderemos a usar BCryptPasswordEncoder, y veremos cómo autenticarnos con HTTP Basic.

Fin del Punto 6.1.

Implementación acumulativa M6 — actualizada

El proyecto acumulativo incorpora PublicInfoController:

```
@RestController
@RequestMapping("/api/v1/public")
public class PublicInfoController {
    @GetMapping("/info")
    public Map<String, String> info() {
        return Map.of("modulo", "M6", "seguridad", "activa");
    }
}
```

Prueba:

```
curl -i http://localhost:8080/api/v1/public/info
curl -i http://localhost:8080/api/v1/alumnos
```

Observable esperado. El primero es público; el segundo continúa protegido.

Cierre 6.1. No queda permitAll() global; HTTP Basic y el usuario generado son estados de aprendizaje que evolucionarán en los puntos siguientes, no la arquitectura final M6.

6.2 — Autenticación con usuarios en memoria

Estado temporal controlado. Este punto introduce InMemoryUserDetailsManager y mantiene HTTP Basic para poder observar el flujo de autenticación. Los usuarios en memoria se eliminan expresamente en 6.3.6; HTTP Basic se cierra en 6.7.4. No representan el diseño final JWT de M6.

Práctica 6.2 — Autenticación con usuarios en memoria

Preparación y contexto de la fuente

Contexto del ejercicio: Vamos a definir dos usuarios en memoria (uno con rol USER y otro con rol ADMIN), cifrar sus contraseñas con BCrypt, y verificar el acceso a distintos endpoints según el rol.

Requisitos previos: Tener el proyecto mi-proyecto con el SecurityConfig del punto 6.1.

Paso 6.2.1 — Repasar el estado actual

Fuente: p. 1251.

Base de la fuente — preservada

Abre el SecurityConfig y observa la configuración actual. Todos los endpoints están abiertos (permitAll). Vamos a cerrarlos y a definir usuarios.

Pregunta: ¿Qué falta para que la autenticación funcione?

Implementación acumulativa M6 — actualizada

Qué hacemos. Antes de añadir usuarios propios, verificamos el cierre de 6.1:

- spring-boot-starter-security está presente;
- existe un SecurityFilterChain;
- /api/v1/public/** es público;
- el resto requiere autenticación;
- el permitAll() global experimental ya no existe;
- CORS M5 continúa configurado.

Por qué. 6.2 debe evolucionar 6.1, no abrir una implementación paralela.

Archivos.

- src/main/java/es/mecd/demo/miproyecto/config/SecurityConfig.java
- src/main/java/es/mecd/demo/miproyecto/config/CorsConfig.java
- pom.xml

Verificación cuando el árbol M5 esté materializado:

```
./mvnw -B test
```

Observable. El proyecto mantiene el contrato acumulado; todavía no hay usuarios controlados por el curso.

Pregunta. ¿Qué diferencia hay entre definir una política de autorización y definir de dónde salen los usuarios?

Paso 6.2.2 — Añadir un endpoint que devuelva el perfil del usuario

Fuente: p. 1252.

Base de la fuente — preservada

Antes de configurar usuarios, vamos a añadir un endpoint que devuelva la identidad del usuario autenticado. Es útil para verificar que la autenticación funciona.

Crea un PerfilController:

```
package es.mecd.demo.miproyecto.common.controller;

import org.springframework.security.core.Authentication;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.Map;

@RestController
@RequestMapping("/api/v1/perfil")
public class PerfilController {

    @GetMapping
    public Map<String, Object> perfil(Authentication authentication) {

        return Map.of(
            "usuario", authentication.getName(),
            "authorities", authentication.getAuthorities().stream()
                .map(GrantedAuthority::getAuthority)
                .toList()
        );
    }
}
```

```
}
```

Authentication authentication se inyecta automáticamente por Spring Security. Contiene la identidad del usuario autenticado.

authentication.getName() devuelve el nombre de usuario.

authentication.getAuthorities() devuelve la lista de roles y permisos.

Pregunta: ¿Qué devuelve este endpoint si el usuario no está autenticado?

Implementación acumulativa M6 — actualizada

Creamos:

src/main/java/es/mecd/demo/miproyecto/common/controller/PerfilController.java

```
package es.mecd.demo.miproyecto.common.controller;

import java.util.Map;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/v1/perfil")
public class PerfilController {

    @GetMapping
    public Map<String, Object> perfil(Authentication authentication) {
        return Map.of(
            "usuario", authentication.getName(),
            "authorities", authentication.getAuthorities().stream()
                .map(GrantedAuthority::getAuthority)
                .toList());
    }
}
```

Qué demuestra. Spring puede inyectar el Authentication ya resuelto por la cadena. El controlador no vuelve a validar la contraseña.

Comando posterior:

```
curl -i -u ana:ana123 http://localhost:8080/api/v1/perfil
```

Observable esperado una vez completados los pasos siguientes.

```
{
  "usuario": "ana",
  "authorities": ["ROLE_USER"]
}
```

Error posible. authentication nulo porque la ruta se dejó pública accidentalmente. **Corrección.** Mantener /api/v1/perfil dentro de anyRequest().authenticated().

Paso 6.2.3 — Definir el PasswordEncoder

Fuente: p. 1253.

Base de la fuente — preservada

Vamos a definir un bean PasswordEncoder en el SecurityConfig:

```
@Bean
public PasswordEncoder passwordEncoder() {

    return PasswordEncoderFactories.createDelegatingPasswordEncoder();
}
```

PasswordEncoderFactories.createDelegatingPasswordEncoder() crea un

↳ DelegatingPasswordEncoder con BCrypt como algoritmo por

defecto. Permite migrar a otros algoritmos en el futuro sin invalidar hashes.

Añade el import:

```
import org.springframework.security.crypto.factory.PasswordEncoderFactories;
import org.springframework.security.crypto.password.PasswordEncoder;
```

Pregunta: ¿Por qué usar DelegatingPasswordEncoder en lugar de BCryptPasswordEncoder directamente?

Implementación acumulativa M6 — actualizada

El encoder es un bean estable; no pertenece sólo al estado temporal de usuarios en memoria porque 6.3 también lo necesitará.

En SecurityConfig:

```
@Bean
PasswordEncoder passwordEncoder() {
    return PasswordEncoderFactories.createDelegatingPasswordEncoder();
}
```

Por qué no guardamos ana123 directamente. La contraseña que conoce el usuario es una entrada; el sistema conserva/compara una representación codificada.

Verificación conceptual:

```
String hash1 = passwordEncoder.encode("ana123");
String hash2 = passwordEncoder.encode("ana123");

assert !hash1.equals(hash2);
assert passwordEncoder.matches("ana123", hash1);
assert passwordEncoder.matches("ana123", hash2);
```

Que dos hashes sean diferentes no significa que una contraseña sea incorrecta: BCrypt incorpora salt.

Error posible. NoSuchBeanDefinitionException: PasswordEncoder. **Corrección.** Comprobar que el método está anotado con @Bean y que su clase está bajo el escaneo de Spring.

Pregunta. ¿Por qué matches funciona aunque dos llamadas a encode puedan producir hashes diferentes?

Paso 6.2.4 — Definir el UserDetailsService

Fuente: p. 1254.

Base de la fuente — preservada

Añade otro bean UserDetailsService:

```
@Bean
public UserDetailsService userDetailsService(PasswordEncoder passwordEncoder) {
    UserDetails ana = User.builder()
        .username("ana")
        .password(passwordEncoder.encode("ana123"))

        .roles("USER")
        .build();

    UserDetails admin = User.builder()
        .username("admin")
        .password(passwordEncoder.encode("admin123"))
        .roles("ADMIN", "USER")
        .build();

    return new InMemoryUserDetailsManager(ana, admin);
}
```

User.builder() construye el usuario paso a paso.

.username("ana") define el nombre de usuario.

.password(passwordEncoder.encode("ana123")) cifra la contraseña con el PasswordEncoder inyectado.

.roles("USER") añade la authority ROLE_USER.

new InMemoryUserDetailsManager(...) crea el manager con los dos usuarios.

Añade los imports:

```
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;

import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
```

Pregunta: ¿Qué authorities tiene el usuario admin? ¿Y el usuario ana?

Implementación acumulativa M6 — actualizada

Creamos una configuración explícitamente temporal:

src/main/java/es/mecd/demo/miproyecto/config/InMemoryUserConfig.java

```
@Configuration
public class InMemoryUserConfig {

    @Bean
    UserDetailsService userDetailsService(PasswordEncoder passwordEncoder) {
        UserDetails ana = User.builder()
            .username("ana")
            .password(passwordEncoder.encode("ana123"))
            .roles("USER")
            .build();

        UserDetails admin = User.builder()
            .username("admin")
```

```

        .password(passwordEncoder.encode("admin123"))
        .roles("ADMIN", "USER")
        .build();

    return new InMemoryUserDetailsManager(ana, admin);
}
}

```

Estado temporal T6.2-A — INTRODUCE. A partir de aquí dejamos de depender del usuario aleatorio de desarrollo generado por Boot y controlamos las identidades didácticas.

Observable. ana tiene ROLE_USER; admin tiene ROLE_ADMIN y ROLE_USER.

Error posible. Escribir `.roles("ROLE_ADMIN")`. **Corrección.** Con `roles(...)` se usa "ADMIN"; Spring añade ROLE_.

Pregunta. ¿Y el usuario ana necesita el rol ADMIN para consultar un recurso ordinario?

Paso 6.2.5 — Cerrar los endpoints

Fuente: p. 1256.

Base de la fuente — preservada

Modifica el SecurityFilterChain para que los endpoints requieran autenticación:

```

@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/v1/public/**").permitAll()
            .requestMatchers("/h2-console/**").permitAll()
            .requestMatchers("/swagger-ui/**",
                ↪ "/v3/api-docs/**").permitAll()
            .requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
            .anyRequest().authenticated()
        )
        .csrf(csrf -> csrf.disable())
        .httpBasic(Customizer.withDefaults());
    return http.build();
}

```

Reglas:

- `/api/v1/public/**`: acceso libre.
- `/h2-console/**`: acceso libre (consola de H2).
- `/swagger-ui/**` y `/v3/api-docs/**`: acceso libre (documentación).
- `/api/v1/admin/**`: requiere rol ADMIN.
- `anyRequest().authenticated()`: el resto requiere autenticación.

Pregunta: ¿Qué pasa si un usuario con rol USER intenta acceder a `/api/v1/admin/...`?

Implementación acumulativa M6 — actualizada

Evolucionamos el SecurityFilterChain:

http

```
.csrf(AbstractHttpConfigurer::disable)
.cors(Customizer.withDefaults())
.headers(headers -> headers.frameOptions(frame -> frame.sameOrigin()))
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/api/v1/public/**").permitAll()
    .requestMatchers("/h2-console/**").permitAll()
    .requestMatchers(
        "/swagger-ui/**",
        "/swagger-ui.html",
        "/v3/api-docs/**").permitAll()
    .requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
    .requestMatchers("/api/v1/gestor/**")
        .hasAnyRole("GESTOR", "ADMIN")
    .anyRequest().authenticated())
.httpBasic(Customizer.withDefaults());
```

Adaptación acumulativa respecto a la fuente.

- Conservamos cors(Customizer.withDefaults()) porque M5 ya tiene CORS funcional.
- sameOrigin() permite que la consola H2 pueda mostrarse dentro de frame en el estado de desarrollo.
- No añadimos permitAll() global.
- Swagger/OpenAPI sigue accesible para poder inspeccionar el contrato del API durante el curso.

Importante. H2 público es una comodidad del entorno pedagógico. No debe extrapolarse sin más a producción.

Pregunta. ¿Qué pasaría si anyRequest() apareciera antes que las reglas concretas?

Paso 6.2.6 — Arrancar y probar sin autenticación

Fuente: p. 1257.

Base de la fuente — preservada

Reinicia la aplicación. Prueba:

Endpoint protegido sin credenciales

```
curl -i http://localhost:8080/api/v1/alumnos
```

Endpoint público sin credenciales

```
curl -i http://localhost:8080/api/v1/public/info
```

El primero devolverá 401 con la cabecera WWW-Authenticate: Basic. El segundo devolverá 200.

Pregunta: ¿Por qué un endpoint devuelve 401 y el otro 200?

Implementación acumulativa M6 — actualizada

Una vez materializado el proyecto acumulativo completo:

```
./mvnw spring-boot:run
```

En otra terminal:

```
curl -i http://localhost:8080/api/v1/alumnos
```

```
curl -i http://localhost:8080/api/v1/public/info
```

Observables.

- /api/v1/alumnos → 401 Unauthorized y, en el estado Basic, cabecera WWW-Authenticate.
- /api/v1/public/info → 200 OK.

Diagnóstico. Si ambos devuelven 200, una regla demasiado amplia ha abierto el API. Si ambos devuelven 401, la excepción pública no se está aplicando.

Pregunta. ¿Por qué el mismo SecurityFilterChain puede producir 200 y 401 según la URL?

Paso 6.2.7 — Autenticarse con el usuario ana

Fuente: p. 1258.

Base de la fuente — preservada

Prueba con el usuario ana y su contraseña:

```
curl -i -u ana:ana123 http://localhost:8080/api/v1/alumnos
```

Verás la lista de alumnos con código 200. El usuario ana está autenticado y tiene rol USER.

Prueba ahora el endpoint de perfil:

```
curl -i -u ana:ana123 http://localhost:8080/api/v1/perfil
```

Verás algo como:

```
{
  "usuario": "ana",
  "authorities": ["ROLE_USER"]
}
```

Pregunta: ¿Qué authorities tiene el usuario ana según la respuesta?

Implementación acumulativa M6 — actualizada

```
curl -i -u ana:ana123 http://localhost:8080/api/v1/alumnos
```

```
curl -i -u ana:ana123 http://localhost:8080/api/v1/perfil
```

Observable. La consulta ordinaria debe estar autorizada y el perfil debe mostrar:

```
{
  "usuario": "ana",
  "authorities": ["ROLE_USER"]
}
```

El orden de una colección de authorities no debe usarse como contrato si hay varias; comprobaremos contenido, no una ordenación accidental.

Pregunta. ¿Qué authority tiene ana y de dónde proviene el prefijo ROLE_?

Paso 6.2.8 — Autenticarse con el usuario admin

Fuente: p. 1259.

Base de la fuente — preservada

Prueba con el usuario admin:

```
curl -i -u admin:admin123 http://localhost:8080/api/v1/perfil
```

Verás:

```
{
  "usuario": "admin",
  "authorities": ["ROLE_ADMIN", "ROLE_USER"]
}
```

El usuario admin tiene dos roles: ADMIN y USER.

Pregunta: ¿Por qué el usuario admin tiene también ROLE_USER? ¿Es necesario?

Implementación acumulativa M6 — actualizada

```
curl -i -u admin:admin123 http://localhost:8080/api/v1/perfil
```

Observable. El perfil contiene ROLE_ADMIN y ROLE_USER.

Por qué conserva también USER. En este diseño didáctico ADMIN puede ejercer las capacidades ordinarias de USER además de las administrativas. No es una regla intrínseca de Spring Security; es una decisión de roles del curso.

Pregunta. ¿Sería obligatorio que todo ADMIN tuviera también USER? No: depende del modelo de autorización.

Paso 6.2.9 — Probar la autorización por rol

Fuente: p. 1259.

Base de la fuente — preservada

Prueba un endpoint de admin con el usuario ana:

```
curl -i -u ana:ana123 http://localhost:8080/api/v1/admin/usuarios
```

Verás un 403 Forbidden. El usuario ana está autenticado, pero no tiene el rol ADMIN.

Ahora con el usuario admin:

```
curl -i -u admin:admin123 http://localhost:8080/api/v1/admin/usuarios
```

Si el endpoint existe, verás un 200 o 404, pero no un 403. El usuario admin está autorizado.

Como el endpoint /api/v1/admin/usuarios no existe en nuestro proyecto, verás un 404, pero no un 403. La diferencia entre 401, 403 y 404 es importante.

Pregunta: ¿Qué diferencia hay entre un 403 y un 404 en este caso?

Implementación acumulativa M6 — actualizada

Primero con un usuario autenticado pero no autorizado:

```
curl -i -u ana:ana123 http://localhost:8080/api/v1/admin/usuarios
```

Observable: 403 Forbidden.

Luego:

```
curl -i -u admin:admin123 http://localhost:8080/api/v1/admin/usuarios
```

En este momento el endpoint administrativo aún no existe. El usuario admin supera la autorización y la resolución del recurso puede terminar en 404 Not Found. Lo importante de este paso es que **ya no es 403**.

Diferencia observable.

- 401: no existe una autenticación válida.
- 403: existe identidad, pero carece del permiso.

- 404: la seguridad permitió continuar, pero no hay recurso/mapping coincidente.

Pregunta. ¿Por qué un 404 después de autenticarse como admin es una señal distinta a un 403?

Paso 6.2.10 — Probar con credenciales incorrectas

Fuente: p. 1260.

Base de la fuente — preservada

Prueba con una contraseña incorrecta:

```
curl -i -u ana:contraseñaIncorrecta http://localhost:8080/api/v1/alumnos
```

Verás un 401. El usuario existe, pero la contraseña no coincide.

Prueba con un usuario que no existe:

```
curl -i -u pedro:loQueSea http://localhost:8080/api/v1/alumnos
```

También un 401. El usuario no existe. Spring Security no distingue entre “usuario no existe” y “contraseña incorrecta” en la respuesta: en ambos casos devuelve 401. Es una medida de seguridad para no revelar qué usuarios existen.

Pregunta: ¿Por qué Spring Security devuelve el mismo error para “usuario no existe” y “contraseña incorrecta”?

Implementación acumulativa M6 — actualizada

```
curl -i -u ana:contraseñaIncorrecta \
  http://localhost:8080/api/v1/alumnos
```

```
curl -i -u pedro:loQueSea \
  http://localhost:8080/api/v1/alumnos
```

Observable: ambos casos producen 401.

No devolvemos “el usuario existe pero la contraseña falla” frente a “el usuario no existe”. Dar respuestas diferentes facilitaría enumerar cuentas válidas.

Pregunta. ¿Qué información podría obtener un atacante si la API distinguiera públicamente ambos casos?

Paso 6.2.11 — Errores comunes del ejercicio

Fuente: p. 1229.

Base de la fuente — preservada

Error Causa Solución

Credenciales mal enviadas o 401 siempre Verificar el -u en curl usuario no existe

El usuario no tiene el rol 403 con usuario autenticado Revisar .roles(...) necesario

No se ha cifrado con La contraseña no coincide Usar passwordEncoder.encode(...) el PasswordEncoder

Error Causa Solución

NoSuchBeanDefinitionException: Falta el bean Añadirlo PasswordEncoder

Dependencia circular entre Ciclo infinito al arrancar Revisar la inyección beans

Falta el import de alguna La aplicación no arranca Añadirlo clase

El nivel de log no está en Los logs no muestran autenticación Activarlo DEBUG

UsernameNotFoundException no se El usuario no se busca Revisar UserDetailsService lanza correctamente

Implementación acumulativa M6 — actualizada

Error	Causa probable	Corrección
401 siempre	credenciales mal enviadas o usuario inexistente	revisar <code>curl -u</code> y <code>UserDetailsService</code>
403 con usuario autenticado	no posee el rol necesario	revisar <code>.roles(...)</code> y la regla de URL
la contraseña no coincide	no se codificó con el mismo <code>PasswordEncoder</code>	crearla con <code>passwordEncoder.encode(...)</code>
<code>NoSuchBeanDefinitionException</code>	falta el bean	declarar el <code>@Bean</code> estable
<code>PasswordEncoder</code> ciclo al arrancar	dependencia circular entre configuración/beans	separar responsabilidades e inyección
la aplicación no compila	import equivocado o ausente	usar imports de Spring Security actuales
no aparecen logs de seguridad	nivel DEBUG no activado	activarlo sólo para diagnóstico
búsqueda de usuario incorrecta	<code>UserDetailsService</code> no resuelve el username esperado	revisar implementación/datos
CORS preflight falla	Security intercepta OPTIONS sin integrar CORS	conservar <code>.cors(...)</code> y probar preflight
se usa <code>NoOpPasswordEncoder</code>	atajo inseguro	eliminarlo y usar el encoder del curso

Comprobación de cierre. No convertir un error de test en “solución” eliminando el test heredado.

Paso 6.2.12 — Reto resuelto: tercer usuario con roles combinados

Fuente: p. 1262.

Base de la fuente — preservada

Reto: Añadir un usuario gestor con los roles GESTOR y USER, y proteger un endpoint para que solo sea accesible por GESTOR o ADMIN.

Solución paso a paso:

Paso 1: Añadir el usuario al `UserDetailsService`:

```
UserDetails gestor = User.builder()
    .username("gestor")
    .password(passwordEncoder.encode("gestor123"))
    .roles("GESTOR", "USER")
    .build();

return new InMemoryUserDetailsManager(ana, admin, gestor);
```

Paso 2: Añadir un endpoint protegido por ambos roles:

```
@GetMapping("/api/v1/gestor/documentos")
public Map<String, String> documentos() {
    return Map.of("mensaje", "Lista de documentos");
}
```

Paso 3: Configurar la autorización:

```
.requestMatchers("/api/v1/gestor/**").hasAnyRole("GESTOR", "ADMIN")
```

`hasAnyRole(...)` permite el acceso si el usuario tiene al menos uno de los roles indicados. El usuario gestor tiene `ROLE_GESTOR`, y el usuario admin tiene `ROLE_ADMIN`. Los dos pueden acceder.

Paso 4: Probar:

Ana (USER) → 403

```
curl -i -u ana:anal23 http://localhost:8080/api/v1/gestor/documentos
```

Gestor → 200

```
curl -i -u gestor:gestor123 http://localhost:8080/api/v1/gestor/documentos
```

Admin → 200

```
curl -i -u admin:admin123 http://localhost:8080/api/v1/gestor/documentos
```

Pregunta: ¿Qué diferencia hay entre `hasRole("ADMIN")` y `hasAnyRole("GESTOR", "ADMIN")`?

Resultado esperado global

Al final del ejercicio, deberías tener:

- Un bean `PasswordEncoder` con `DelegatingPasswordEncoder`.
- Un bean `UserDetailsService` con tres usuarios: ana (USER), admin (ADMIN, USER), gestor (GESTOR, USER).
- Un `SecurityFilterChain` con reglas de autorización por endpoint.
- Un `PerfilController` que devuelve la identidad del usuario autenticado.
- La capacidad de autenticarse con curl y ver los roles en la respuesta.

Resumen técnico del ejercicio El ejercicio ha demostrado:

- `UserDetails`: contrato de usuario en Spring Security.
- `User.builder()`: construcción de usuarios.
- `InMemoryUserDetailsManager`: usuarios en memoria.
- `PasswordEncoder`: cifrado con BCrypt.
- `.roles(...)`: crea authorities con prefijo `ROLE_`.
- `hasRole` vs `hasAnyRole`: una o varias roles.
- `Authentication`: inyectado en el controlador.
- `HTTP Basic`: credenciales en cada petición.
- Diferencia 401 vs 403: no autenticado vs sin permisos.

Conclusión y enlace al siguiente punto En este punto 6.2 hemos configurado autenticación con usuarios en memoria:

- `UserDetails` y `UserDetailsService`: el modelo de usuario.

- InMemoryUserDetailsManager: usuarios en memoria. Solo para desarrollo.
- PasswordEncoder y BCrypt: cifrado de contraseñas.
- .roles(...) y .authorities(...):
- Authentication como parámetro: acceso a la identidad.
- HTTP Basic: autenticación con cabecera Authorization.
- Diferencia 401 vs 403.

En la práctica, hemos definido tres usuarios con distintos roles, hemos cifrado sus contraseñas, hemos protegido endpoints por rol, y hemos probado la autenticación con curl.

La idea clave: los usuarios en memoria son útiles para desarrollo y pruebas, pero no para producción. Persisten solo mientras la aplicación está arrancada y no permiten gestión dinámica.

En el siguiente punto, 6.3 — Usuarios en base de datos, sustituiremos el InMemoryUserDetailsManager por un UserDetailsService personalizado que carga los usuarios desde una tabla de la base de datos. Veremos cómo se modela la entidad Usuario, cómo se gestionan los roles y cómo se integra con la autenticación.

Fin del Punto 6.2.

Implementación acumulativa M6 — actualizada

El reto fuente exige un usuario gestor con GESTOR y USER, y una ruta accesible por GESTOR o ADMIN.

1. Evolucionar InMemoryUserConfig

```
UserDetails gestor = User.builder()
    .username("gestor")
    .password(passwordEncoder.encode("gestor123"))
    .roles("GESTOR", "USER")
    .build();

return new InMemoryUserDetailsManager(ana, admin, gestor);
```

2. Crear el endpoint

src/main/java/es/mecd/demo/miproyecto/auth/GestorController.java

```
@RestController
@RequestMapping("/api/v1/gestor")
public class GestorController {

    @GetMapping("/documentos")
    public Map<String, String> documentos() {
        return Map.of("mensaje", "Lista de documentos");
    }
}
```

3. Autorizar cualquiera de los dos roles

```
.requestMatchers("/api/v1/gestor/**")
.hasAnyRole("GESTOR", "ADMIN")
```

4. Probar

```
# Ana (USER) -> 403
curl -i -u ana:anal23 \
  http://localhost:8080/api/v1/gestor/documentos

# Gestor -> 200
curl -i -u gestor:gestor123 \
  http://localhost:8080/api/v1/gestor/documentos

# Admin -> 200
curl -i -u admin:admin123 \
  http://localhost:8080/api/v1/gestor/documentos
```

Pregunta. ¿Qué diferencia existe entre `hasRole("ADMIN")` y `hasAnyRole("GESTOR", "ADMIN")`?

Resultado esperado global de 6.2

Al cerrar el punto deben existir:

- PasswordEncoder basado en DelegatingPasswordEncoder;
- UserDetailsService temporal con ana, admin y gestor;
- reglas de autorización por endpoint;
- PerfilController;
- endpoint de gestor del reto;
- autenticación Basic verificable con curl;
- diferencias 401/403 demostradas;
- CORS heredado de M5 preservado.

Restauraciones/evoluciones registradas

- **T6.2-A — usuarios en memoria:** permanece activo únicamente hasta **6.3.6**, donde se elimina al sustituirlo por UserDetailsService de base de datos.
- **T6.2-B — HTTP Basic:** continúa para observar el mecanismo durante 6.3–6.6 y se **desactiva en 6.7.4** al entrar en funcionamiento el filtro JWT.
- El PasswordEncoder **no se elimina:** se conserva para registrar/verificar contraseñas persistentes.

6.3 — Usuarios en base de datos

Este punto **cierra** los usuarios en memoria de 6.2. HTTP Basic continúa sólo como mecanismo pedagógico de transporte de credenciales hasta que 6.7 lo sustituya por JWT.

Práctica 6.3 — Usuarios en base de datos

Preparación y contexto de la fuente

Contexto del ejercicio: Vamos a sustituir los usuarios en memoria por usuarios en base de datos. Crearemos las entidades Usuario y Rol, sus repositorios, un UserDetailsService personalizado, un endpoint de registro público, y verificaremos que la autenticación y la autorización funcionan con los usuarios de la BD.

Requisitos previos: Tener el proyecto mi-proyecto con el SecurityConfig y los usuarios en memoria del punto 6.2.

Paso 6.3.1 — Crear el paquete auth

Fuente: p. 1290.

Base de la fuente — preservada

Crea el paquete `es.mecd.demo.miproyecto.auth` para agrupar todo lo relacionado con autenticación. Dentro crearás las entidades, repositorios, servicios y controladores.

Pregunta: ¿Por qué conviene tener un paquete específico para la autenticación?

Implementación acumulativa M6 — actualizada

Qué hacemos. Consolidamos bajo:

`src/main/java/es/mecd/demo/miproyecto/auth/`

las entidades, repositorios, servicio de identidad, DTOs y operaciones de autenticación.

El `GestorController` creado en el reto 6.2 ya está en este paquete y se conserva.

Por qué. La organización por funcionalidad sigue el patrón de M5: los elementos de una misma capacidad permanecen juntos.

Observable. El paquete `auth` es una evolución acumulativa, no un proyecto paralelo.

Paso 6.3.2 — Crear la entidad Rol

Fuente: p. 1290.

Base de la fuente — preservada

Crea la clase `Rol`:

```
package es.mecd.demo.miproyecto.auth;

import jakarta.persistence.*;

import java.util.HashSet;
import java.util.Set;

@Entity
@Table(name = "roles")
public class Rol {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 50)
    private String nombre;

    @Column(length = 200)
    private String descripcion;

    @ManyToMany(mappedBy = "roles")
    private Set<Usuario> usuarios = new HashSet<>();

    public Rol() {
```

```

    }

    public Rol(String nombre, String descripcion) {
        this.nombre = nombre;
        this.descripcion = descripcion;
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }

    public String getDescripcion() { return descripcion; }
    public void setDescripcion(String descripcion) { this.descripcion =
        ↪ descripcion; }

    public Set<Usuario> getUsuarios() { return usuarios; }
    public void setUsuarios(Set<Usuario> usuarios) { this.usuarios = usuarios; }
}

```

`@ManyToMany(mappedBy = "roles")` indica que el propietario es el campo roles de `Usuario`. `Rol` no define la tabla intermedia.

Pregunta: ¿Qué pasaría si Rol también tuviera `@JoinTable`?

Implementación acumulativa M6 — actualizada

Archivo:

src/main/java/es/mecd/demo/miproyecto/auth/Rol.java

```

@Entity
@Table(name = "roles")
public class Rol {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 50)
    private String nombre;

    @Column(length = 200)
    private String descripcion;

    @ManyToMany(mappedBy = "roles")
    private Set<Usuario> usuarios = new HashSet<>();

    protected Rol() {
    }

    public Rol(String nombre, String descripcion) {
        this.nombre = nombre;
        this.descripcion = descripcion;
    }

    // getters/setters

```

```
}
```

Observable. Rol no define @JoinTable; es el lado inverso.

Error posible. Definir @JoinTable también aquí. **Corrección.** Sólo Usuario.roles es propietario.

Pregunta. ¿Qué ocurriría si ambos lados intentasen crear su propia tabla de unión?

Paso 6.3.3 — Crear la entidad Usuario

Fuente: p. 1292.

Base de la fuente — preservada

Crea la clase Usuario:

```
package es.mecd.demo.miproyecto.auth;

import jakarta.persistence.*;

import java.time.LocalDateTime;
import java.util.HashSet;
import java.util.Set;

@Entity
@Table(name = "usuarios")
public class Usuario {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 50)
    private String username;

    @Column(nullable = false)
    private String password;

    @Column(nullable = false, unique = true, length = 150)
    private String email;

    @Column(nullable = false)
    private boolean activo = true;

    @Column(name = "fecha_creacion")
    private LocalDateTime fechaCreacion = LocalDateTime.now();

    @ManyToMany(fetch = FetchType.EAGER)
    @JoinTable(
        name = "usuarios_rols",
        joinColumns = @JoinColumn(name = "usuario_id"),
        inverseJoinColumns = @JoinColumn(name = "rol_id")
    )
    private Set<Rol> roles = new HashSet<>();

    public Usuario() {
    }
}
```

```
    // getters y setters
}
```

`@JoinTable` define la tabla intermedia usuarios_roles con las columnas usuario_id y rol_id.

fetch = FetchType.EAGER carga los roles al cargar el usuario. Spring Security los necesita en la autenticación.

Pregunta: ¿Por qué EAGER en los roles y no LAZY?

Implementación acumulativa M6 — actualizada

Archivo:

src/main/java/es/mecd/demo/miproyecto/auth/Usuario.java

Campos:

- id;
- username único;
- password codificada;
- email único;
- activo;
- fechaCreacion;
- roles.

Relación:

```
@ManyToMany(fetch = FetchType.EAGER)
@JoinTable(
    name = "usuarios_roles",
    joinColumns = @JoinColumn(name = "usuario_id"),
    inverseJoinColumns = @JoinColumn(name = "rol_id"))
private Set<Rol> roles = new HashSet<>();
```

Observable. Hibernate podrá construir tres tablas: usuarios, roles y usuarios_roles.

Nota técnica. EAGER se mantiene porque este modelo necesita authorities durante autenticación. En un dominio más pesado evaluaríamos LAZY + consulta explícita.

Pregunta. ¿Por qué Set encaja mejor que List para roles?

Paso 6.3.4 — Crear los repositorios

Fuente: p. 1295.

Base de la fuente — preservada

Crea RolRepository:

```
package es.mecd.demo.miproyecto.auth;

import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;

public interface RolRepository extends JpaRepository<Rol, Long> {

    Optional<Rol> findByNombre(String nombre);
```

```
        boolean existsByNombre(String nombre);
    }
```

Crea UsuarioRepository:

```
package es.mecd.demo.miproyecto.auth;

import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;

public interface UsuarioRepository extends JpaRepository<Usuario, Long> {

    Optional<Usuario> findByUsername(String username);

    boolean existsByUsername(String username);

    boolean existsByEmail(String email);
}
```

Pregunta: ¿Por qué hay métodos específicos para buscar por username y por email?

Implementación acumulativa M6 — actualizada

RolRepository:

```
public interface RolRepository extends JpaRepository<Rol, Long> {
    Optional<Rol> findByNombre(String nombre);
    boolean existsByNombre(String nombre);
}
```

UsuarioRepository:

```
public interface UsuarioRepository extends JpaRepository<Usuario, Long> {
    Optional<Usuario> findByUsername(String username);
    boolean existsByUsername(String username);
    boolean existsByEmail(String email);
}
```

Por qué. Spring Data deriva las consultas a partir de sus nombres y mantiene los detalles SQL fuera de la capa de seguridad.

Observable. Podemos localizar una identidad por el mismo username que recibe Spring Security y validar duplicados antes de guardar.

Paso 6.3.5 — Crear el UserDetailsService personalizado

Fuente: p. 1296.

Base de la fuente — preservada

Crea UsuarioDetailsService:

```
package es.mecd.demo.miproyecto.auth;

import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;

import org.springframework.security.core.userdetails.User;
```

```

import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

@Service
public class UsuarioDetailsService implements UserDetailsService {

    private final UsuarioRepository usuarioRepository;

    public UsuarioDetailsService(UsuarioRepository usuarioRepository) {
        this.usuarioRepository = usuarioRepository;
    }

    @Override
    @Transactional(readOnly = true)
    public UserDetails loadUserByUsername(String username) throws
        UsernameNotFoundException {
        Usuario usuario = usuarioRepository.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException(
                "Usuario no encontrado: " + username));

        List<GrantedAuthority> authorities = usuario.getRoles().stream()
            .map(rol -> (GrantedAuthority) new SimpleGrantedAuthority("ROLE_" +
                rol.getNombre()))
            .toList();

        return User.builder()
            .username(usuario.getUsername())
            .password(usuario.getPassword())
            .authorities(authorities)
            .accountExpired(false)
            .accountLocked(!usuario.isActivo())
            .credentialsExpired(false)
            .disabled(!usuario.isActivo())
            .build();
    }
}

```

`@Service` hace que **Spring Security** lo detecte como `UserDetailsService`.

`@Transactional(readOnly = true)` mantiene la sesión abierta para cargar los roles.

"ROLE_" + rol.getNombre() añade el prefijo `ROLE_` a cada rol.

Pregunta: ¿Qué authorities tendría un usuario con el rol ADMIN?

Implementación acumulativa M6 — actualizada

Creamos `UsuarioDetailsService`.

```

@Service
public class UsuarioDetailsService implements UserDetailsService {

    private final UsuarioRepository usuarioRepository;

```

```

public UsuarioDetailsService(UsuarioRepository usuarioRepository) {
    this.usuarioRepository = usuarioRepository;
}

@Override
@Transactional(readOnly = true)
public UserDetails loadUserByUsername(String username) {
    Usuario usuario = usuarioRepository.findByUsername(username)
        .orElseThrow(() -> new UsernameNotFoundException(
            "Usuario no encontrado: " + username));

    List<GrantedAuthority> authorities = usuario.getRoles().stream()
        .map(rol -> (GrantedAuthority)
            new SimpleGrantedAuthority("ROLE_" + rol.getNombre()))
        .toList();

    return User.builder()
        .username(usuario.getUsername())
        .password(usuario.getPassword())
        .authorities(authorities)
        .accountExpired(false)
        .accountLocked(!usuario.isActivo())
        .credentialsExpired(false)
        .disabled(!usuario.isActivo())
        .build();
}
}

```

Observable. Un rol persistido como ADMIN se expone a Spring Security como ROLE_ADMIN.

Error posible. Guardar ROLE_ADMIN y volver a prefijarlo. **Corrección.** En nuestra base se guardan nombres sin ROLE_.

Paso 6.3.6 — Eliminar los usuarios en memoria

Fuente: p. 1299.

Base de la fuente — preservada

Abre el SecurityConfig y elimina el bean UserDetailsService que creaba el InMemoryUserDetailsManager del punto 6.2. Si no lo eliminas, habrá dos beans del mismo tipo y Spring Security fallará al arrancar.

Deja solo el bean PasswordEncoder y el bean SecurityFilterChain.

Pregunta: ¿Por qué no puede haber dos beans UserDetailsService?

Implementación acumulativa M6 — actualizada

Estado T6.2-A — CLOSE.

Eliminamos físicamente:

config/InMemoryUserConfig.java

No se elimina:

```

@Bean
PasswordEncoder passwordEncoder()

```

porque la autenticación persistente y el registro siguen necesitando el encoder.

Gate negativo. El proyecto final del paso no puede contener:

InMemoryUserDetailsManager

en código de producción.

Por qué. Dos fuentes de usuario compitiendo harían ambiguo el mecanismo de autenticación y, sobre todo, dejarían vivo un estado pedagógico que 6.3 está diseñado para sustituir.

Paso 6.3.7 — Crear el inicializador de usuarios

Fuente: p. 1299.

Base de la fuente — preservada

Crea UsuariosInicialesConfig:

```
package es.mecd.demo.miproyecto.auth;

import org.springframework.boot.CommandLineRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.crypto.password.PasswordEncoder;

@Configuration
public class UsuariosInicialesConfig {

    @Bean
    public CommandLineRunner cargarUsuarios(
        UsuarioRepository usuarioRepository,
        RolRepository rolRepository,
        PasswordEncoder passwordEncoder) {
        return args -> {
            if (usuarioRepository.count() > 0) {
                return;
            }

            Rol rolAdmin = rolRepository.save(new Rol("ADMIN", "Administrador"));
            Rol rolUser = rolRepository.save(new Rol("USER", "Usuario"));

            Usuario admin = new Usuario();
            admin.setUsername("admin");
            admin.setPassword(passwordEncoder.encode("admin123"));
            admin.setEmail("admin@educacion.gob.es");
            admin.setActivo(true);
            admin.getRoles().add(rolAdmin);
            admin.getRoles().add(rolUser);
            usuarioRepository.save(admin);

            Usuario ana = new Usuario();
            ana.setUsername("ana");

            ana.setPassword(passwordEncoder.encode("ana123"));
            ana.setEmail("ana@educacion.gob.es");
            ana.setActivo(true);
            ana.getRoles().add(rolUser);
```

```

        usuarioRepository.save(ana);
    };
}

```

Pregunta: ¿Qué pasa si reinicias la aplicación? ¿Se duplican los usuarios?

Implementación acumulativa M6 — actualizada

Creamos UsuariosInicialesConfig.

La fuente utiliza un guard global `count() > 0`. Conservamos la explicación, pero el proyecto acumulativo mejora el algoritmo a **find-or-create** para permitir evolución sin duplicados.

Roles materializados:

ADMIN
USER
GESTOR

Usuarios:

admin -> ADMIN + USER
ana -> USER
gestor -> GESTOR + USER

gestor es una continuidad deliberada del reto ya completado en 6.2.

Observable. Reiniciar no crea duplicados y tampoco hace desaparecer el reto anterior.

Error que evita la mejora. Una base que ya contenga admin y ana no impediría añadir más adelante GESTOR.

Paso 6.3.8 — Arrancar y verificar

Fuente: p. 1301.

Base de la fuente — preservada

Reinicia la aplicación. En los logs verás que Hibernate crea las tablas usuarios, roles y usuarios_roles. Después, el CommandLineRunner inserta los usuarios.

Abre la consola de H2 y ejecuta:

```

SELECT * FROM usuarios;
SELECT * FROM roles;
SELECT * FROM usuarios_roles;

```

Verás los usuarios, los roles y la tabla intermedia.

Pregunta: ¿Qué información contiene la tabla usuarios_roles?

Implementación acumulativa M6 — actualizada

Cuando el árbol M5 esté completamente materializado:

```
./mvnw spring-boot:run
```

En H2:

```

SELECT * FROM usuarios;
SELECT * FROM roles;
SELECT * FROM usuarios_roles;

```

Observable.

- usuarios contiene identidades persistentes;
- roles contiene nombres de rol;
- usuarios_roles contiene pares usuario_id/rol_id.

Pregunta. ¿Por qué la tabla de unión no necesita guardar username ni nombre del rol?

Paso 6.3.9 — Probar la autenticación

Fuente: p. 1302.

Base de la fuente — preservada

Prueba a autenticarte con los usuarios de la base de datos:

```
# Ana (rol USER)
curl -i -u ana:ana123 http://localhost:8080/api/v1/perfil
```

```
# Admin (roles ADMIN, USER)
curl -i -u admin:admin123 http://localhost:8080/api/v1/perfil
```

Verás que las credenciales funcionan. El UsuarioDetailsService carga los usuarios de la base de datos.

Pregunta: ¿Qué authorities devuelve el endpoint /api/v1/perfil para cada usuario?

Implementación acumulativa M6 — actualizada

```
curl -i -u ana:ana123 \
  http://localhost:8080/api/v1/perfil

curl -i -u admin:admin123 \
  http://localhost:8080/api/v1/perfil

curl -i -u gestor:gestor123 \
  http://localhost:8080/api/v1/gestor/documentos
```

Observables.

- ana → ROLE_USER;
- admin → ROLE_ADMIN, ROLE_USER;
- gestor → supera la ruta GESTOR.

Evidencia conceptual. Las mismas credenciales de 6.2 siguen funcionando, pero ya no proceden de InMemoryUserDetailsManager.

Paso 6.3.10 — Crear los DTOs de registro

Fuente: p. 1302.

Base de la fuente — preservada

Crea los DTOs de registro:

```
package es.mecd.demo.miproyecto.auth;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;
```

```

public class RegistroRequestDTO {

    @NotBlank(message = "El nombre de usuario es obligatorio")
    @Size(min = 3, max = 50, message = "El nombre de usuario debe tener entre 3 y
    ↪ 50 caracteres")
    private String username;

    @NotBlank(message = "La contraseña es obligatoria")
    @Size(min = 8, message = "La contraseña debe tener al menos 8 caracteres")
    private String password;

    @NotBlank(message = "El email es obligatorio")
    @Email(message = "El email no tiene un formato válido")
    private String email;

    // getters y setters
}
java
package es.mecd.demo.miproyecto.auth;

import com.fasterxml.jackson.annotation.JsonProperty;

import java.util.List;

public class UsuarioResponseDTO {

    @JsonProperty("id")
    private String identificador;

    private String username;
    private String email;
    private List<String> roles;

    // getters y setters
}

```

Pregunta: ¿Por qué el DTO de respuesta no tiene campo password?

Implementación acumulativa M6 — actualizada

RegistroRequestDTO:

```

@NotBlank
@Size(min = 3, max = 50)
private String username;

@NotBlank
@Size(min = 8)
private String password;

@NotBlank
@email
private String email;

```

UsuarioResponseDTO contiene:

id

username
email
roles

y **no contiene password**.

Pregunta. ¿Por qué no devolvemos siquiera el hash de la contraseña?

Paso 6.3.11 — Crear AuthService y AuthController

Fuente: p. 1304.

Base de la fuente — preservada

Crea AuthService:

```
package es.mecd.demo.miproyecto.auth;

import es.mecd.demo.miproyecto.common.exception.RecursoDuplicadoException;
import es.mecd.demo.miproyecto.common.exception.RecursoNoEncontradoException;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class AuthService {

    private final UsuarioRepository usuarioRepository;
    private final RolRepository rolRepository;
    private final PasswordEncoder passwordEncoder;

    public AuthService(UsuarioRepository usuarioRepository,
                      RolRepository rolRepository,
                      PasswordEncoder passwordEncoder) {
        this.usuarioRepository = usuarioRepository;
        this.rolRepository = rolRepository;
        this.passwordEncoder = passwordEncoder;
    }

    @Transactional

    public UsuarioResponseDTO registrar(RegistroRequestDTO request) {
        if (usuarioRepository.existsByUsername(request.getUsername())) {
            throw new RecursoDuplicadoException("Usuario", "username",
                ↪ request.getUsername());
        }
        if (usuarioRepository.existsByEmail(request.getEmail())) {
            throw new RecursoDuplicadoException("Usuario", "email", request.getEmail());
        }

        Rol rolUser = rolRepository.findByNombre("USER")
            .orElseThrow(() -> new RecursoNoEncontradoException("Rol", "USER"));

        Usuario usuario = new Usuario();
        usuario.setUsername(request.getUsername());
        usuario.setPassword(passwordEncoder.encode(request.getPassword()));
        usuario.setEmail(request.getEmail());
    }
}
```

```

        usuario.setActivo(true);
        usuario.getRoles().add(rolUser);

        return toDTO(usuarioRepository.save(usuario));
    }

    private UsuarioResponseDTO toDTO(Usuario usuario) {
        UsuarioResponseDTO dto = new UsuarioResponseDTO();
        dto.setIdentificador(usuario.getId() != null ? usuario.getId().toString() :
        ↪ null);
        dto.setUsername(usuario.getUsername());

        dto.setEmail(usuario.getEmail());
        dto.setRoles(usuario.getRoles().stream()
            .map(Rol::getNombre)
            .toList());
        return dto;
    }
}

```

Crea AuthController:

```

package es.mecd.demo.miproyecto.auth;

import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/v1/auth")
public class AuthController {

    private final AuthService authService;

    public AuthController(AuthService authService) {
        this.authService = authService;
    }

    @PostMapping("/registro")
    public ResponseEntity<UsuarioResponseDTO> registrar(
        @Valid @RequestBody RegistroRequestDTO request) {
        UsuarioResponseDTO usuario = authService.registrar(request);
        return ResponseEntity.status(HttpStatus.CREATED).body(usuario);
    }
}

```

Pregunta: ¿Por qué el endpoint de registro debe ser público?

Implementación acumulativa M6 — actualizada

AuthService.registrar(...):

1. rechaza username repetido;
2. rechaza email repetido;

3. exige que exista USER;
4. codifica contraseña;
5. asigna USER;
6. guarda;
7. proyecta a DTO.

El controlador expone:

POST /api/v1/auth/registro

y responde 201 Created.

Continuidad con M5. Se reutilizan las excepciones específicas ya existentes:

- `RecursoDuplicadoException`;
- `RecursoNoEncontradoException`;
- `OperacionNoPermitidaException` para el reto de contraseña.

No se vuelve a introducir la clase genérica `NegocioException`, que M5 conserva sólo como compatibilidad y marca `@Deprecated`.

Paso 6.3.12 — Configurar el endpoint público y probar

Fuente: p. 1308.

Base de la fuente — preservada

Asegúrate de que el `SecurityFilterChain` permite el acceso a `/api/v1/auth/**`:

```
.requestMatchers("/api/v1/auth/**").permitAll()
```

Prueba a registrar un usuario:

```
curl -i -X POST http://localhost:8080/api/v1/auth/registro \
-H "Content-Type: application/json" \
-d '{
  "username": "pedro",
  "password": "pedro1234",
  "email": "pedro@educacion.gob.es"
}'
```

Verás un 201 con el usuario creado (sin contraseña). Ahora autentícate con él:

```
curl -i -u pedro:pedro1234 http://localhost:8080/api/v1/perfil
```

Verás que el usuario pedro está autenticado con el rol USER.

Pregunta: ¿Qué pasa si intentas registrar un usuario con el mismo username que uno existente?

Implementación acumulativa M6 — actualizada

En `SecurityConfig` usamos un matcher mínimo:

```
.requestMatchers(HttpMethod.POST, "/api/v1/auth/registro").permitAll()
```

No abrimos todavía todo `/api/v1/auth/**`.

Prueba:

```
curl -i -X POST \
  http://localhost:8080/api/v1/auth/registro \
-H "Content-Type: application/json" \
-d '{
  "username": "pedro",
```

```
"password": "pedro1234",
"email": "pedro@educacion.gob.es"
}'
```

Observable: 201 Created, sin contraseña en la respuesta.

Después:

```
curl -i -u pedro:pedro1234 \
http://localhost:8080/api/v1/perfil
```

Debe mostrar ROLE_USER.

Repite el registro con el mismo username o email.

Observable: conflicto de negocio manejado por el sistema de errores heredado de M5.

Paso 6.3.13 — Errores comunes del ejercicio

Fuente: p. 1309.

Base de la fuente — preservada

Error Causa Solución

Dos UserDetailsService o falta NoSuchBeanDefinitionException Dejar solo uno uno

LazyInitializationException Falta @Transactional o EAGER Añadirlos

DataIntegrityViolationException al Username o email duplicado Validar antes registrar

Contraseña en texto plano No se llamó a encode Añadirlo

Añadir prefijo al hasRole no funciona Los roles no tienen prefijo ROLE_ transformar

El endpoint de registro devuelve 401 No es público Añadir permitAll

Dos beans que se inyectan Ciclo infinito al arrancar Revisar la inyección mutuamente

La tabla intermedia no se crea Falta @JoinTable Añadirlo

Error Causa Solución

No se modifica la colección del Añadir el rol El registro no guarda los roles usuario a usuario.getRoles()

Implementación acumulativa M6 — actualizada

Error	Causa	Corrección
más de un UserDetailsService	no se cerró 6.2	eliminar InMemoryUserConfig
LazyInitializationException	roles fuera del contexto de persistencia	EAGER o carga explícita dentro de transacción
duplicado al registrar	username/email existentes	validar en servicio + constraint DB
contraseña visible en BD	faltó encode	codificar antes de persistir
hasRole siempre falla	authorities sin ROLE_	transformar ADMIN → ROLE_ADMIN
registro devuelve 401	matcher no público	permitir sólo POST
ciclo de beans	configuración acoplada	/api/v1/auth/registro
		constructor injection y responsabilidades separadas

Error	Causa	Corrección
no aparece tabla intermedia	mapping incompleto	revisar @JoinTable
roles no guardados	se modificó el lado incorrecto	añadir al Set de Usuario
usuario inexistente produce error técnico	excepción incorrecta	lanzar UsernameNotFoundException

Gate adicional. Registrar no puede permitir que el cliente escoja ADMIN.

Paso 6.3.14 — Reto resuelto: cambiar la contraseña

Fuente: p. 1311.

Base de la fuente — preservada

Reto: Añadir un endpoint PUT /api/v1/perfil/password que permita al usuario autenticado cambiar su contraseña.

Solución paso a paso:

Paso 1: Crear el DTO:

```
public class CambioPasswordRequestDTO {

    @NotBlank(message = "La contraseña actual es obligatoria")
    private String passwordActual;

    @NotBlank(message = "La nueva contraseña es obligatoria")
    @Size(min = 8, message = "La nueva contraseña debe tener al menos 8
        ↪ caracteres")
    private String passwordNueva;

    // getters y setters
}
```

Paso 2: Añadir el método al AuthService:

```
@Transactional
public void cambiarPassword(String username, CambioPasswordRequestDTO request) {
    Usuario usuario = usuarioRepository.findByUsername(username)
        .orElseThrow(() -> new RecursoNoEncontradoException("Usuario",
            ↪ username));

    if (!passwordEncoder.matches(request.getPasswordActual(),
        ↪ usuario.getPassword())) {
        throw new NegocioException("La contraseña actual no es correcta");
    }

    usuario.setPassword(passwordEncoder.encode(request.getPasswordNueva()));
    usuarioRepository.save(usuario);
}
```

Paso 3: Añadir el endpoint al AuthController:

```
@PutMapping("/password")
public ResponseEntity<Void> cambiarPassword(
    Authentication auth,
    @Valid @RequestBody CambioPasswordRequestDTO request) {
```

```

        authService.cambiarPassword(auth.getName(), request);
        return ResponseEntity.noContent().build();
    }

```

Paso 4: Probar:

```

curl -i -X PUT -u pedro:pedro1234 \
  http://localhost:8080/api/v1/auth/password \
  -H "Content-Type: application/json" \
  -d '{
    "passwordActual": "pedro1234",
    "passwordNueva": "pedro5678"
  }'

```

Verás un 204 No Content. Prueba a autenticarte con la nueva contraseña:

```
curl -i -u pedro:pedro5678 http://localhost:8080/api/v1/perfil
```

Pregunta: ¿Por qué es importante verificar la contraseña actual antes de cambiarla?

Resultado esperado global

Al final del ejercicio, deberías tener:

- Entidades Usuario y Rol con relación @ManyToMany.
- Repositorios UsuarioRepository y RolRepository.
- Un UsuarioDetailsService que carga usuarios de la base de datos.
- Un UsuariosInicialesConfig que crea usuarios iniciales.
- Un endpoint de registro público.
- Un endpoint para cambiar la contraseña (reto).
- Autenticación y autorización funcionando con usuarios de la base de datos.

Resumen técnico del ejercicio El ejercicio ha demostrado:

- @ManyToMany: relación entre Usuario y Rol con @JoinTable.
- UserDetailsService personalizado: carga usuarios de la BD.
- @Transactional: en el UserDetailsService para cargar los roles.
- Prefijo ROLE_: al transformar roles a authorities.
- @JoinTable: tabla intermedia usuarios_rols.
- Registro: endpoint público, DTOs, contraseña cifrada, rol por defecto.
- Inicialización: CommandLineRunner.
- Cambio de contraseña: verificar la actual antes de cambiar.
- Buenas prácticas: contraseña cifrada, username único, DTOs sin contraseña.

MÓDULO 6 — Bloque 2

Implementación acumulativa M6 — actualizada

El enunciado de la fuente pide:

```
PUT /api/v1/perfil/password
```

y esa será la ruta canónica de nuestra solución.

DTO

```
public class CambioPasswordRequestDTO {

    @NotBlank
    private String passwordActual;

    @NotBlank
    @Size(min = 8)
    private String passwordNueva;
}
```

Servicio

```
@Transactional
public void cambiarPassword(
    String username,
    CambioPasswordRequestDTO request) {

    Usuario usuario = usuarioRepository.findByUsername(username)
        .orElseThrow(() ->
            new RecursoNoEncontradoException("Usuario", username));

    if (!passwordEncoder.matches(
        request.getPasswordActual(),
        usuario.getPassword())) {
        throw new OperacionNoPermitidaException(
            "La contraseña actual no es correcta");
    }

    usuario.setPassword(
        passwordEncoder.encode(request.getPasswordNueva()));
}
```

No es necesario llamar a save para una entidad gestionada dentro de la transacción, aunque hacerlo tampoco sería incorrecto.

Controlador

El endpoint se añade al PerfilController:

```
@PutMapping("/password")
public ResponseEntity<Void> cambiarPassword(
    Authentication authentication,
    @Valid @RequestBody CambioPasswordRequestDTO request) {
    authService.cambiarPassword(authentication.getName(), request);
    return ResponseEntity.noContent().build();
}
```

Prueba

```
curl -i -X PUT -u pedro:pedro1234 \
  http://localhost:8080/api/v1/perfil/password \
  -H "Content-Type: application/json" \
  -d '{
    "passwordActual": "pedro1234",
    "passwordNueva": "pedro5678"
  }'
```

Observable: 204 No Content.

Después:

```
curl -i -u pedro:pedro5678 \  
http://localhost:8080/api/v1/perfil
```

La nueva contraseña funciona.

La antigua deja de funcionar:

```
curl -i -u pedro:pedro1234 \  
http://localhost:8080/api/v1/perfil
```

Observable: 401.

Corrección explícita de la fuente

El snippet fuente coloca por error práctico el método bajo `/api/v1/auth/password` mientras el mismo punto abre `/api/v1/auth/**`. Eso podría convertir el cambio de contraseña en endpoint público. Nuestra solución conserva el **enunciado** del reto (`/api/v1/perfil/password`) y garantiza que permanezca autenticado.

Resultado esperado global 6.3

Al cerrar el punto tenemos:

- entidades Usuario y Rol;
- relación @ManyToMany;
- repositorios;
- UsuarioDetailsService persistente;
- inicialización idempotente;
- cierre de usuarios en memoria;
- autenticación con usuarios de base de datos;
- registro público limitado a una ruta;
- cambio de contraseña autenticado;
- continuidad del usuario/rol GESTOR;
- HTTP Basic todavía temporal, pendiente de cierre en 6.7.

6.4 — Autorización por roles

Práctica 6.4 — Autorización por roles

Preparación y contexto de la fuente

Contexto del ejercicio: Vamos a proteger endpoints por roles usando tanto reglas de URL como `@PreAuthorize`. Añadiremos un endpoint de administración que solo sea accesible por ADMIN, y un endpoint de perfil que cada usuario pueda consultar solo el suyo.

Requisitos previos: Tener el proyecto mi-proyecto con usuarios en base de datos del punto 6.3.

Paso 6.4.1 — Repasar el estado actual

Fuente: p. 1251.

Base de la fuente — preservada

Abre el SecurityConfig y observa las reglas actuales. Probablemente tienes algo como:

```
.requestMatchers("/api/v1/auth/**").permitAll()
.requestMatchers("/api/v1/public/**").permitAll()
.requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
.anyRequest().authenticated()
```

Vamos a profundizar en la autorización.

Pregunta: ¿Qué endpoint de admin tienes actualmente? ¿Funciona la regla?

Implementación acumulativa M6 — actualizada

Venimos de 6.3 con:

- usuarios persistentes;
- UsuarioDetailsService;
- roles USER, ADMIN, GESTOR;
- registro público específico;
- /api/v1/admin/** protegido por ADMIN;
- HTTP Basic todavía activo;
- CORS/OpenAPI/H2 preservados.

Comprobación estructural:

```
grep -R "InMemoryUserDetailsService" src/main/java || true
```

No debe aparecer.

Pregunta. ¿La regla /api/v1/admin/** protege una operación concreta o toda una zona del API?

Paso 6.4.2 — Activar la seguridad por método

Fuente: p. 1336.

Base de la fuente — preservada

Añade @EnableMethodSecurity al SecurityConfig:

```
import org.springframework.security.config.annotation.method.configuration.
    EnableMethodSecurity;

@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {
    // ...
}
```

Sin esta anotación, las anotaciones @PreAuthorize se ignoran.

Pregunta: ¿Qué pasaría si olvidas esta anotación?

Implementación acumulativa M6 — actualizada

Evoluciona SecurityConfig:

```

@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {
    // ...
}

```

Observable. A partir de ahora Spring evalúa @PreAuthorize en beans interceptados.

Error crítico. Añadir @PreAuthorize sin @EnableMethodSecurity: la anotación queda sin el efecto esperado.

Paso 6.4.3 — Crear un UsuarioController de administración

Fuente: p. 1336.

Base de la fuente — preservada

Crea el UsuarioController en el paquete auth:

```

package es.mecd.demo.miproyecto.auth;

import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
@RequestMapping("/api/v1/admin/usuarios")
public class UsuarioController {

    private final UsuarioService usuarioService;

    public UsuarioController(UsuarioService usuarioService) {
        this.usuarioService = usuarioService;
    }

    @GetMapping
    @PreAuthorize("hasRole('ADMIN')")
    public List<UsuarioResponseDTO> listar() {
        return usuarioService.listarTodos();
    }

    @GetMapping("/{id}")
    @PreAuthorize("hasRole('ADMIN')")
    public UsuarioResponseDTO consultar(@PathVariable Long id) {
        return usuarioService.consultar(id);
    }
}

```

@PreAuthorize("hasRole('ADMIN')") requiere el rol ADMIN. Si un usuario con rol USER
↪ intenta acceder, recibirá 403.

Pregunta: ¿Qué doble protección tiene este endpoint: la regla de URL y la anotación?

Implementación acumulativa M6 — actualizada

Creamos:

src/main/java/es/mecd/demo/miproyecto/auth/UsuarioController.java

```
@RestController
@RequestMapping("/api/v1/admin/usuarios")
public class UsuarioController {

    private final UsuarioService usuarioService;

    public UsuarioController(UsuarioService usuarioService) {
        this.usuarioService = usuarioService;
    }

    @GetMapping
    @PreAuthorize("hasRole('ADMIN')")
    public List<UsuarioResponseDTO> listar() {
        return usuarioService.listarTodos();
    }

    @GetMapping("/{id}")
    @PreAuthorize("hasRole('ADMIN')")
    public UsuarioResponseDTO consultar(@PathVariable Long id) {
        return usuarioService.consultar(id);
    }
}
```

Doble barrera.

1. /api/v1/admin/** exige ADMIN en SecurityFilterChain.
2. Los métodos vuelven a exigir ADMIN.

Pregunta. Si la URL ya exige ADMIN, ¿qué protege la anotación ante un futuro cambio de ruta?

Paso 6.4.4 — Crear el UsuarioService

Fuente: p. 1338.

Base de la fuente — preservada

Crea el UsuarioService:

```
package es.mecd.demo.miproyecto.auth;

import es.mecd.demo.miproyecto.common.exception.RecursoNoEncontradoException;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

@Service
public class UsuarioService {

    private final UsuarioRepository usuarioRepository;

    public UsuarioService(UsuarioRepository usuarioRepository) {
```

```

        this.usuarioRepository = usuarioRepository;
    }

    @Transactional(readOnly = true)
    public List<UsuarioResponseDTO> listarTodos() {
        return usuarioRepository.findAll().stream()
            .map(this::toDTO)
            .toList();
    }

    @Transactional(readOnly = true)
    public UsuarioResponseDTO consultar(Long id) {
        return usuarioRepository.findById(id)
            .map(this::toDTO)
            .orElseThrow(() -> new RecursoNoEncontradoException("Usuario", id));
    }

    private UsuarioResponseDTO toDTO(Usuario usuario) {
        UsuarioResponseDTO dto = new UsuarioResponseDTO();
        dto.setIdentificador(usuario.getId() != null ? usuario.getId().toString()
        ↪ : null);
        dto.setUsername(usuario.getUsername());
        dto.setEmail(usuario.getEmail());
        dto.setRoles(usuario.getRoles().stream()
            .map(Rol::getNombre)
            .toList());
        return dto;
    }
}

```

Pregunta: ¿Por qué el servicio no tiene @PreAuthorize? ¿Podría tenerlo?

Implementación acumulativa M6 — actualizada

El servicio necesita UsuarioRepository para leer usuarios y RolRepository para el reto de cambio de roles.

```

@Service
public class UsuarioService {
    // ...

    @Transactional(readOnly = true)
    public List<UsuarioResponseDTO> listarTodos() {
        return usuarioRepository.findAll().stream()
            .map(this::toDTO)
            .toList();
    }

    @Transactional(readOnly = true)
    public UsuarioResponseDTO consultar(Long id) {
        return usuarioRepository.findById(id)
            .map(this::toDTO)
            .orElseThrow(() ->
                new RecursoNoEncontradoException("Usuario", id));
    }
}

```

El mapeo ordena los nombres de rol para producir respuestas deterministas.

Pregunta. ¿Podría existir `@PreAuthorize` también en el servicio? Sí. La capa de método no está limitada a controladores; la decisión depende de dónde queramos fijar el límite de seguridad.

Paso 6.4.5 — Añadir reglas de URL y de método combinadas

Fuente: p. 1340.

Base de la fuente — preservada

Ahora tenemos dos capas de protección: la URL y el método. Modifica el `SecurityFilterChain` para que las reglas de URL sean generales y las anotaciones se encarguen de lo específico:

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {

    http

        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/v1/auth/**").permitAll()
            .requestMatchers("/api/v1/public/**").permitAll()
            .requestMatchers("/h2-console/**").permitAll()
            .requestMatchers("/swagger-ui/**", "/v3/api-docs/**").permitAll()
            .requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
            .anyRequest().authenticated()
        )
        .csrf(csrf -> csrf.disable())
        .httpBasic(Customizer.withDefaults());

    return http.build();
}
```

La regla `/api/v1/admin/**` requiere `ADMIN`. La anotación `@PreAuthorize` en los métodos también. Es redundante en este caso, pero es habitual: la URL bloquea el acceso antes de llegar al método.

Pregunta: ¿Qué pasa si un usuario con rol `USER` intenta acceder a `/api/v1/admin/usuarios`? ¿Se evalúa la anotación?

Implementación acumulativa M6 — actualizada

La configuración acumulativa conserva:

```
.requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
.anyRequest().authenticated()
```

y el controlador añade:

```
@PreAuthorize("hasRole('ADMIN')")
```

Observable. Un `USER` puede ser rechazado por la cadena antes de que el controlador llegue siquiera a evaluarse.

Pregunta. ¿Por qué esto no significa que `@PreAuthorize` sea inútil?

Paso 6.4.6 — Probar la autorización con distintos usuarios

Fuente: p. 1341.

Base de la fuente — preservada

Reinicia la aplicación y prueba:

```
# Ana (USER) intenta acceder a la lista de usuarios
curl -i -u ana:ana123 http://localhost:8080/api/v1/admin/usuarios
```

```
# Admin intenta acceder
curl -i -u admin:admin123 http://localhost:8080/api/v1/admin/usuarios
```

Ana recibe 403 (autenticada pero sin permisos). Admin recibe 200 con la lista.

Pregunta: ¿Qué diferencia hay entre el 403 de Ana y un 401?

Implementación acumulativa M6 — actualizada

```
# USER autenticado
curl -i -u ana:ana123 \
  http://localhost:8080/api/v1/admin/usuarios
```

```
# ADMIN
curl -i -u admin:admin123 \
  http://localhost:8080/api/v1/admin/usuarios
```

```
# Anónimo
curl -i \
  http://localhost:8080/api/v1/admin/usuarios
```

Esperado:

```
ana      -> 403
admin    -> 200
anónimo  -> 401
```

Pregunta. ¿Por qué Ana recibe 403 y no 401?

Paso 6.4.7 — Crear un endpoint para consultar el propio perfil

Fuente: p. 1342.

Base de la fuente — preservada

Vamos a añadir un endpoint que cada usuario pueda consultar solo su propio perfil. Usaremos `@AuthenticationPrincipal`:

```
package es.mecd.demo.miproyecto.auth;

import org.springframework.security.core.annotation.AuthenticationPrincipal;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.Map;

@RestController
@RequestMapping("/api/v1/perfil")
public class PerfilController {

    private final UsuarioService usuarioService;
```

```

public PerfilController(UsuarioService usuarioService) {
    this.usuarioService = usuarioService;
}

@GetMapping
public Map<String, Object> perfil(@AuthenticationPrincipal UserDetails user) {
    return Map.of(
        "usuario", user.getUsername(),
        "authorities", user.getAuthorities().stream()
            .map(a -> a.getAuthority())
            .toList()
    );
}
}

```

`@AuthenticationPrincipal UserDetails user` inyecta el `UserDetails` del usuario autenticado. Es más cómodo

que `Authentication` cuando solo se necesita el usuario.

Pregunta: ¿Qué diferencia hay entre `@AuthenticationPrincipal` y `Authentication`?

Implementación acumulativa M6 — actualizada

El perfil ya existe desde 6.2/6.3. Lo evolucionamos para usar la forma propuesta por la fuente:

```

@GetMapping
public Map<String, Object> perfil(
    @AuthenticationPrincipal UserDetails user) {
    return Map.of(
        "usuario", user.getUsername(),
        "authorities", user.getAuthorities().stream()
            .map(GrantedAuthority::getAuthority)
            .sorted()
            .toList());
}

```

El cambio de contraseña conserva `Authentication`, porque en ese método sólo necesitamos `getName()` y su contrato ya está probado.

Observable. `@AuthenticationPrincipal` entrega el `UserDetails` que Spring colocó en el contexto.

Pregunta. ¿Cuándo preferirías recibir `Authentication` completo?

Paso 6.4.8 — Crear `UsuarioPrincipal` para 6.7

Fuente: p. 1344.

Base de la fuente — preservada

Vamos a crear una clase `UsuarioPrincipal` que implemente `UserDetails` y exponga el ID del usuario. La creamos ahora para poder usarla en el punto 6.7 (filtro JWT). En este punto (6.4) todavía no la integramos en el flujo principal: el `UsuarioDetailsService` sigue devolviendo `User` de Spring Security.

Nota sobre `UsuarioPrincipal`. En este punto creamos la clase `UsuarioPrincipal` para tenerla lista para el punto 6.7. Sin embargo, todavía no la usamos en el flujo principal: el `UsuarioDetails-`

Service sigue devolviendo el User de Spring Security. En el punto 6.7, cuando implementemos el filtro JWT, construiremos un UsuarioPrincipal a partir de los claims del token. Entonces el principal será consistente en toda la aplicación.

Nota sobre expresiones SpEL con String como principal. Si el principal es un String (como el username) y usamos una expresión SpEL que accede a propiedades (por ejemplo, `#id == authentication.principal.id`), Spring lanza `SpelEvaluationException: Property or field 'id' cannot be found on object of type 'java.lang.String'`. La expresión falla, y el endpoint devuelve un 500 en lugar de 403 o 200. Por eso, cuando queremos usar expresiones SpEL que accedan a propiedades del usuario (ID, email, etc.), necesitamos que el principal sea un objeto con esas propiedades. Ese es el motivo de crear UsuarioPrincipal: un UserDetails personalizado que expone el ID y el email además del username.

Crea la clase:

```
package es.mecd.demo.miproyecto.auth;

import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;

import java.util.Collection;

public class UsuarioPrincipal implements UserDetails {

    private final Long id;
    private final String username;
    private final String email;
    private final Collection<? extends GrantedAuthority> authorities;

    public UsuarioPrincipal(Long id, String username, String email,
                           Collection<? extends GrantedAuthority> authorities) {
        this.id = id;
        this.username = username;
        this.email = email;
        this.authorities = authorities;
    }

    public Long getId() { return id; }
    public String getEmail() { return email; }

    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
        return authorities;
    }

    @Override
    public String getPassword() {
        return null;
    }

    @Override
    public String getUsername() {
        return username;
    }

    @Override
    public boolean isAccountNonExpired() { return true; }
```

```

@Override
public boolean isAccountNonLocked() { return true; }

@Override
public boolean isCredentialsNonExpired() { return true; }

@Override
public boolean isEnabled() { return true; }

}

```

Pregunta: ¿Por qué no podemos usar esta clase en el UsuarioDetailsService todavía?

Implementación acumulativa M6 — actualizada

Creemos ahora, sin integrarlo todavía:

```

public class UsuarioPrincipal implements UserDetails {
    private final Long id;
    private final String username;
    private final String email;
    private final Collection<? extends GrantedAuthority> authorities;

    // constructor + getters + contrato UserDetails
}

```

getPassword() devuelve null: este principal está diseñado para representar una identidad ya validada desde token en 6.7, no para almacenar credenciales.

Estado T6.4-A — PREPARED_NOT_ACTIVE.

- La clase existe.
- UsuarioDetailsService sigue devolviendo User estándar.
- La integración se cierra en 6.7.

Pregunta. ¿Por qué no sustituimos ya el principal activo si la fuente quiere enseñar primero la transición?

Paso 6.4.9 — Probar la expresión de autorización

Fuente: p. 1347.

Base de la fuente — preservada

Cuando en 6.7 integremos el UsuarioPrincipal en el filtro JWT, podremos usar expresiones como #id == authentication.principal.id. Por ahora, no podemos probarla porque el principal es el User de Spring Security, que no tiene getId().

Pregunta: ¿Qué error daría la expresión #id == authentication.principal.id si el principal fuera un String?

Implementación acumulativa M6 — actualizada

La fuente propone la expresión futura:

```
#id == authentication.principal.id
```

pero también advierte que **todavía no puede funcionar** con el principal actual, porque User de Spring Security no tiene getId().

Por tanto este paso se ejecuta como **prueba negativa controlada**, no como código final roto.

Si se añade temporalmente una expresión que accede a:

```
authentication.principal.id
```

la evaluación puede lanzar SpelEvaluationException.

Restauración obligatoria. Elimina la expresión temporal antes de continuar. No queda ningún endpoint final de 6.4 dependiendo de principal.id.

El caso funcional queda diferido y trazado a 6.7, donde el principal será UsuarioPrincipal.

Paso 6.4.10 — Escribir tests de autorización

Fuente: p. 1347.

Base de la fuente — preservada

Añade tests al UsuarioControllerTest:

```
@WebMvcTest(UsuarioController.class)
class UsuarioControllerTest {

    @Autowired

    private MockMvc mockMvc;

    @MockBean
    private UsuarioService usuarioService;

    @Test
    void listar_debeDevolver403_cuandoNoEsAdmin() throws Exception {
        mockMvc.perform(get("/api/v1/admin/usuarios")
            .with(user("ana").roles("USER")))
            .andExpect(status().isForbidden());
    }

    @Test
    void listar_debeDevolver200_cuandoEsAdmin() throws Exception {
        mockMvc.perform(get("/api/v1/admin/usuarios")
            .with(user("admin").roles("ADMIN")))
            .andExpect(status().isOk());
    }

    @Test
    void listar_debeDevolver401_cuandoNoAutenticado() throws Exception {
        mockMvc.perform(get("/api/v1/admin/usuarios"))
            .andExpect(status().isUnauthorized());
    }
}
```

.with(user("ana").roles("USER")) simula un usuario autenticado con rol USER. Es equivalente a @WithMockUser, pero se puede

aplicar a una sola petición.

Los tres tests cubren: 403 (autenticado sin permisos), 200 (autenticado con permisos), 401 (no autenticado).

Pregunta: ¿Por qué es importante testear los tres escenarios?

Implementación acumulativa M6 — actualizada

La fuente usa @MockBean; M6 moderniza el ejemplo a @MockitoBean.

```
@WebMvcTest(UsuarioController.class)
@Import(SecurityConfig.class)
class UsuarioControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockitoBean
    private UsuarioService usuarioService;

    @Test
    void listar_debeDevolver403_cuandoNoEsAdmin()
        throws Exception {
        mockMvc.perform(get("/api/v1/admin/usuarios")
            .with(user("ana").roles("USER"))
            .andExpect(status().isForbidden());
    }

    @Test
    void listar_debeDevolver200_cuandoEsAdmin()
        throws Exception {
        mockMvc.perform(get("/api/v1/admin/usuarios")
            .with(user("admin").roles("ADMIN"))
            .andExpect(status().isOk());
    }

    @Test
    void listar_debeDevolver401_cuandoNoAutenticado()
        throws Exception {
        mockMvc.perform(get("/api/v1/admin/usuarios"))
            .andExpect(status().isUnauthorized());
    }
}
```

Objetivo. Cubrir las tres ramas semánticamente distintas: 401, 403 y 200.

Aislamiento real del slice MVC. @WebMvcTest no levanta toda la aplicación, pero sí puede descubrir componentes web como JwtAuthenticationFilter. Además, @AutoConfigureMockMvc(addFilters = false) evita aplicar los filtros a las peticiones de MockMvc, pero **no impide que Spring cree el bean del filtro** durante el arranque del contexto. Como el filtro recibe JwtService y TokenRevocationService por constructor, el slice debe proporcionar esos colaboradores con @MockitoBean cuando no carga la configuración completa de M6.

```
@MockitoBean
private JwtService jwtService;

@MockitoBean
private TokenRevocationService tokenRevocationService;
```

Así este test sigue verificando únicamente 401/403/200 del controlador y de su `TestSecurityConfig`, mientras los tests JWT dedicados prueban el mecanismo real. Esta separación evita convertir un slice rápido en un `@SpringBootTest` y, sobre todo, evita un falso negativo de arranque por una dependencia que no forma parte del objetivo del test.

Paso 6.4.11 — Errores comunes del ejercicio

Fuente: p. 1229.

Base de la fuente — preservada

Error Causa Solución

@PreAuthorize no se aplica Falta @EnableMethodSecurity Añadirlo aplica

hasRole("ROLE_ADMIN") no funciona añadir el prefijo Usar hasRole("ADMIN") funciona

hasAuthority("ADMIN") no funciona Las authorities tienen Usar hasAuthority("ROLE_ADMIN") o hasRole("ADMIN") funciona prefijo ROLE_

Error Causa Solución

La expresión SpEL no funciona El principal no tiene la Usar un principal personalizado compila propiedad

El endpoint devuelve 403 El rol no tiene el prefijo Revisar la transformación con el rol correcto correcto

El test devuelve 200 sin Falta la regla de URL Añadir `anyRequest().authenticated()` autenticación

@PreAuthorize en método Spring no intercepta Hacerlo público privado privados

Llamada interna no aplica No pasa por el proxy Mover a otro bean la anotación

El usuario no está El 403 no es el esperado Verificar el orden de las reglas autenticado

Implementación acumulativa M6 — actualizada

Error	Causa	Solución
@PreAuthorize no se aplica	falta @EnableMethodSecurity	activarlo
hasRole("ROLE_ADMIN") falla	hasRole gestiona el prefijo	usar hasRole("ADMIN")
hasAuthority("ADMIN") falla	authority real es ROLE_ADMIN	usar literal correcto
SpEL no resuelve id	principal actual no expone id	esperar a UsuarioPrincipal
403 con rol esperado	transformación ROLE_ inconsistente	revisar
endpoint anónimo da 200	falta regla general	UsuarioDetailsService
anotación en privado	proxy no intercepta como se espera	mantener <code>anyRequest().authenticated()</code>
llamada interna elude seguridad	self-invocation	proteger método público
test usa API obsoleta	@MockBean antiguo	mover frontera a otro bean
test 401/403 confundido	identidad/rol mal preparados	usar @MockitoBean
		separar escenario anónimo y autenticado

Paso 6.4.12 — Reto resuelto: endpoint de cambio de rol

Fuente: p. 1351.

Base de la fuente — preservada

Reto: Añadir un endpoint PUT /api/v1/admin/usuarios/{id}/roles que permita a un ADMIN cambiar los roles de un usuario.

Solución paso a paso:

Paso 1: Crear el DTO:

```
public class CambioRolesRequestDTO {

    @NotEmpty(message = "Debe indicar al menos un rol")
    private List<String> roles;

    public List<String> getRoles() { return roles; }
    public void setRoles(List<String> roles) { this.roles = roles; }
}
```

Paso 2: Añadir el método al UsuarioService:

```
@Transactional
public UsuarioResponseDTO cambiarRoles(Long id, List<String> nombresRoles) {
    Usuario usuario = usuarioRepository.findById(id)
        .orElseThrow(() -> new RecursoNoEncontradoException("Usuario", id));

    Set<Rol> roles = new HashSet<>();

    for (String nombre : nombresRoles) {
        Rol rol = rolRepository.findByNombre(nombre)
            .orElseThrow(() -> new RecursoNoEncontradoException("Rol",
                nombre));
        roles.add(rol);
    }

    usuario.setRoles(roles);
    return toDTO(usuarioRepository.save(usuario));
}
```

Paso 3: Añadir el endpoint al UsuarioController:

```
@PutMapping("/{id}/roles")
@PreAuthorize("hasRole('ADMIN')")
public UsuarioResponseDTO cambiarRoles(
    @PathVariable Long id,
    @Valid @RequestBody CambioRolesRequestDTO request) {
    return usuarioService.cambiarRoles(id, request.getRoles());
}
```

Paso 4: Probar:

```
curl -i -X PUT -u admin:admin123 \
    http://localhost:8080/api/v1/admin/usuarios/2/roles \
    -H "Content-Type: application/json" \
    -d '{"roles": ["USER", "GESTOR"]}'
```

Verás el usuario con sus nuevos roles.

Pregunta: ¿Por qué el endpoint requiere rol ADMIN? ¿Qué pasaría si un usuario normal pudiera cambiar sus propios roles?

Resultado esperado global

Al final del ejercicio, deberías tener:

- @EnableMethodSecurity activado.
- Un UsuarioController protegido por ADMIN con @PreAuthorize.
- Una clase UsuarioPrincipal creada pero no integrada todavía.
- Endpoints que usan @AuthenticationPrincipal.
- Tests que cubren 401, 403 y 200.

Resumen técnico del ejercicio El ejercicio ha demostrado:

- @EnableMethodSecurity: activa la seguridad por método.
- @PreAuthorize: en métodos y clases.
- hasRole vs hasAuthority: con o sin prefijo.
- SpEL: expresiones con parámetros y principal.
- @AuthenticationPrincipal: inyección del usuario.
- UsuarioPrincipal: principal personalizado, creado aquí, integrado en 6.7.
- Combinación de URL y método.
- Tests: user(...) con roles.

Conclusión y enlace al siguiente punto En este punto 6.4 hemos profundizado en la autorización por roles:

- Autorización por URL: reglas generales en el SecurityFilterChain.
- Autorización por método: @PreAuthorize, @Secured, @PostAuthorize.
- @EnableMethodSecurity: activa la seguridad por método.
- hasRole vs hasAuthority: con o sin prefijo.
- hasAnyRole, hasAllRoles:
- SpEL: expresiones complejas.
- Acceso al usuario: Authentication, @AuthenticationPrincipal, SecurityContextHolder.
- Principal personalizado: UsuarioPrincipal (creado aquí, integrado en 6.7).
- Buenas prácticas: URL + método, roles en constantes, testear.

En la práctica, hemos protegido endpoints con @PreAuthorize, hemos creado un principal personalizado, y hemos escrito tests que cubren 401, 403 y 200.

La idea clave: la autorización se aplica en dos capas. La URL protege áreas generales. El método protege operaciones específicas. Combinar las dos da flexibilidad y seguridad.

En el siguiente punto, 6.5 — Introducción a JWT, profundizaremos en los JSON Web Tokens: qué son, cómo se estructuran, qué diferencia hay entre autenticación con sesión y stateless, y por qué JWT es el estándar en APIs REST modernas.

Fin del Punto 6.4.

Implementación acumulativa M6 — actualizada

El reto exige:

PUT /api/v1/admin/usuarios/{id}/roles

DTO

```
public class CambioRolesRequestDTO {
    @NotEmpty(message = "Debe indicar al menos un rol")
    private List<String> roles;

    // getter/setter
}
```

Servicio

```
@Transactional
public UsuarioResponseDTO cambiarRoles(
    Long id,
    List<String> nombresRoles) {

    Usuario usuario = usuarioRepository.findById(id)
        .orElseThrow(() ->
            new RecursoNoEncontradoException(
                "Usuario", id));

    Set<Rol> roles = new HashSet<>();
    for (String nombre : nombresRoles) {
        Rol rol = rolRepository.findByNombre(nombre)
            .orElseThrow(() ->
                new RecursoNoEncontradoException(
                    "Rol", nombre));

        roles.add(rol);
    }

    usuario.setRoles(roles);
    return toDTO(usuario);
}
```

Dentro de la transacción no necesitamos un save redundante para una entidad gestionada; la fuente lo incluye y sería válido, pero M6 deja que dirty checking persista el cambio.

Controlador

```
@PostMapping("/{id}/roles")
@PreAuthorize("hasRole('ADMIN')")
public UsuarioResponseDTO cambiarRoles(
    @PathVariable Long id,
    @Valid @RequestBody CambioRolesRequestDTO request) {
    return usuarioService.cambiarRoles(
        id,
        request.getRoles());
}
```

Prueba

```
curl -i -X PUT -u admin:admin123 \
    http://localhost:8080/api/v1/admin/usuarios/2/roles \
    -H "Content-Type: application/json" \
    -d '{"roles":["USER","GESTOR"]}'
```

Observable: el usuario queda con el conjunto exacto de roles solicitado.


```

    "alg": "HS256",
    "typ": "JWT"
}

```

Payload:

```

json
{
  "sub": "ana",
  "roles": ["ROLE_USER"],
  "iat": 1516239022
}

```

Signature: una cadena Base64Url.

Observa que el token está firmado con una clave que jwt.io conoce (es un ejemplo didáctico). Si cambias el payload, la firma se invalida.

Pregunta: ¿Qué información contiene el payload? ¿Es legible?

Implementación acumulativa M6 — actualizada

La fuente propone inspeccionar este JWT didáctico:

```

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhbmEiLCJyb2xlcyI6WyJST0xFX1VTRVIiXSwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c

```

Sin los saltos de línea forma una sola cadena.

Header:

```

{
  "alg": "HS256",
  "typ": "JWT"
}

```

Payload:

```

{
  "sub": "ana",
  "roles": ["ROLE_USER"],
  "iat": 1516239022
}

```

Observable. Header y payload son legibles sin conocer la clave.

La fuente usa jwt.io para visualizarlo. Sólo se deben pegar allí **tokens didácticos**, nunca access/refresh tokens reales de un sistema.

Pregunta. Si el payload se puede leer, ¿qué aporta entonces la firma?

Paso 6.5.2 — Crear una clase para generar un JWT manualmente

Fuente: p. 1373.

Base de la fuente — preservada

Vamos a crear una clase que genere un JWT con Java estándar. No usaremos librerías; solo java.util.Base64 y javax.crypto.Mac.

Crea una clase JwtManual en un paquete de pruebas (por ejemplo, es.mecd.demo.miproyecto.jwt):

```

package es.mecd.demo.miproyecto.jwt;

import javax.crypto.Mac;
import javax.crypto.spec.SecretKeySpec;
import java.nio.charset.StandardCharsets;
import java.util.Base64;

public class JwtManual {

    private static final String SECRET =
        ↪ "claveSecretaMuyLargaParaHMACSHA256ConAlMenos32Bytes";

    public static void main(String[] args) throws Exception {
        String header = "{\"alg\":\"HS256\",\"typ\":\"JWT\"}";
        String payload =
            ↪ "{\"sub\":\"ana\",\"roles\":[\"ROLE_USER\"],\"iat\":1516239022}";

        String headerBase64 = base64UrlEncode(header);
        String payloadBase64 = base64UrlEncode(payload);

        String firmaInput = headerBase64 + "." + payloadBase64;
        String firma = firmarHMACSHA256(firmaInput, SECRET);

        String jwt = firmaInput + "." + firma;
        System.out.println("JWT generado:");
        System.out.println(jwt);
    }

    private static String base64UrlEncode(String input) {
        return Base64.getUrlEncoder()
            .withoutPadding()
            .encodeToString(input.getBytes(StandardCharsets.UTF_8));
    }

    private static String firmarHMACSHA256(String input, String secret) throws
        ↪ Exception {
        Mac mac = Mac.getInstance("HmacSHA256");
        SecretKeySpec keySpec = new SecretKeySpec(
            secret.getBytes(StandardCharsets.UTF_8), "HmacSHA256");
        mac.init(keySpec);
        byte[] firma = mac.doFinal(input.getBytes(StandardCharsets.UTF_8));
        return Base64.getUrlEncoder().withoutPadding().encodeToString(firma);
    }
}

```

Base64.getUrlEncoder().withoutPadding() codifica en Base64Url sin relleno (=).

Mac.getInstance("HmacSHA256") obtiene una instancia del algoritmo HMAC-SHA256.

SecretKeySpec envuelve la clave secreta.

mac.doFinal(...) calcula el HMAC.

Ejecuta el main y verás un JWT generado.

Pregunta: ¿Cuántas partes tiene el token? ¿Cómo se separan?

Implementación acumulativa M6 — actualizada

Creamos en código de pruebas:

```
src/test/java/es/mecd/demo/miproyecto/jwt/JwtManual.java
```

La clase usa exclusivamente Java estándar:

```
Base64.getUrlEncoder().withoutPadding()
Mac.getInstance("HmacSHA256")
SecretKeySpec
```

Flujo:

```
JSON header
  ↓ Base64Url
header64
```

```
JSON payload
  ↓ Base64Url
payload64
```

```
header64.payload64
  ↓ HMAC-SHA256(secret)
signature64
```

```
header64.payload64.signature64
```

La constante incluida en la clase está marcada DIDACTIC_SECRET: es una clave de ejemplo y **no** un secreto de producción.

Estado T6.5-A — INTRODUCER. Firma JWT manual sólo para aprendizaje. La implementación operativa se reemplaza por JJWT en 6.6.

Paso 6.5.3 — Decodificar el token generado

Fuente: p. 1375.

Base de la fuente — preservada

Copia el token generado y pégalo en jwt.io. Verás el header y el payload decodificados. La firma no la podrá verificar jwt.io porque no conoce tu clave.

Pregunta: ¿Puede jwt.io verificar la firma? ¿Por qué?

Implementación acumulativa M6 — actualizada

La utilidad incorpora:

```
public static String decodificarPayload(String jwt) {
    String[] partes = jwt.split("\\.", -1);
    if (partes.length != 3) {
        throw new IllegalArgumentException("Token mal formado");
    }

    byte[] decoded = Base64.getUrlDecoder()
        .decode(partes[1]);
    return new String(decoded, StandardCharsets.UTF_8);
}
```

Ejecuta la clase:

```
./mvnw -Dtest=JwtManualTest test
```

o su main desde el IDE.

Observable. El JSON del payload aparece en texto plano.

Pregunta. ¿Ha sido necesaria la clave para leerlo?

Paso 6.5.4 — Modificar el payload sin actualizar la firma

Fuente: p. 1375.

Base de la fuente — preservada

Vamos a modificar el token para ver qué pasa. Coge el token generado, cambia una letra del payload (por ejemplo, cambia ana por pedro) y pégalo en jwt.io.

Verás que la firma ya no coincide. jwt.io muestra un error: "Invalid signature".

Pregunta: ¿Qué demuestra este experimento?

Implementación acumulativa M6 — actualizada

Toma un token válido y sustituye sólo su segunda parte por otro payload Base64Url, conservando la firma antigua.

Ejemplo conceptual:

```
header.payload_original.firma_original
      ↓ cambiar
header.payload_modificado.firma_original
```

Observable. La decodificación sigue funcionando, pero la verificación criptográfica falla.

Esto demuestra por qué:

payload legible ≠ payload confiable

Restauración. El token manipulado sólo se usa como dato de prueba y no se reutiliza en pasos posteriores como token válido.

Paso 6.5.5 — Verificar el token con Java

Fuente: p. 1376.

Base de la fuente — preservada

Vamos a verificar el token con Java. Añade un método verificar:

```
public static boolean verificar(String jwt, String secret) throws Exception {
    String[] partes = jwt.split("\\.");
    if (partes.length != 3) {
        return false;
    }

    String firmaInput = partes[0] + "." + partes[1];
    String firmaEsperada = firmarHMACSHA256(firmaInput, secret);
    return firmaEsperada.equals(partes[2]);
}
```

Y modifica el main para verificarlo:

```

public static void main(String[] args) throws Exception {

    // ... generación del token

    String jwt = firmaInput + "." + firma;
    System.out.println("JWT generado:");
    System.out.println(jwt);

    boolean valido = verificar(jwt, SECRET);
    System.out.println("Token válido: " + valido);

    // Modificamos el payload y verificamos de nuevo
    String jwtModificado = jwt.replace("ana", "pedro");
    System.out.println("Token modificado válido: " + verificar(jwtModificado,
        ↪ SECRET));
}

```

jwt.split("\\.") divide el token en sus tres partes. El \\ es porque el punto es un carácter especial en expresiones regulares.

Ejecuta y verás:

JWT generado:

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

Token válido: true

Token modificado válido: false

Pregunta: ¿Por qué el token modificado no es válido? ¿Qué garantiza la firma?

Implementación acumulativa M6 — actualizada

JwtManual.verificar(jwt, secret):

1. exige exactamente tres partes;
2. vuelve a firmar header.payload;
3. decodifica firma esperada y recibida;
4. compara los bytes con MessageDigest.isEqual.

Ejemplo:

```

boolean valido = JwtManual.verificar(
    jwt,
    JwtManual.DIDACTIC_SECRET);

```

Observable. Token intacto → true; token manipulado → false.

Importante. Esta verificación manual valida la **firma**, no interpreta todavía exp. La expiración real será validada por la biblioteca en 6.6.

Paso 6.5.6 — Probar con distintos payloads

Fuente: p. 1378.

Base de la fuente — preservada

Modifica el payload para incluir más claims:

```

String payload = "{\"sub\":\"ana\", \"email\":\"ana@educacion.gob.es\", \"roles\": [
    ↪ \"ROLE_USER\", \"ROLE_GESTOR\"], \"iat\":1516239
022, \"exp\":999999999}";

```

Ejecuta y decodifica en jwt.io. Verás los nuevos claims.

Pregunta: ¿Qué claims registrados has añadido? ¿Qué representan?

Implementación acumulativa M6 — actualizada

La fuente amplía los claims:

```
{
  "sub": "ana",
  "email": "ana@educacion.gob.es",
  "roles": ["ROLE_USER", "ROLE_GESTOR"],
  "iat": 1516239022,
  "exp": 9999999999
}
```

Genera el token y decodifícalo localmente.

Observables.

- sub, iat y exp son claims registrados;
- email y roles son datos de aplicación en este ejemplo;
- añadir claims aumenta el tamaño del token;
- el contenido sigue siendo legible.

Pregunta. ¿Qué dato de esa lista no debería añadirse si no es necesario para tomar decisiones?

Paso 6.5.7 — Reflexionar sobre la seguridad

Fuente: p. 1378.

Base de la fuente — preservada

Responde a estas preguntas:

- ¿Se puede leer el payload sin la clave? ¿Por qué?
- ¿Se puede modificar el payload sin invalidar la firma? ¿Por qué?
- ¿Qué pasa si se filtra la clave secreta?
- ¿Por qué no se debe meter la contraseña en el payload?

Pregunta: ¿Qué medidas de seguridad son importantes al usar JWT?

Implementación acumulativa M6 — actualizada

Responde y justifica:

1. ¿Se puede leer el payload sin la clave? **Sí**, porque Base64Url no cifra.
2. ¿Se puede cambiar sin invalidar la firma? **No**, salvo que el atacante pueda crear una firma válida.
3. ¿Qué ocurre si se filtra una clave HS256? Un atacante podría emitir tokens que el servidor aceptaría como firmados por nosotros.
4. ¿Debe viajar una contraseña dentro del payload? **No**.
5. ¿Un token firmado necesita HTTPS? **Sí**: la firma no impide que otro capture y reutilice el token.
6. ¿Debemos registrar tokens completos en logs? **No**.

Observable. El modelo de amenaza queda explícito antes de introducir la librería.

Paso 6.5.8 — Comparar con un token real

Fuente: p. 1379.

Base de la fuente — preservada

Cuando en el punto 6.6 generemos tokens reales, verás que tienen la misma estructura. La diferencia es que la librería (JJWT) se encarga de:

- Codificar el header y el payload en Base64Url.
- Calcular la firma.
- Añadir los claims automáticamente (iat, exp).
- Verificar la firma y la expiración.

Pero por debajo, hace exactamente lo que hemos hecho a mano.

Pregunta: ¿Qué ventaja tiene usar una librería en lugar de hacerlo a mano?

Implementación acumulativa M6 — actualizada

En 6.6 JJWT hará de forma robusta lo que aquí hemos hecho manualmente:

- construir/serializar claims;
- codificar las partes;
- firmar;
- parsear;
- verificar firma;
- verificar expiración;
- convertir errores en excepciones específicas.

La estructura compacta seguirá siendo:

`header.payload.signature`

Cierre conceptual. Comprender la implementación manual sirve para diagnosticar; no justifica reimplementar JWT en código productivo.

Paso 6.5.9 — Errores comunes del ejercicio

Fuente: p. 1379.

Base de la fuente — preservada

Error Causa Solución

IllegalArgumentException en Caracteres no válidos Usar getURLEncoder Base64

Error Causa Solución

La firma no coincide La clave no es la misma Verificar la clave

Se usó getEncoder en lugar El token tiene = al final Usar withoutPadding de getURLEncoder

El token no se puede decodificar Formato incorrecto Verificar las tres partes

Verificar el nombre NoSuchAlgorithmException El algoritmo no está disponible (HmacSHA256)

Caracteres extraños en el payload Codificación incorrecta Usar UTF-8

Implementación acumulativa M6 — actualizada

Error	Causa	Corrección
IllegalArgumentException Base64 firma no coincide	alfabeto/formato incorrecto clave o contenido distintos	usar <code>getUrlEncoder/getUrlDecoder</code> verificar <code>header.payload</code> y clave
aparece = al final no hay tres partes	encoder estándar/padding token mal formado	<code>withoutPadding()</code> validar estructura antes de acceder
NoSuchAlgorithmException	nombre de algoritmo incorrecto	HmacSHA256
caracteres extraños “decodifica, luego es válido”	charset implícito confundir lectura con verificación	usar UTF-8 verificar firma
token real enviado a web externa	exposición de credencial	usar tokens de ejemplo o decoder local

Paso 6.5.10 — Experimentar con la expiración

Fuente: p. 1380.

Base de la fuente — preservada

Añade un claim exp con una fecha en el pasado:

```
String payload = "{\"sub\":\"ana\",\"exp\":1}";
```

Decodifica en [jwt.io](#). Verás que exp es una fecha muy antigua. En una verificación real, el servidor rechazaría este token porque ha expirado.

Pregunta: ¿Qué pasa si el token tiene exp en el pasado? ¿Lo rechazaría el servidor?

Implementación acumulativa M6 — actualizada

Genera un payload:

```
{
  "sub": "ana",
  "exp": 1
}
```

Al decodificarlo se ve que exp representa un instante muy antiguo.

Punto crítico. Nuestra función manual verificar sólo comprueba HMAC y, por diseño pedagógico, podría devolver true aunque exp esté vencido.

Esto no contradice JWT: demuestra que el consumidor debe validar **firma + claims temporales**. JWT se encargará de esa política en 6.6.

Pregunta. ¿Qué peligro tendría verificar únicamente la firma?

Paso 6.5.11 — Verificar el token con una clave incorrecta

Fuente: p. 1381.

Base de la fuente — preservada

Modifica el main para verificar con una clave distinta:

```
boolean valido = verificar(jwt, "otraClave");
System.out.println("Con clave incorrecta: " + valido);
```

Verás que devuelve false. La firma no coincide porque la clave es distinta.

Pregunta: ¿Qué garantiza que solo quien tiene la clave puede emitir tokens válidos?

Implementación acumulativa M6 — actualizada

```
boolean valido = JwtManual.verificar(
    jwt,
    "otraClaveQueNoEsLaOriginal");
```

Observable: false.

Con HS256, la capacidad de generar una firma válida depende del secreto compartido.

Pregunta. ¿Por qué una fuga de esa clave es más grave que la exposición del payload?

Paso 6.5.12 — Reto resuelto: decodificar un token sin verificar la firma

Fuente: p. 1381.

Base de la fuente — preservada

Reto: Escribir un método que decodifique el payload de un JWT sin verificar la firma. Útil para inspeccionar un token durante el desarrollo.

Solución paso a paso:

Paso 1: Añadir el método:

```
public static String decodificarPayload(String jwt) {
    String[] partes = jwt.split("\\.");
    if (partes.length != 3) {
        throw new IllegalArgumentException("Token mal formado");
    }
    byte[] decoded = Base64.getUrlDecoder().decode(partes[1]);
    return new String(decoded, StandardCharsets.UTF_8);
}
```

Paso 2: Usarlo en el main:

```
String payloadDecodificado = decodificarPayload(jwt);
System.out.println("Payload decodificado: " + payloadDecodificado);
```

Paso 3: Ejecutar y ver el payload en texto plano.

Pregunta: ¿Por qué es útil decodificar el payload sin verificar la firma? ¿Cuándo no se debe hacer?

Resultado esperado global

Al final del ejercicio, deberías haber:

- Entendido la estructura de un JWT: header, payload, signature.
- Generado un JWT manualmente con Java estándar.
- Verificado su firma con HMAC-SHA256.

- Comprobado que modificar el payload invalida la firma.
- Decodificado el payload sin verificar la firma.
- Reflexionado sobre las implicaciones de seguridad.

Resumen técnico del ejercicio El ejercicio ha demostrado:

- Estructura: tres partes separadas por puntos.
- Base64Url: codificación de header y payload.
- Firma: HMAC-SHA256 con clave secreta.
- Integridad: modificar el payload invalida la firma.
- Expiración: exp en el payload.
- No cifrado: el payload es legible.
- Verificación: recalcular la firma y comparar.

Conclusión y enlace al siguiente punto En este punto 6.5 hemos introducido JWT:

- Qué es: token autocontenido con header, payload y signature.
- Qué resuelve: autenticación stateless para APIs REST y microservicios.
- Estructura: tres partes separadas por puntos, codificadas en Base64Url.
- Claims: registrados, públicos y privados.
- Firma: garantiza la integridad. Simétrica o asimétrica.
- Ciclo de vida: emisión, uso, expiración, renovación.
- Casos de uso: APIs REST, microservicios, apps móviles.
- Cuándo no usar: sesión tradicional, invalidación inmediata, datos sensibles.
- Buenas prácticas: HTTPS, tokens cortos, verificar firma, no meter datos sensibles.

En la práctica, hemos generado un JWT manualmente con Java estándar, hemos verificado su firma, hemos comprobado que modificar el payload lo invalida, y hemos decodificado el payload sin verificar la firma.

La idea clave: un JWT es un token autocontenido y firmado. La firma garantiza que no ha sido modificado, pero no lo cifra. Por eso nunca se debe meter información sensible.

En el siguiente punto, 6.6 — Generación de tokens JWT, usaremos la librería JWT para generar tokens reales con firma, expiración y claims personalizados. Veremos cómo configurar la clave secreta, cómo añadir claims y cómo devolver el token al cliente.

Fin del Punto 6.5.

Implementación acumulativa M6 — actualizada

El método queda implementado en JwtManual:

```
public static String decodificarPayload(String jwt) {
    String[] partes = jwt.split("\\.", -1);
    if (partes.length != 3) {
        throw new IllegalArgumentException("Token mal formado");
    }
    byte[] decoded = Base64.getUrlDecoder()
        .decode(partes[1]);
    return new String(decoded, StandardCharsets.UTF_8);
}
```

Uso:

```
String payload = JwtManual.decodificarPayload(jwt);
System.out.println(payload);
```

Regla de seguridad.

DECODIFICAR = inspeccionar
VERIFICAR = decidir si podemos confiar

Nunca se debe autorizar una operación basándose únicamente en un payload decodificado.

Resultado esperado global 6.5

Al finalizar:

- comprendemos header/payload/signature;
- hemos generado JWT manual con Java estándar;
- hemos verificado HMAC-SHA256;
- modificar payload invalida firma;
- clave incorrecta invalida firma;
- Base64Url no cifra;
- exp necesita validación explícita;
- podemos decodificar localmente sin verificar;
- la implementación manual queda clasificada como **didáctica** y será sustituida como mecanismo real por JJWT en 6.6.

6.6 — Generación de tokens JWT

Práctica 6.6 — Generación de tokens JWT

Preparación y contexto de la fuente

Contexto del ejercicio: Vamos a añadir JJWT al proyecto, configurar la clave secreta, crear un JwtService que genere tokens, y añadir un endpoint de login que devuelva el token al cliente. También añadiremos un endpoint de refresh.

Requisitos previos: Tener el proyecto mi-proyecto con usuarios en base de datos del punto 6.4.

Paso 6.6.1 — Añadir las dependencias de JJWT

Fuente: p. 1407.

Base de la fuente — preservada

Abre el pom.xml y añade las tres dependencias:

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.12.3</version>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.12.3</version>
  <scope>runtime</scope>
```

```

</dependency>

<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.12.3</version>
  <scope>runtime</scope>
</dependency>

```

Recarga Maven.

Pregunta: ¿Por qué jjwt-impl y jjwt-jackson tienen scope=runtime?

Implementación acumulativa M6 — actualizada

Archivo: M6/proyecto/pom.xml.

M6 usa JWT **0.13.0**: jjwt-api en compile y jjwt-impl/jjwt-jackson en runtime. Es la modernización del 0.12.3 mostrado en la fuente.

```
./mvnw -B -DskipTests compile
```

Observable: Maven resuelve los tres módulos y el código puede importar Jwts, Claims, Decoders y Keys.

Error/corrección: si falta un módulo runtime, la compilación puede pasar pero el parseo/serialización falla al ejecutar; conservar los tres.

Pregunta: ¿por qué la implementación y Jackson no necesitan exponerse al compilador de nuestro código?

Paso 6.6.2 — Configurar las propiedades de JWT

Fuente: p. 1408.

Base de la fuente — preservada

Añade a application.properties:

```

jwt.secret=${JWT_SECRET:claveDeDesarrolloConAlMenos32BytesParaHS256}
jwt.expiration=3600000
jwt.refresh-expiration=604800000

```

jwt.secret se lee de la variable de entorno JWT_SECRET. Si no está definida, se usa el valor por defecto (solo para desarrollo).

jwt.expiration=3600000 es 1 hora en milisegundos.

jwt.refresh-expiration=604800000 es 7 días en milisegundos.

En producción, siempre se define JWT_SECRET como variable de entorno.

Pregunta: ¿Por qué hay un valor por defecto para jwt.secret?

Implementación acumulativa M6 — actualizada

M6 añade jwt.secret, jwt.expiration y jwt.refresh-expiration a dev/test/prod. Dev y test usan secretos Base64 no productivos; producción exige \${JWT_SECRET}.

```

jwt.secret=${JWT_SECRET}
jwt.expiration=${JWT_EXPIRATION:900000}
jwt.refresh-expiration=${JWT_REFRESH_EXPIRATION:604800000}

```

Observable: prod no tiene secreto embebido; si JWT_SECRET falta o no es Base64 fuerte, el arranque falla cerrado.

Corrección a la fuente: si JwtConfig decodifica Base64, el fallback de desarrollo debe ser Base64 real. M6 elimina la contradicción "texto cualquiera + Decoders.BASE64".

Paso 6.6.3 — Crear la configuración de la clave

Fuente: p. 1409.

Base de la fuente — preservada

Crea JwtConfig en el paquete auth:

```
package es.mecd.demo.miproyecto.auth;

import io.jsonwebtoken.io.Decoders;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import javax.crypto.SecretKey;

@Configuration
public class JwtConfig {

    @Value("${jwt.secret}")
    private String secret;

    @Bean
    public SecretKey jwtSecretKey() {
        byte[] clave = Decoders.BASE64.decode(secret);
        return Keys.hmacShaKeyFor(clave);
    }
}
```

`@Value("${jwt.secret}")` inyecta la propiedad.

`Decoders.BASE64.decode(secret)` decodifica el string en Base64.

`Keys.hmacShaKeyFor(clave)` construye la `SecretKey`. Si la clave no es lo bastante larga, lanza `WeakKeyException`.

Pregunta: ¿Qué pasa si `jwt.secret` no está bien formado en Base64?

Implementación acumulativa M6 — actualizada

Archivo: `auth/JwtConfig.java`.

```
@Bean
SecretKey jwtSecretKey() {
    return Keys.hmacShaKeyFor(Decoders.BASE64.decode(secret));
}
```

La implementación captura errores de formato y los convierte en un fallo de configuración explícito.

Observable: una clave <256 bits o Base64 inválido impide iniciar un sistema aparentemente seguro con una clave débil.

Paso 6.6.4 — Crear el JwtService

Fuente: p. 1410.

Base de la fuente — preservada

Crea JwtService:

```
package es.mecd.demo.miproyecto.auth;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.JwtException;
import io.jsonwebtoken.Jwts;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.stereotype.Service;

import javax.crypto.SecretKey;
import java.util.Date;
import java.util.List;

@Service
public class JwtService {

    private final SecretKey clave;
    private final long expirationMs;
    private final long refreshExpirationMs;

    public JwtService(SecretKey clave,
                     @Value("${jwt.expiration}") long expirationMs,
                     @Value("${jwt.refresh-expiration}") long refreshExpirationMs) {
        this.clave = clave;

        this.expirationMs = expirationMs;
        this.refreshExpirationMs = refreshExpirationMs;
    }

    public String generarToken(Usuario usuario) {
        Date ahora = new Date();
        Date expiracion = new Date(ahora.getTime() + expirationMs);

        List<String> roles = usuario.getRoles().stream()
            .map(rol -> "ROLE_" + rol.getNombre())
            .toList();

        return Jwts.builder()
            .subject(usuario.getUsername())
            .claim("id", usuario.getId())
            .claim("email", usuario.getEmail())
            .claim("roles", roles)
            .issuedAt(ahora)
            .expiration(expiracion)
            .signWith(clave)
            .compact();
    }
}
```

```
public String generarRefreshToken(Usuario usuario) {
    Date ahora = new Date();

    Date expiration = new Date(ahora.getTime() + refreshExpirationMs);

    return Jwts.builder()
        .subject(usuario.getUsername())
        .claim("tipo", "refresh")
        .issuedAt(ahora)
        .expiration(expiration)
        .signWith(clave)
        .compact();
}

public String extraerUsername(String token) {
    return parsearClaims(token).getSubject();
}

public Claims extraerClaims(String token) {
    return parsearClaims(token);
}

public List<GrantedAuthority> extraerAuthorities(String token) {
    Claims claims = parsearClaims(token);
    List<String> roles = claims.get("roles", List.class);
    if (roles == null) {
        return List.of();
    }

    return roles.stream()
        .map(rol -> (GrantedAuthority) new SimpleGrantedAuthority(rol))
        .toList();
}

public boolean esValido(String token) {
    try {
        parsearClaims(token);
        return true;
    } catch (JwtException | IllegalArgumentException e) {
        return false;
    }
}

public long getExpirationMs() {
    return expirationMs;
}

private Claims parsearClaims(String token) {
    return Jwts.parser()
        .verifyWith(clave)
        .build()
        .parseSignedClaims(token)
        .getPayload();
}
}
```

Analicemos los elementos:

`Jwts.builder()` empieza a construir el token.

`.subject(...)` añade el claim sub.

`.claim("id", ...)` añade un claim personalizado.

`.issuedAt(...)` añade iat.

`.expiration(...)` añade exp.

`.signWith(clave)` firma con la clave.

`.compact()` devuelve el token como string.

`Jwts.parser().verifyWith(clave).build().parseSignedClaims(token).getPayload()` parsea el token, verifica la firma y devuelve los claims.

`extraerClaims(String token)` expone los claims completos del token. Es útil cuando se necesita acceder a más de un claim (por ejemplo, el ID y el email a la vez). En el punto 6.7 lo usaremos para construir un `UsuarioPrincipal` a partir de los claims.

`JwtException` es la excepción que lanza JWT si el token es inválido (firma incorrecta, expirado, mal formado).

`parsearClaims` es privado y `extraerClaims` es público. `parsearClaims` es un método interno que encapsula la lógica de parseo con JWT. `extraerClaims` es la API pública del servicio para obtener los claims. Esta separación permite cambiar la implementación interna (por ejemplo, añadir caché o logs) sin afectar al resto del código.

Pregunta: ¿Qué diferencia hay entre `parseSignedClaims` y `parseClaimsJwt`?

Implementación acumulativa M6 — actualizada

Archivo: `auth/JwtService.java`.

Implementa:

- `generarToken(Usuario);`
- `generarRefreshToken(Usuario);`
- `extraerUsername / extraerId / extraerClaims;`
- `extraerAuthorities;`
- `esValido;`
- `esAccessTokenValido;`
- `esRefreshTokenValido.`

El access contiene `tipo=access`, `jti`, `id/email/roles`; refresh contiene `tipo=refresh` y `jti`, sin roles.

```
Claims claims = Jwts.parser()
    .verifyWith(clave)
    .build()
    .parseSignedClaims(token)
    .getPayload();
```

Estado T6.5-A — CLOSE/EVOLVE. A partir de aquí JWT es la implementación operativa; `JwtManual` queda sólo como ejercicio/test.

Error/corrección: un claim numérico puede llegar como cualquier `Number`; M6 convierte con `longValue()` y no asume que siempre sea `Long`.

Paso 6.6.5 — Crear los DTOs de login

Fuente: p. 1416.

Base de la fuente — preservada

Crea LoginRequestDTO:

```
package es.mecd.demo.miproyecto.auth;

import jakarta.validation.constraints.NotBlank;

public class LoginRequestDTO {

    @NotBlank(message = "El nombre de usuario es obligatorio")
    private String username;

    @NotBlank(message = "La contraseña es obligatoria")
    private String password;

    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }

    public String getPassword() { return password; }

    public void setPassword(String password) { this.password = password; }
}
```

Crea LoginResponseDTO:

```
package es.mecd.demo.miproyecto.auth;

import com.fasterxml.jackson.annotation.JsonProperty;

public class LoginResponseDTO {

    @JsonProperty("access_token")
    private String accessToken;

    @JsonProperty("refresh_token")
    private String refreshToken;

    @JsonProperty("token_type")
    private String tokenType = "Bearer";

    @JsonProperty("expires_in")
    private long expiresIn;

    public LoginResponseDTO(String accessToken, String refreshToken, long
        ↪ expiresIn) {
        this.accessToken = accessToken;

        this.refreshToken = refreshToken;
        this.expiresIn = expiresIn;
    }

    public String getAccessToken() { return accessToken; }
    public String getRefreshToken() { return refreshToken; }
```

```

    public String getTokenType() { return tokenType; }
    public long getExpiresIn() { return expiresIn; }
}

```

Crea RefreshRequestDTO:

```

package es.mecd.demo.miproyecto.auth;

import jakarta.validation.constraints.NotBlank;

public class RefreshRequestDTO {

    @NotBlank(message = "El refresh token es obligatorio")
    private String refreshToken;

    public String getRefreshToken() { return refreshToken; }
    public void setRefreshToken(String refreshToken) { this.refreshToken =
        ↪ refreshToken; }
}

```

Pregunta: ¿Por qué LoginResponseDTO tiene @JsonProperty con nombres en snake_case?

Implementación acumulativa M6 — actualizada

Archivos:

- LoginRequestDTO;
- LoginResponseDTO;
- RefreshRequestDTO.

LoginResponseDTO devuelve snake_case con @JsonProperty y expires_in en segundos.

```

{
  "access_token": "...",
  "refresh_token": "...",
  "token_type": "Bearer",
  "expires_in": 3600
}

```

Error/corrección: no devolver expires_in=3600000; convertir ms → s.

Paso 6.6.6 — Ampliar AuthService con login y refresh

Fuente: p. 1419.

Base de la fuente — preservada

Modifica el AuthService:

```

@Service
public class AuthService {

    private final AuthenticationManager authenticationManager;
    private final JwtService jwtService;
    private final UsuarioRepository usuarioRepository;
    private final RolRepository rolRepository;
    private final PasswordEncoder passwordEncoder;

    public AuthService(AuthenticationManager authenticationManager,
        JwtService jwtService,

```

```

        UsuarioRepository usuarioRepository,
        RolRepository rolRepository,
        PasswordEncoder passwordEncoder) {
    this.authenticationManager = authenticationManager;
    this.jwtService = jwtService;

    this.usuarioRepository = usuarioRepository;
    this.rolRepository = rolRepository;
    this.passwordEncoder = passwordEncoder;
}

public LoginResponseDTO login(LoginRequestDTO request) {
    try {
        authenticationManager.authenticate(
            new UsernamePasswordAuthenticationToken(
                request.getUsername(), request.getPassword()));
    } catch (BadCredentialsException e) {
        throw new NegocioException("Credenciales incorrectas");
    }

    Usuario usuario = usuarioRepository.findByUsername(request.getUsername())
        .orElseThrow(() -> new RecursoNoEncontradoException("Usuario",
            ↪ request.getUsername()));

    String token = jwtService.generarToken(usuario);
    String refreshToken = jwtService.generarRefreshToken(usuario);

    return new LoginResponseDTO(token, refreshToken, jwtService.getExpirationMs() /
        ↪ 1000);
}

public LoginResponseDTO refresh(String refreshToken) {
    if (!jwtService.esValido(refreshToken)) {

        throw new NegocioException("Refresh token inválido o expirado");
    }

    String username = jwtService.extraerUsername(refreshToken);
    Usuario usuario = usuarioRepository.findByUsername(username)
        .orElseThrow(() -> new RecursoNoEncontradoException("Usuario",
            ↪ username));

    String nuevoToken = jwtService.generarToken(usuario);
    String nuevoRefreshToken = jwtService.generarRefreshToken(usuario);

    return new LoginResponseDTO(nuevoToken, nuevoRefreshToken,
        ↪ jwtService.getExpirationMs() / 1000);
}

// ... resto de métodos (registrar, etc.)
}

```

Añade los imports:

```

import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.BadCredentialsException;

```

```
import org.springframework.security.authentication.
    UsernamePasswordAuthenticationToken;
```

authenticationManager.authenticate(...) valida las credenciales con el UserDetailsService y el PasswordEncoder.

BadCredentialsException se lanza si las credenciales son incorrectas.

jwtService.getExpirationMs() / 1000 convierte milisegundos a segundos.

Pregunta: ¿Por qué el refresh genera un nuevo refresh token además del token de acceso?

Implementación acumulativa M6 — actualizada

AuthService.login usa AuthenticationManager; no compara passwords manualmente. Captura AuthenticationException y devuelve una excepción funcional 401 sin enumerar usuarios.

refresh exige:

1. firma/expiración válidas;
2. tipo=refresh;
3. jti no consumido;
4. usuario aún existente;
5. generar un nuevo access y **nuevo refresh**.

TokenRevocationService.consumeRefresh hace que el refresh anterior sea de un solo uso dentro del proceso.

Corrección a la fuente: jwtService.esValido() por sí solo permitiría usar un access token como refresh. M6 separa tipos y lo rechaza.

Paso 6.6.7 — Ampliar AuthController

Fuente: p. 1422.

Base de la fuente — preservada

Añade los endpoints al AuthController:

```
@PostMapping("/login")
public ResponseEntity<LoginResponseDTO> login(
    @Valid @RequestBody LoginRequestDTO request) {
    return ResponseEntity.ok(authService.login(request));
}

@PostMapping("/refresh")
public ResponseEntity<LoginResponseDTO> refresh(
    @Valid @RequestBody RefreshRequestDTO request) {
    return ResponseEntity.ok(authService.refresh(request.getRefreshToken()));
}
```

Pregunta: ¿Por qué el endpoint de login debe ser público?

Implementación acumulativa M6 — actualizada

Endpoints:

```
POST /api/v1/auth/registro
POST /api/v1/auth/login
POST /api/v1/auth/refresh
POST /api/v1/auth/logout
```

Login y refresh reciben DTO validado; logout extrae el Bearer y delega revocación.

Paso 6.6.8 — Configurar la seguridad para permitir login

Fuente: p. 1423.

Base de la fuente — preservada

Asegúrate de que el SecurityFilterChain permite /api/v1/auth/**:

```
.requestMatchers("/api/v1/auth/**").permitAll()
```

Si no está, el login requeriría autenticación, y nadie podría autenticarse.

Pregunta: ¿Qué pasa si olvidas permitir /api/v1/auth/**?

Implementación acumulativa M6 — actualizada

M6 **no** abre /api/v1/auth/** completo. Sólo son públicos:

```
.requestMatchers(  
    HttpMethod.POST,  
    "/api/v1/auth/registro",  
    "/api/v1/auth/login",  
    "/api/v1/auth/refresh")  
.permitAll()
```

Mejora: logout queda autenticado y un endpoint futuro bajo /auth no se hace público accidentalmente.

Observable: login/registro/refresh funcionan sin credencial previa; logout no.

Paso 6.6.9 — Arrancar y probar el login

Fuente: p. 1423.

Base de la fuente — preservada

Reinicia la aplicación. Prueba el login:

```
curl -i -X POST http://localhost:8080/api/v1/auth/login \  
-H "Content-Type: application/json" \  
-d '{"username": "ana", "password": "ana123"}'
```

Verás algo como:

```
{  
  "access_token": "eyJhbGciOiJIUzI1NiJ9...",  
  "refresh_token": "eyJhbGciOiJIUzI1NiJ9...",  
  "token_type": "Bearer",  
  "expires_in": 3600  
}
```

Pregunta: ¿Qué contiene el access_token? ¿Qué contiene el refresh_token?

Implementación acumulativa M6 — actualizada

```
curl -i -X POST http://localhost:8080/api/v1/auth/login \  
-H "Content-Type: application/json" \  
-d '{"username": "ana", "password": "ana123"}'
```

Esperado: 200 con access/refresh/Bearer/expiración.

Prueba contraseña incorrecta: **401 CREDENCIALES_INVALIDAS**, no 500 y sin indicar si el username existe.

Paso 6.6.10 — Decodificar el token

Fuente: p. 1424.

Base de la fuente — preservada

Copia el access_token y pégalo en jwt.io. Verás el payload:

```
{
  "sub": "ana",
  "id": 2,
  "email": "ana@educacion.gob.es",
  "roles": ["ROLE_USER"],
  "iat": 1700000000,
  "exp": 1700003600
}
```

El token contiene el nombre de usuario, el ID, el email y los roles. El cliente puede decodificarlo para saber cuándo expira y qué roles tiene.

Pregunta: ¿Qué campos del payload son claims registrados y cuáles son personalizados?

Implementación acumulativa M6 — actualizada

Usa el decoder local de 6.5 o jwt.io **sólo con tokens didácticos/locales**. El access debe mostrar sub, jti, tipo=access, id, email, roles, iat y exp.

Pregunta: ¿qué claims son registrados (sub, jti, iat, exp) y cuáles son privados (tipo, id, email, roles)?

Regla: visualizar no equivale a verificar.

Paso 6.6.11 — Probar el refresh

Fuente: p. 1425.

Base de la fuente — preservada

Prueba a renovar el token con el refresh token:

```
curl -i -X POST http://localhost:8080/api/v1/auth/refresh \
  -H "Content-Type: application/json" \
  -d '{"refresh_token": "eyJhbGciOiJIUzI1NiJ9..."}'
```

Verás un nuevo par de tokens. Prueba con un refresh token inválido:

```
curl -i -X POST http://localhost:8080/api/v1/auth/refresh \
  -H "Content-Type: application/json" \
  -d '{"refresh_token": "tokenInvalido"}'
```

Verás un 400 con el mensaje “Refresh token inválido o expirado”.

Pregunta: ¿Qué pasa si el refresh token ha expirado?

Implementación acumulativa M6 — actualizada

```
curl -i -X POST http://localhost:8080/api/v1/auth/refresh \
  -H "Content-Type: application/json" \
  -d '{"refresh_token":"<REFRESH>"}'
```

Primer uso: 200 + nuevo par. Segundo uso del mismo refresh: **401 TOKEN_INVALIDO**.

Prueba también un access token en /refresh: debe ser 401 por tipo incorrecto.

Observable: la rotación está demostrada, no sólo documentada.

Paso 6.6.12 — Errores comunes del ejercicio

Fuente: p. 1426.

Base de la fuente — preservada

Error Causa Solución

Generar una clave más WeakKeyException La clave tiene menos de 32 bytes larga

BadCredentialsException no Se propaga al cliente como 500 Capturarla y devolver 401 capturada

El token no se genera Falta el bean SecretKey Añadirlo en JwtConfig

Verificar que se usa la El token no es válido Clave distinta al firmar y verificar misma clave

El endpoint de login devuelve 401 No es público Añadir permitAll

Se pasan milisegundos en lugar de expires_in es incorrecto Dividir por 1000 segundos

El refresh token no se valida Falta jwtService.esValido(...) Añadirlo

Error Causa Solución

El token contiene caracteres JWT lo maneja Codificación incorrecta extraños automáticamente

Implementación acumulativa M6 — actualizada

Error	Observable	Corrección
clave corta/Base64 inválido	arranque falla	secreto Base64 >=256 bits
credenciales malas producen 500	excepción sin traducir	CredencialesInvalidasException → 401
login devuelve 401 antes de autenticar	matcher cerrado	abrir sólo login/registro/refresh
expires_in enorme	ms enviados como s	dividir por 1000
access usado como refresh	tipo no validado	esRefreshTokenValido
refresh reutilizable	sin registro de uso	consumir jti
claims leídos sin firma	decoder usado para seguridad	parseSignedClaims
secreto en repo prod	configuración insegura	\${JWT_SECRET}

Materialización exacta del contrato de error en el proyecto final

La corrección anterior no queda sólo como recomendación textual. El snapshot final M6 la materializa en tres piezas concretas y trazables:

- `CredencialesInvalidasException.java`: representa el rechazo de usuario/contraseña con código estable `CREDENCIALES_INVALIDAS`;
- `TokenInvalidoException.java`: representa JWT expirado, revocado, mal firmado o de tipo incorrecto con código `TOKEN_INVALIDO`;
- `AuthExceptionHandler.java`: traduce ambas excepciones a HTTP 401 usando el mismo `ErrorResponse` heredado de M5, incluido `traceId`.

La prueba de login/refresh y el runtime gate verifican que esos errores no se convierten en 500 y que no revelan si falló el usuario, la contraseña o la firma concreta. Estas clases forman parte del resultado programado de este paso y quedan enlazadas explícitamente en `TRAZABILIDAD_M6.md`.

Paso 6.6.13 — Reto resuelto: endpoint para logout

Fuente: p. 1427.

Base de la fuente — preservada

Reto: Añadir un endpoint POST `/api/v1/auth/logout` que invalide el token. Como JWT es stateless, la invalidación se hace con una lista negra de tokens.

Solución paso a paso:

Paso 1: Crear una clase `TokenBlacklistService`:

```
package es.mecd.demo.miproyecto.auth;

import org.springframework.stereotype.Service;

import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;

@Service
public class TokenBlacklistService {

    private final Set<String> blacklist = ConcurrentHashMap.newKeySet();

    public void add(String token) {
        blacklist.add(token);
    }

    public boolean contains(String token) {
        return blacklist.contains(token);
    }
}
```

Paso 2: Añadir el endpoint al `AuthController`:

```
@PostMapping("/logout")
public ResponseEntity<Void> logout(
    @RequestHeader("Authorization") String authHeader) {
    String token = authHeader.replace("Bearer ", "");
    authService.logout(token);
    return ResponseEntity.noContent().build();
}
```

Paso 3: Añadir el método al `AuthService`:

```
public void logout(String token) {
    tokenBlacklistService.add(token);
}
```

```
}
```

Paso 4: En el filtro JWT (que veremos en 6.7), verificar la lista negra antes de aceptar el token.

Paso 5: Probar:

Login

```
TOKEN=$(curl -s -X POST http://localhost:8080/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"ana","password":"ana123"}' | jq -r '.access_token')
```

Usar el token

```
curl -i -H "Authorization: Bearer $TOKEN" http://localhost:8080/api/v1/perfil
```

Logout

```
curl -i -X POST -H "Authorization: Bearer $TOKEN" \
  http://localhost:8080/api/v1/auth/logout
```

El token ya no es válido

```
curl -i -H "Authorization: Bearer $TOKEN" http://localhost:8080/api/v1/perfil
```

Pregunta: ¿Por qué la lista negra en memoria no es una solución ideal en producción?

Resultado esperado global

Al final del ejercicio, deberías tener:

- Las dependencias de JWT añadidas.
- La configuración de la clave en application.properties.
- Un bean SecretKey en JwtConfig.
- Un JwtService que genera y valida tokens, con extraerClaims.
- Un endpoint de login que devuelve el token de acceso y el refresh token.
- Un endpoint de refresh que renueva los tokens.
- (Opcional) Un endpoint de logout con lista negra.

Resumen técnico del ejercicio El ejercicio ha demostrado:

- JWT: tres módulos (api, impl, jackson).
- Clave: SecretKey generada con Keys.hmacShaKeyFor.
- Configuración: clave en variable de entorno.
- Generación: Jwts.builder() con subject, claim, issuedAt, expiration, signWith, compact.
- Token de acceso: con roles y claims.
- Refresh token: sin roles, más duración.
- Login: AuthenticationManager + JwtService.
- Refresh: rotación de tokens.
- Logout: lista negra.
- extraerClaims: método público para obtener los claims.
- Buenas prácticas: HTTPS, tokens cortos, rotar clave, no meter datos sensibles.

Conclusión y enlace al siguiente punto En este punto 6.6 hemos generado tokens JWT reales con la librería JWT:

- JWT: librería estándar para JWT en Java.
- Clave secreta: `SecretKey` generada con `Keys.hmacShaKeyFor`, guardada en variable de entorno.
- Generación: con `Jwts.builder()`.
- Token de acceso: corta duración, con claims de usuario.
- Refresh token: larga duración, sin roles, rotativo.
- Login: `AuthenticationManager` valida credenciales, `JwtService` genera tokens.
- Refresh: renovación de tokens sin volver a pedir credenciales.
- Logout: lista negra de tokens.
- `extraerClaims`: método público para acceder a los claims del token.

En la práctica, hemos añadido JWT, configurado la clave, creado un `JwtService`, implementado el login y el refresh, y verificado los tokens generados con `jwt.io`.

La idea clave: la generación de tokens es el primer paso del flujo JWT. El cliente recibe un token firmado con la identidad y los permisos. En el siguiente punto, aprenderemos a que el servidor verifique ese token en cada petición con un filtro personalizado.

En el siguiente punto, 6.7 — Filtro JWT, implementaremos un filtro que intercepta las peticiones, extrae el token de la cabecera `Authorization`, verifica su firma, y coloca la identidad del usuario en el `SecurityContext`.

MÓDULO 6 — Bloque 3

Implementación acumulativa M6 — actualizada

`TokenRevocationService` guarda `jti` → exp de access tokens revocados. Logout:

```
curl -i -X POST \
  -H "Authorization: Bearer $TOKEN" \
  http://localhost:8080/api/v1/auth/logout
```

Esperado: 204 No Content.

El filtro 6.7 consultará esa revocación; después del logout, reutilizar el token debe producir 401.

Limitación explicada: el mapa en memoria no sirve como solución distribuida; cada instancia tendría una blacklist distinta. Producción requiere almacenamiento compartido o una política equivalente.

Cierre 6.6

JWT operativo, secretos coherentes, tipos access/refresh, refresh rotativo y logout diseñado. El siguiente punto activa el consumo de access tokens en cada petición.

6.7 — Filtro JWT

Práctica 6.7 — Filtro JWT

Preparación y contexto de la fuente

Contexto del ejercicio: Vamos a implementar un filtro JWT que intercepte las peticiones, extraiga el token de la cabecera `Authorization`, lo verifique con el `JwtService` y coloque un Usuario-

Principal en el SecurityContext. Después, registraremos el filtro en la cadena de seguridad y probaremos que los endpoints protegidos funcionan con el token.

Requisitos previos: Tener el proyecto mi-proyecto con el JwtService y el UsuarioPrincipal del punto 6.6 y 6.4.

Paso 6.7.1 — Verificar JwtService y UsuarioPrincipal

Fuente: p. 1450.

Base de la fuente — preservada

Abre el JwtService y verifica que tiene el método extraerClaims (lo añadimos en el punto 6.6). Y abre UsuarioPrincipal y verifica que tiene el constructor con (Long id, String username, String email, Collection authorities).

Si no los tienes, vuelve al punto 6.6 y al 6.4 respectivamente.

Pregunta: ¿Por qué el filtro necesita extraerClaims?

Implementación acumulativa M6 — actualizada

Comprueba que JwtService expone extraerClaims, extraerAuthorities y validación de access; UsuarioPrincipal debe aceptar (Long id, String username, String email, Collection<GrantedAuthority>).

```
grep -R "class JwtService\|class UsuarioPrincipal"
↪ src/main/java/es/mecd/demo/miproyecto/auth
```

Por qué: el filtro no consulta DB; construye principal desde claims ya verificados.

Paso 6.7.2 — Crear JwtAuthenticationFilter

Fuente: p. 1450.

Base de la fuente — preservada

Crea la clase JwtAuthenticationFilter en el paquete auth:

```
package es.mecd.demo.miproyecto.auth;

import io.jsonwebtoken.Claims;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.security.authentication.
    ↪ UsernamePasswordAuthenticationToken;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.context.SecurityContextHolder;
import
    ↪ org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;
```

```

import java.util.List;

@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private static final Logger log =
        ↪ LoggerFactory.getLogger(JwtAuthenticationFilter.class);

    private final JwtService jwtService;

    public JwtAuthenticationFilter(JwtService jwtService) {
        this.jwtService = jwtService;
    }

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain)
        throws ServletException, IOException {

        String authHeader = request.getHeader("Authorization");

        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            filterChain.doFilter(request, response);
            return;
        }

        String token = authHeader.substring(7);

        if (!jwtService.esValido(token)) {
            log.debug("Token inválido en petición {}", request.getRequestURI());
            filterChain.doFilter(request, response);

            return;
        }

        try {
            Claims claims = jwtService.extraerClaims(token);
            Long id = claims.get("id", Long.class);
            String username = claims.getSubject();
            String email = claims.get("email", String.class);
            List<GrantedAuthority> authorities = jwtService.extraerAuthorities(token);

            UsuarioPrincipal principal = new UsuarioPrincipal(id, username, email,
            ↪ authorities);

            UsernamePasswordAuthenticationToken auth =
                new UsernamePasswordAuthenticationToken(principal, null,
                ↪ authorities);
            auth.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
            SecurityContextHolder.getContext().setAuthentication(auth);

            log.debug("Usuario {} autenticado en petición {}", username,
            ↪ request.getRequestURI());
        } catch (Exception e) {
            log.debug("Error al construir el principal para la petición {}: {}",

```

```

        request.getRequestURI(), e.getMessage());
    }

    filterChain.doFilter(request, response);
}
}

```

Analicemos cada parte:

@Component registra el filtro como bean.

extends OncePerRequestFilter hereda la lógica que garantiza una sola ejecución.

private static final Logger log = ... logger para depurar. SLF4J, ya conocido.

private final JwtService jwtService dependencia inyectada por constructor.

String authHeader = request.getHeader("Authorization") lee la cabecera.

if (authHeader == null || !authHeader.startsWith("Bearer")) comprueba si hay token. Si no, deja pasar la petición sin autenticar.

String token = authHeader.substring(7) extrae el token.

if (!jwtService.esValido(token)) verifica el token. Si no es válido, deja pasar sin autenticar.

Claims claims = jwtService.extraerClaims(token) obtiene los claims.

Long id = claims.get("id", Long.class) extrae el ID. Fíjate en que JWT convierte el número del JSON a Long (puede devolver Integer en algunos casos, pero Long.class funciona porque el claim se serializó como número largo).

String username = claims.getSubject() extrae el sub.

String email = claims.get("email", String.class) extrae el email.

List authorities = jwtService.extraerAuthorities(token) extrae los roles.

UsuarioPrincipal principal = new UsuarioPrincipal(id, username, email, authorities) construye el principal. Fíjate en que no consulta la base de datos: todos los datos vienen del token.

new UsernamePasswordAuthenticationToken(principal, null, authorities) construye el Authentication autenticado. El segundo parámetro es null porque las credenciales ya no se necesitan.

SecurityContextHolder.getContext().setAuthentication(auth) coloca el Authentication en el SecurityContext.

filterChain.doFilter(request, response) continúa con la cadena.

try-catch captura cualquier excepción al construir el principal. Si algo falla, la petición sigue sin autenticar. El AuthorizationFilter decidirá después.

Pregunta: ¿Qué pasa si el token es válido pero el usuario ya no existe en la base de datos?

Implementación acumulativa M6 — actualizada

Archivo: auth/JwtAuthenticationFilter.java.

El filtro:

1. lee Authorization;
2. si no empieza por Bearer, continúa;
3. exige access token válido;
4. comprueba jti no revocado;
5. lee id/username/email/roles;

6. construye UsuarioPrincipal;
7. crea UsernamePasswordAuthenticationToken con authorities;
8. lo coloca en SecurityContextHolder;
9. siempre continúa la cadena.

No imprime el token completo en logs.

Error/corrección: olvidar filterChain.doFilter deja la petición bloqueada.

Paso 6.7.3 — Registrar el filtro en la cadena

Fuente: p. 1455.

Base de la fuente — preservada

Modifica el SecurityConfig:

```
@Bean
public SecurityFilterChain filterChain(
    HttpSecurity http,

    JwtAuthenticationFilter jwtAuthenticationFilter) throws Exception {
    http
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/v1/auth/**").permitAll()
            .requestMatchers("/api/v1/public/**").permitAll()
            .requestMatchers("/h2-console/**").permitAll()
            .requestMatchers("/swagger-ui/**", "/v3/api-docs/**").permitAll()
            .requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
            .anyRequest().authenticated()
        )
        .csrf(csrf -> csrf.disable())
        .sessionManagement(session -> session
            .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .addFilterBefore(jwtAuthenticationFilter,
            UsernamePasswordAuthenticationFilter.class);
    return http.build();
}
```

JwtAuthenticationFilter jwtAuthenticationFilter inyecta el filtro.

.sessionManagement(...) configura la política STATELESS.

.addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class) añade el filtro antes del UsernamePasswordAuthenticationFilter.

Añade los imports:

```
import org.springframework.security.config.http.SessionCreationPolicy;
```

Pregunta: ¿Qué pasaría si no se configurara STATELESS?

Implementación acumulativa M6 — actualizada

```
.sessionManagement(session -> session
    .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
.addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)
```

Observable: no se crea sesión HTTP y el JWT tiene oportunidad de autenticar antes de autorización.

Paso 6.7.4 — Desactivar HTTP Basic

Fuente: p. 1457.

Base de la fuente — preservada

Con JWT, ya no se usa HTTP Basic. Elimina la línea:

```
.httpBasic(Customizer.withDefaults());
```

Si la dejas, Spring Security seguirá aceptando autenticación HTTP Basic, lo que puede ser un problema de seguridad. El cliente debe usar JWT.

También puedes eliminar el import de Customizer si ya no se usa.

Pregunta: ¿Qué pasa si dejas HTTP Basic activado junto con JWT?

Implementación acumulativa M6 — actualizada

Se elimina definitivamente:

```
.httpBasic(Customizer.withDefaults())
```

Estado T6.2-B — CLOSE. Desde aquí, usuario/contraseña sólo se envía al login; el resto usa Bearer.

Gate negativo: SecurityConfig final no puede contener `.httpBasic(`.

Paso 6.7.5 — Arrancar y probar

Fuente: p. 1457.

Base de la fuente — preservada

Reinicia la aplicación. Prueba:

```
# Login para obtener el token
TOKEN=$(curl -s -X POST http://localhost:8080/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"ana","password":"anal23"}' | jq -r '.access_token')

echo "Token: $TOKEN"

# Petición sin token a endpoint protegido → 401
curl -i http://localhost:8080/api/v1/perfil

# Petición con token → 200
curl -i -H "Authorization: Bearer $TOKEN" http://localhost:8080/api/v1/perfil

# Petición a endpoint público sin token → 200
curl -i http://localhost:8080/api/v1/public/info
```

Observa los resultados:

- Sin token a endpoint protegido: 401.
- Con token: 200.
- Sin token a endpoint público: 200.

Pregunta: ¿Qué diferencia hay entre las tres respuestas?

Implementación acumulativa M6 — actualizada

```
TOKEN=$(curl -s -X POST http://localhost:8080/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"ana","password":"ana123"}' | jq -r '.access_token')

curl -i http://localhost:8080/api/v1/perfil
curl -i -H "Authorization: Bearer $TOKEN" http://localhost:8080/api/v1/perfil
curl -i http://localhost:8080/api/v1/public/info
```

Esperado: **401 / 200 / 200**.

Paso 6.7.6 — Verificar UsuarioPrincipal

Fuente: p. 1459.

Base de la fuente — preservada

Modifica el PerfilController para que devuelva más información del usuario autenticado. Ahora podemos usar UsuarioPrincipal porque el filtro lo construye:

```
@RestController
@RequestMapping("/api/v1/perfil")
public class PerfilController {

    @GetMapping
    public Map<String, Object> perfil(@AuthenticationPrincipal UsuarioPrincipal
    ↪ user) {
        return Map.of(
            "id", user.getId(),
            "username", user.getUsername(),
            "email", user.getEmail(),
            "authorities", user.getAuthorities().stream()
                .map(GrantedAuthority::getAuthority)
                .toList()
        );
    }
}
```

`@AuthenticationPrincipal UsuarioPrincipal user` inyecta directamente el
 ↪ `UsuarioPrincipal` que el filtro construyó a partir del token.

Reinicia y prueba:

```
curl -H "Authorization: Bearer $TOKEN" http://localhost:8080/api/v1/perfil
```

Verás:

```
{
  "id": 2,
  "username": "ana",
  "email": "ana@educacion.gob.es",
  "authorities": ["ROLE_USER"]
}
```

Pregunta: ¿Por qué ahora sí funciona `@AuthenticationPrincipal UsuarioPrincipal`?

Implementación acumulativa M6 — actualizada

PerfilController pasa a recibir:

```
@AuthenticationPrincipal UsuarioPrincipal user
```

y devuelve id, username, email y authorities.

Estado T6.4-A — CLOSE/EVOLVE. El principal preparado en 6.4 ya está activo para peticiones JWT.

Paso 6.7.7 — Probar con rol ADMIN

Fuente: p. 1460.

Base de la fuente — preservada

Prueba con el usuario admin:

```
TOKEN=$(curl -s -X POST http://localhost:8080/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"admin123"}' | jq -r '.access_token')
```

Endpoint de admin → 200

```
curl -i -H "Authorization: Bearer $TOKEN"
↪ http://localhost:8080/api/v1/admin/usuarios
```

Prueba con ana:

```
TOKEN=$(curl -s -X POST http://localhost:8080/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"ana","password":"ana123"}' | jq -r '.access_token')
```

Endpoint de admin → 403

```
curl -i -H "Authorization: Bearer $TOKEN"
↪ http://localhost:8080/api/v1/admin/usuarios
```

Pregunta: ¿Qué diferencia hay entre el 403 de ana y un 401?

Implementación acumulativa M6 — actualizada

Login como admin + /api/v1/admin/usuarios → 200. Login como ana → 403.

```
curl -i -H "Authorization: Bearer $TOKEN_ADMIN" \
  http://localhost:8080/api/v1/admin/usuarios
```

Diferencia: 403 significa que el filtro autenticó correctamente, pero authorities no satisfacen ADMIN.

Paso 6.7.8 — Probar con un token inválido

Fuente: p. 1461.

Base de la fuente — preservada

Prueba con un token manipulado:

```
curl -i -H "Authorization: Bearer tokenInvalido" \
  http://localhost:8080/api/v1/perfil
```

Verás un 401. El filtro detecta que el token no es válido, no autentica, y el AuthorizationFilter rechaza la petición.

Pregunta: ¿Qué diferencia hay entre un token inválido y un token expirado?

Implementación acumulativa M6 — actualizada

```
curl -i -H "Authorization: Bearer tokenInvalido" \
  http://localhost:8080/api/v1/perfil
```

Esperado: 401. El filtro no convierte el token inválido en 500; continúa sin identidad y la cadena rechaza el recurso protegido.

Paso 6.7.9 — Probar con cabecera mal formada

Fuente: p. 1462.

Base de la fuente — preservada

Prueba sin el prefijo Bearer:

```
curl -i -H "Authorization: $TOKEN" http://localhost:8080/api/v1/perfil
```

Verás un 401. El filtro no reconoce la cabecera porque no empieza por Bearer, así que no autentica.

Pregunta: ¿Por qué el filtro es estricto con el prefijo Bearer?

Implementación acumulativa M6 — actualizada

```
curl -i -H "Authorization: $TOKEN" http://localhost:8080/api/v1/perfil
```

Esperado: 401. Bearer es parte del contrato y evita interpretar otros esquemas como JWT.

Paso 6.7.10 — Depurar el filtro

Fuente: p. 1462.

Base de la fuente — preservada

Activa el log DEBUG para Spring Security y para el filtro:

```
logging.level.org.springframework.security=DEBUG
logging.level.es.mecd.demo.miproyecto.auth=DEBUG
```

Reinicia y haz una petición con token. En los logs verás:

```
Procesando petición GET /api/v1/perfil con cabecera Authorization: Bearer
  eyJhbGciOi...
```

```
Usuario ana autenticado en petición /api/v1/perfil
```

Con eso, puedes ver qué está pasando en el filtro.

Pregunta: ¿Qué información añaden los logs que no tenías antes?

Implementación acumulativa M6 — actualizada

Dev ya registra el paquete de la aplicación en DEBUG. Añade temporalmente si lo necesitas:

```
logging.level.org.springframework.security=DEBUG
```

Observa ruta, usuario y decisiones; **no** copies tokens completos al log. Restaura el nivel global tras depurar.

Paso 6.7.11 — Errores comunes del ejercicio

Fuente: p. 1229.

Base de la fuente — preservada

Error Causa Solución

401 siempre El filtro no se ejecuta o no autentica Verificar addFilterBefore

El Authentication no está Usar el constructor con 403 con token válido autenticado authorities

Error Causa Solución

La petición se queda bloqueada Falta filterChain.doFilter Añadirlo al final

NullPointerException authHeader es null Comprobar antes de usar

500 al parsear el token Excepción no capturada Capturar JwtException

El token no se lee La cabecera no empieza por Bearer Verificar el formato

La sesión se crea Falta STATELESS Añadirlo

El endpoint público devuelve El filtro no es tolerante Dejar pasar sin token 401

Implementación acumulativa M6 — actualizada

Error	Causa	Corrección
401 siempre	filtro no registrado	addFilterBefore
403 con token correcto	authorities vacías/incorrectas	construir Authentication con roles
petición colgada	falta doFilter	continuar cadena
500 token inválido	parseo no controlado	esAccessTokenValido + catch
públicos devuelven 401	filtro rechaza ausencia	continuar sin autenticar
sesión aparece	falta STATELESS	fijar policy
refresh autentica recursos	no se distingue tipo	sólo tipo=access
token de logout sigue activo	blacklist ignorada	consultar jti revocado

Paso 6.7.12 — Reto resuelto: SpEL con ID del usuario

Fuente: p. 1464.

Base de la fuente — preservada

Reto: Añadir un endpoint GET /api/v1/perfil/{id} que solo permita acceder al usuario con ese ID o a un admin. Usaremos una expresión SpEL que acceda al ID del UsuarioPrincipal.

Solución paso a paso:

Paso 1: Añadir el endpoint al PerfilController:

```
@GetMapping("/{id}")
@PreAuthorize("#id == authentication.principal.id or hasRole('ADMIN')")
public Map<String, Object> perfilPorId(@PathVariable Long id,
```

```

        @AuthenticationPrincipal UsuarioPrincipal
        ↪ user) {

    return Map.of(
        "id", user.getId(),
        "username", user.getUsername(),
        "email", user.getEmail()
    );
}

```

#id == authentication.principal.id compara el parámetro id con el ID del
 ↪ UsuarioPrincipal. Como el principal es

un UsuarioPrincipal con getId(), la expresión funciona.

or hasRole('ADMIN') permite también el acceso a los admins.

Paso 2: Probar:

Ana consulta su propio perfil (id=2) → 200

```

TOKEN_ANA=$(curl -s -X POST http://localhost:8080/api/v1/auth/login \
    -H "Content-Type: application/json" \
    -d '{"username":"ana","password":"anal23"}' | jq -r '.access_token')

```

```

curl -i -H "Authorization: Bearer $TOKEN_ANA" \
    http://localhost:8080/api/v1/perfil/2

```

Ana consulta el perfil de admin (id=1) → 403

```

curl -i -H "Authorization: Bearer $TOKEN_ANA" \
    http://localhost:8080/api/v1/perfil/1

```

Admin consulta cualquier perfil → 200

```

TOKEN_ADMIN=$(curl -s -X POST http://localhost:8080/api/v1/auth/login \
    -H "Content-Type: application/json" \
    -d '{"username":"admin","password":"admin123"}' | jq -r '.access_token')

```

```

curl -i -H "Authorization: Bearer $TOKEN_ADMIN" \
    http://localhost:8080/api/v1/perfil/2

```

Pregunta: ¿Qué habría pasado si el principal fuera un String en lugar de un UsuarioPrincipal?

Resultado esperado global

Al final del ejercicio, deberías tener:

- Un JwtAuthenticationFilter que extrae y verifica el token.
- El filtro registrado en la cadena de seguridad.
- STATELESS configurado.
- HTTP Basic desactivado.
- Un PerfilController que usa @AuthenticationPrincipal UsuarioPrincipal.
- Un endpoint con expresión SpEL que accede al ID del principal.
- Endpoints protegidos que funcionan con token.
- Endpoints públicos que funcionan sin token.

Resumen técnico del ejercicio El ejercicio ha demostrado:

- OncePerRequestFilter: clase base para filtros.
- Extracción del token: cabecera Authorization: Bearer .

- Verificación: `jwtService.esValido(token)`.
- Construcción del `UsuarioPrincipal`: a partir de los claims del token.
- `SecurityContextHolder`: donde se guarda el `Authentication`.
- Registro: `addFilterBefore(...)`.
- `STATELESS`: no crear sesiones.
- Tolerancia: dejar pasar sin token.
- Expresión SpEL: `#id == authentication.principal.id`.
- Depuración: logs, `DEBUG`.

Conclusión y enlace al siguiente punto En este punto 6.7 hemos implementado el filtro JWT:

- Qué es: un filtro personalizado que intercepta peticiones y autentica con JWT.
- `OncePerRequestFilter`: clase base para garantizar una sola ejecución.
- Extracción: cabecera `Authorization: Bearer` .
- Verificación: con el `JwtService`.
- Construcción del `UsuarioPrincipal`: a partir de los claims del token, sin consultar la base de datos.
- `SecurityContextHolder`: donde se guarda la identidad.
- Registro: en el `SecurityFilterChain` con `addFilterBefore`.
- `STATELESS`: no crear sesiones.
- Tolerancia: dejar pasar sin token.

En la práctica, hemos implementado el filtro, lo hemos registrado, y hemos probado que los endpoints protegidos funcionan con token y los públicos sin él.

La idea clave: el filtro JWT es el corazón de la autenticación stateless. Sin él, el servidor ignora el token. Con él, cada petición lleva su identidad.

En el siguiente punto, 6.8 — Protección de endpoints con JWT, integraremos todo lo que hemos construido: configuraremos el `SecurityFilterChain` completo con reglas por endpoint, añadiremos manejadores de error personalizados, y probaremos el flujo completo de login → petición autenticada → refresh → logout.

Fin del Punto 6.7.

Implementación acumulativa M6 — actualizada

Endpoint:

```
@GetMapping("/{id}")
@PreAuthorize("#id == authentication.principal.id or hasRole('ADMIN')")
public UsuarioResponseDTO perfilPorId(@PathVariable Long id) {
    return usuarioService.consultar(id);
}
```

Prueba:

- Ana consulta su ID → 200.
- Ana consulta ID ajeno → 403.
- Admin consulta cualquier ID → 200 **con el usuario solicitado**.

Corrección a la fuente: el ejemplo original devolvía los claims del principal incluso cuando admin pedía otro id; M6 consulta el recurso correcto tras autorizar.

Cierre 6.7

Filtro activo, STATELESS, HTTP Basic cerrado y UsuarioPrincipal integrado. Sólo queda consolidar errores y el flujo completo.

6.8 — Protección de endpoints con JWT**Práctica 6.8 — Protección de endpoints con JWT****Preparación y contexto de la fuente**

Contexto del ejercicio: Vamos a integrar todo lo que hemos construido en un SecurityFilterChain completo, con manejadores de error personalizados. Probaremos el flujo completo: registro, login, peticiones autenticadas, refresh y logout.

Requisitos previos: Tener el proyecto mi-proyecto con el filtro JWT del punto 6.7.

Paso 6.8.1 — Crear JwtAuthenticationEntryPoint

Fuente: p. 1487.

Base de la fuente — preservada

Crea la clase JwtAuthenticationEntryPoint en el paquete auth:

```
package es.mecd.demo.miproyecto.auth;

import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.security.core.AuthenticationException;
import org.springframework.security.web.AuthenticationEntryPoint;
import org.springframework.stereotype.Component;

import java.io.IOException;

import java.time.Instant;

@Component
public class JwtAuthenticationEntryPoint implements AuthenticationEntryPoint {

    @Override
    public void commence(HttpServletRequest request,
                        HttpServletResponse response,
                        AuthenticationException authException) throws IOException {
        response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
        response.setContentType("application/json");
        response.setCharacterEncoding("UTF-8");

        String json = ""
            {
                "timestamp": "%s",
                "status": 401,
                "codigo": "NO_AUTENTICADO",
                "mensaje": "Se requiere autenticación para acceder a este
↵ recurso",
                "path": "%s"
```

```

        }
        """,formatted(Instant.now().toString(), request.getRequestURI());

        response.getWriter().write(json);
    }
}

```

implements AuthenticationEntryPoint implementa la interfaz.

commence(...) es el método que se ejecuta cuando un usuario no autenticado intenta acceder a un recurso protegido.

response.getWriter().write(json) escribe la respuesta JSON.

Pregunta: ¿Por qué este manejador no puede usar @RestControllerAdvice?

Implementación acumulativa M6 — actualizada

Implementa AuthenticationEntryPoint y delega en SecurityErrorWriter.

Esperado 401: JSON ErrorResponse con timestamp, status, codigo=NO_AUTENTICADO, mensaje, path y traceId.

Por qué no @RestControllerAdvice: la petición puede fallar dentro de Security antes de entrar en MVC.

Paso 6.8.2 — Crear JwtAccessDeniedHandler

Fuente: p. 1489.

Base de la fuente — preservada

Crea la clase JwtAccessDeniedHandler:

```

package es.mecd.demo.miproyecto.auth;

import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.security.access.AccessDeniedException;
import org.springframework.security.web.access.AccessDeniedHandler;
import org.springframework.stereotype.Component;

import java.io.IOException;

import java.time.Instant;

@Component
public class JwtAccessDeniedHandler implements AccessDeniedHandler {

    @Override
    public void handle(HttpServletRequest request,
                      HttpServletResponse response,
                      AccessDeniedException accessDeniedException) throws
        IOException {
        response.setStatus(HttpServletResponse.SC_FORBIDDEN);
        response.setContentType("application/json");
        response.setCharacterEncoding("UTF-8");
    }
}

```

```

String json = ""
    {
        "timestamp": "%s",
        "status": 403,
        "codigo": "ACCESO_DENEGADO",
        "mensaje": "No tiene permisos para acceder a este recurso",
        "path": "%s"
    }
    """.formatted(Instant.now().toString(), request.getRequestURI());

response.getWriter().write(json);
}
}

```

implements AccessDeniedHandler implementa la interfaz para manejar 403.

Pregunta: ¿Qué diferencia hay entre un 401 y un 403 en este contexto?

Implementación acumulativa M6 — actualizada

Implementa AccessDeniedHandler y devuelve el mismo contrato con status 403 y código=ACCESO_DENEGADO.

Observable: 401 y 403 ya no dependen de páginas HTML ni de formatos distintos al resto de M5.

Paso 6.8.3 — Actualizar SecurityConfig

Fuente: p. 1491.

Base de la fuente — preservada

Modifica el SecurityConfig para incluir los manejadores:

```

package es.mecd.demo.miproyecto.config;

import es.mecd.demo.miproyecto.auth.JwtAccessDeniedHandler;
import es.mecd.demo.miproyecto.auth.JwtAuthenticationEntryPoint;
import es.mecd.demo.miproyecto.auth.JwtAuthenticationFilter;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.method.configuration.
    ↪ EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.
    ↪ EnableWebSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.factory.PasswordEncoderFactories;

import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.
    ↪ UsernamePasswordAuthenticationFilter;

@Configuration
@EnableWebSecurity

```

```

@EnableMethodSecurity
public class SecurityConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return PasswordEncoderFactories.createDelegatingPasswordEncoder();
    }

    @Bean
    public SecurityFilterChain filterChain(
        HttpSecurity http,
        JwtAuthenticationFilter jwtAuthenticationFilter,
        JwtAuthenticationEntryPoint entryPoint,
        JwtAccessDeniedHandler deniedHandler) throws Exception {
        http
            .csrf(csrf -> csrf.disable())
            .sessionManagement(session -> session
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/v1/auth/**").permitAll()
                .requestMatchers("/api/v1/public/**").permitAll()
                .requestMatchers("/h2-console/**").permitAll()
                .requestMatchers("/swagger-ui/**",
                    ↪ "/v3/api-docs/**").permitAll()
                .requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
                .requestMatchers("/api/v1/gestor/**").hasAnyRole("GESTOR",
                    ↪ "ADMIN")
                .anyRequest().authenticated()
            )
            .exceptionHandling(ex -> ex
                .authenticationEntryPoint(entryPoint)
                .accessDeniedHandler(deniedHandler)
            )
            .addFilterBefore(jwtAuthenticationFilter,
                UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }
}

```

Pregunta: ¿Qué pasa si no se configuran los manejadores de error?

Implementación acumulativa M6 — actualizada

Cadena final:

```

.csrf(AbstractHttpConfigurer::disable)
.cors(Customizer.withDefaults())
.sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
.authorizeHttpRequests(...)
.exceptionHandling(ex -> ex
    .authenticationEntryPoint(entryPoint)
    .accessDeniedHandler(deniedHandler))
.addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)

```

Reglas públicas específicas, ADMIN, GESTOR/ADMIN y anyRequest().authenticated() al final.

Regresión M5: CORS y OpenAPI se conservan explícitamente.

Paso 6.8.4 — Añadir endpoint de gestor

Fuente: p. 1493.

Base de la fuente — preservada

Vamos a añadir un endpoint que requiera el rol GESTOR. Crea un GestorController:

```
package es.mecd.demo.miproyecto.auth;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.Map;

@RestController
@RequestMapping("/api/v1/gestor")
public class GestorController {

    @GetMapping("/documentos")
    public Map<String, String> documentos() {
        return Map.of("mensaje", "Lista de documentos del gestor");
    }
}
```

Pregunta: ¿Quién puede acceder a este endpoint?

Implementación acumulativa M6 — actualizada

GestorController expone:

GET /api/v1/gestor/documentos

Y SecurityConfig exige GESTOR o ADMIN. El endpoint existe desde el reto 6.2 y aquí se verifica bajo JWT, no se duplica.

Paso 6.8.5 — Añadir rol GESTOR al inicializador

Fuente: p. 1494.

Base de la fuente — preservada

Modifica el UsuariosInicialesConfig para crear el rol GESTOR y asignárselo a un usuario:

```
Rol rolGestor = rolRepository.save(new Rol("GESTOR", "Gestor"));
```

```
Usuario gestor = new Usuario();
gestor.setUsername("gestor");
gestor.setPassword(passwordEncoder.encode("gestor123"));
gestor.setEmail("gestor@educacion.gob.es");
gestor.setActivo(true);
gestor.getRoles().add(rolGestor);
gestor.getRoles().add(rolUser);
usuarioRepository.save(gestor);
```

Reinicia la aplicación para que se cargue el nuevo usuario.

Pregunta: ¿Qué roles tiene ahora el usuario gestor?

Implementación acumulativa M6 — actualizada

UsuariosInicialesConfig ya es idempotente y conserva:

gestor / gestor123 → USER + GESTOR

Continuidad: migrar a DB/JWT no puede destruir un reto de puntos anteriores.

Paso 6.8.6 — Probar el flujo completo

Fuente: p. 1495.

Base de la fuente — preservada

Vamos a probar el flujo completo con curl:

1. Login

```
TOKEN=$(curl -s -X POST http://localhost:8080/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"gestor","password":"gestor123"}' | jq -r '.access_token')
```

```
echo "Token obtenido: ${TOKEN:0:50}..."
```

2. Consultar perfil

```
curl -i -H "Authorization: Bearer $TOKEN" http://localhost:8080/api/v1/perfil
```

3. Acceder a endpoint de gestor

```
curl -i -H "Authorization: Bearer $TOKEN"
↳ http://localhost:8080/api/v1/gestor/documentos
```

4. Intentar acceder a endpoint de admin (debe fallar con 403)

```
curl -i -H "Authorization: Bearer $TOKEN"
↳ http://localhost:8080/api/v1/admin/usuarios
```

Observa los códigos de estado:

- Perfil: 200.
- Gestor: 200.
- Admin: 403 (el gestor no tiene rol ADMIN).

Pregunta: ¿Qué diferencia hay entre el 403 de gestor y un 401?

Implementación acumulativa M6 — actualizada

Gestor:

1. login → token;
2. perfil → 200;
3. /gestor/documentos → 200;
4. /admin/usuarios → 403.

Esto distingue identidad válida de permiso insuficiente.

Paso 6.8.7 — Probar sin token

Fuente: p. 1497.

Base de la fuente — preservada

```
# Sin token a endpoint protegido → 401
curl -i http://localhost:8080/api/v1/perfil
```

```
# Sin token a endpoint público → 200
curl -i http://localhost:8080/api/v1/public/info
```

El primer comando devuelve el JSON del JwtAuthenticationEntryPoint:

```
{
  "timestamp": "2025-01-15T10:00:00Z",
  "status": 401,
  "codigo": "NO_AUTENTICADO",
  "mensaje": "Se requiere autenticación para acceder a este recurso",
  "path": "/api/v1/perfil"
}
```

El segundo devuelve 200.

Pregunta: ¿Qué formato tiene la respuesta 401? ¿Es consistente con el resto de la API?

Implementación acumulativa M6 — actualizada

```
curl -i http://localhost:8080/api/v1/perfil
curl -i http://localhost:8080/api/v1/public/info
```

Esperado: 401 JSON con traceId / 200.

Pregunta: ¿el 401 mantiene el contrato de errores M5? Debe hacerlo.

Paso 6.8.8 — Probar refresh token

Fuente: p. 1498.

Base de la fuente — preservada

```
# Login para obtener el refresh token
REFRESH=$(curl -s -X POST http://localhost:8080/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"gestor","password":"gestor123"}' | jq -r '.refresh_token')

# Usar el refresh token
curl -i -X POST http://localhost:8080/api/v1/auth/refresh \
  -H "Content-Type: application/json" \
  -d '{"refresh_token": "$REFRESH"}'
```

Verás un nuevo par de tokens. El refresh token antiguo ya no es válido (si implementaste la rotación).

Pregunta: ¿Qué pasa si intentas usar el mismo refresh token dos veces?

Implementación acumulativa M6 — actualizada

Obtén refresh, úsalo una vez y comprueba que devuelve un par nuevo. Reutiliza el antiguo: 401.

Mejora sobre la fuente: la rotación está implementada, no se limita a emitir otro token mientras el anterior sigue válido.

Paso 6.8.9 — Probar logout

Fuente: p. 1498.

Base de la fuente — preservada

Si implementaste el endpoint de logout con lista negra:

Login

```
TOKEN=$(curl -s -X POST http://localhost:8080/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"gestor","password":"gestor123"}' | jq -r '.access_token')
```

Usar el token

```
curl -i -H "Authorization: Bearer $TOKEN" http://localhost:8080/api/v1/perfil
```

Logout

```
curl -i -X POST -H "Authorization: Bearer $TOKEN"
  http://localhost:8080/api/v1/auth/logout
```

Intentar usar el token de nuevo

```
curl -i -H "Authorization: Bearer $TOKEN" http://localhost:8080/api/v1/perfil
```

El último comando debería devolver 401 si el logout funciona.

Pregunta: ¿Qué limitación tiene la lista negra en memoria?

Implementación acumulativa M6 — actualizada

Login → perfil 200 → logout 204 → mismo access token en perfil → 401.

El filtro consulta jti revocado antes de construir identidad.

Paso 6.8.10 — Escribir tests de integración

Fuente: p. 1499.

Base de la fuente — preservada

Añade tests al AuthControllerTest:

```
@WebMvcTest(AuthController.class)
```

```
class AuthControllerTest {
```

```
    @Autowired
```

```
    private MockMvc mockMvc;
```

```
    @MockBean
```

```
    private AuthService authService;
```

```
    @Test
```

```
    void login_debeDevolver200_cuandoCredencialesValidas() throws Exception {
        LoginResponseDTO response = new LoginResponseDTO("token", "refresh", 3600);
```

```

        when(authService.login(any())).thenReturn(response);

        mockMvc.perform(post("/api/v1/auth/login")
            .contentType(MediaType.APPLICATION_JSON)
            .content("""
                {"username":"ana","password":"ana123"}
            """))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.access_token").value("token"))
            .andExpect(jsonPath("$.token_type").value("Bearer"));
    }

    @Test
    void login_debeDevolverError_cuandoCredencialesInvalidas() throws Exception {

        when(authService.login(any()))
            .thenThrow(new NegocioException("Credenciales incorrectas"));

        mockMvc.perform(post("/api/v1/auth/login")
            .contentType(MediaType.APPLICATION_JSON)
            .content("""
                {"username":"ana","password":"incorrecta"}
            """))
            .andExpect(status().isConflict());
    }
}

```

Pregunta: ¿Qué cubren los dos tests?

Implementación acumulativa M6 — actualizada

M6 incluye tests reales del flujo, además de tests por rol. El ejemplo fuente con `@MockBean` se moderniza al estándar del curso: `@MockitoBean` cuando se usa un mock Spring. Para el filtro/DB se prefiere `@SpringBootTest`.

Ejecutar:

```
./mvnw -B test
```

Paso 6.8.11 — Errores comunes del ejercicio

Fuente: p. 1229.

Base de la fuente — preservada

Error Causa Solución

El 401 devuelve HTML Falta exceptionHandling Configurarlos

Error Causa Solución

El 401 no tiene el formato de la Manejador mal Revisar JwtAuthenticationEntryPoint API implementado

El 403 no se maneja Falta accessDeniedHandler Configurarlos

El endpoint de gestor devuelve El usuario no tiene el rol Asignarlo en el inicializador 403

El refresh no funciona El refresh token está expirado Verificar la configuración

El logout no invalida La lista negra no se consulta Verificar el filtro

anyRequest() bloquea lo público Está mal posicionado Moverlo al final

Los roles no coinciden Prefijo incorrecto Verificar generación y extracción

Implementación acumulativa M6 — actualizada

Error	Causa	Solución
401 HTML	entrypoint ausente	configurar handler JSON
403 no uniforme	handler ausente	JwtAccessDeniedHandler
gestor 403	rol no emitido/asignado	revisar initializer/claims
refresh viejo funciona dos veces	sin consumo de jti	rotación real
logout no invalida	filtro no consulta revocación	blacklist por jti
CORS falla tras Security	.cors() omitido	integrar CORS M5
OpenAPI 401	matchers no preservados	rutas Swagger/docs públicas
anyRequest tapa públicos	orden incorrecto	regla general al final

Paso 6.8.12 — Reto resuelto: consultar el propio usuario

Fuente: p. 1502.

Base de la fuente — preservada

Reto: Añadir un endpoint GET /api/v1/perfil/usuario que devuelva los datos del usuario autenticado, incluyendo el ID y el email, sin consultar la base de datos.

Solución paso a paso:

Paso 1: El filtro JWT ya construye un UsuarioPrincipal con el ID y el email a partir de los claims del token. Así que el endpoint

puede usar @AuthenticationPrincipal UsuarioPrincipal directamente.

Paso 2: Añadir el endpoint al PerfilController:

```
@GetMapping("/usuario")
public Map<String, Object> usuario(@AuthenticationPrincipal UsuarioPrincipal user) {
    return Map.of(
        "id", user.getId(),
        "username", user.getUsername(),
        "email", user.getEmail()
    );
}
```

Paso 3: Probar:

```
curl -H "Authorization: Bearer $TOKEN" http://localhost:8080/api/v1/perfil/usuario
```

Verás:

```
{
  "id": 3,
  "username": "gestor",
  "email": "gestor@educacion.gob.es"
}
```

Pregunta: ¿Por qué es mejor usar los claims del token que consultar la base de datos?

Resultado esperado global

Al final del ejercicio, deberías tener:

- Un `JwtAuthenticationEntryPoint` que devuelve 401 con JSON.
- Un `JwtAccessDeniedHandler` que devuelve 403 con JSON.
- Un `SecurityFilterChain` completo con reglas de autorización.
- Un usuario gestor con rol GESTOR.
- Un endpoint de gestor protegido por rol.
- Tests que cubren el login y los errores.

Resumen técnico del ejercicio

El ejercicio ha demostrado:

- `SecurityFilterChain` completo: CSRF off, stateless, filtro JWT, reglas, manejadores.
- `AuthenticationEntryPoint`: 401 con JSON.
- `AccessDeniedHandler`: 403 con JSON.
- Clasificación de endpoints: públicos, autenticados, con rol.
- Orden de reglas: específicas primero, `anyRequest()` al final.
- Flujo completo: login → petición → refresh → logout.
- Tests: login con `MockMvc`.

Conclusión y enlace al siguiente punto En este punto 6.8 hemos integrado todo lo que hemos construido en el Módulo 6:

- `SecurityFilterChain` completo: con todas las piezas.
- Manejadores de error personalizados: 401 y 403 con JSON.
- Clasificación de endpoints: públicos, autenticados, con rol.
- Orden de reglas: crítico para la seguridad.
- Roles desde el token: sin consultar la base de datos.
- Flujo completo: registro, login, peticiones, refresh, logout.
- Manejo de expiración: el cliente detecta 401 y refresca.

En la práctica, hemos configurado el `SecurityFilterChain` completo, hemos creado los manejadores de error, hemos añadido un usuario gestor, y hemos probado el flujo completo.

La idea clave: la seguridad no es una capa, son varias capas que trabajan juntas. El filtro autentica, las reglas de URL autorizan por área, las anotaciones autorizan por operación. Y todo ello con un formato de error consistente.

En el siguiente punto, 6.9 — Testing de seguridad, profundizaremos en cómo testear la seguridad de la API: `@WithMockUser`, `@WithUserDetails`, `SecurityMockMvcRequestPostProcessors`, tests de autenticación y autorización, y cómo verificar que los endpoints protegidos devuelven los códigos correctos.

Fin del Punto 6.8.

Implementación acumulativa M6 — actualizada

GET `/api/v1/perfil/usuario`

Devuelve id, username y email desde `UsuarioPrincipal`, sin acceso a repositorio.

```
curl -H "Authorization: Bearer $TOKEN" \  
http://localhost:8080/api/v1/perfil/usuario
```

Observable: datos coinciden con claims verificados.

Cierre 6.8

Cadena final, errores uniformes, roles, refresh/logout y rutas públicas/protegidas integrados. 6.9 convierte todo esto en regresión automática.

6.9 — Testing de seguridad

Práctica 6.9 — Testing de seguridad

Preparación y contexto de la fuente

Contexto del ejercicio: Vamos a escribir tests de seguridad para el UsuarioController y el Auth-Controller. Verificaremos los escenarios de autenticación, autorización y estructura de errores.

Requisitos previos: Tener el proyecto mi-proyecto con la configuración de seguridad del punto 6.8.

Paso 6.9.1 — Añadir spring-security-test

Fuente: p. 1528.

Base de la fuente — preservada

Abre el pom.xml y añade la dependencia:

```
<dependency>
  <groupId>org.springframework.security</groupId>

  <artifactId>spring-security-test</artifactId>
  <scope>test</scope>
</dependency>
```

Recarga Maven. Sin esta dependencia, las anotaciones @WithMockUser y los post-processors no están disponibles.

Pregunta: ¿Por qué la dependencia tiene scope=test?

Implementación acumulativa M6 — actualizada

La dependencia ya está en el POM con scope=test.

```
<dependency>
  <groupId>org.springframework.security</groupId>
  <artifactId>spring-security-test</artifactId>
  <scope>test</scope>
</dependency>
```

Observable: están disponibles @WithMockUser, @WithUserDetails y post-processors.

Paso 6.9.2 — Crear test del UsuarioController

Fuente: p. 1529.

Base de la fuente — preservada

Crea UsuarioControllerSecurityTest en src/test/java/es/mecd/demo/miproyecto/auth/:

```
package es.mecd.demo.miproyecto.auth;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.boot.test.mock.mockito.MockBean;
import org.springframework.security.test.context.support.WithMockUser;
import org.springframework.test.web.servlet.MockMvc;

import java.util.List;

import static org.mockito.Mockito.when;
import static
    org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static
    org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static
    org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

@WebMvcTest(UsuarioController.class)
class UsuarioControllerSecurityTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private UsuarioService usuarioService;

    @Test
    void listar_debeDevolver401_cuandoNoAutenticado() throws Exception {
        mockMvc.perform(get("/api/v1/admin/usuarios"))
            .andExpect(status().isUnauthorized())
            .andExpect(jsonPath("$.codigo").value("NO_AUTENTICADO"));
    }

    @Test
    @WithMockUser(roles = "USER")
    void listar_debeDevolver403_cuandoRolUser() throws Exception {
        mockMvc.perform(get("/api/v1/admin/usuarios"))
            .andExpect(status().isForbidden())
            .andExpect(jsonPath("$.codigo").value("ACCESO_DENEGADO"));
    }

    @Test
    @WithMockUser(roles = "ADMIN")
    void listar_debeDevolver200_cuandoRolAdmin() throws Exception {
        when(usuarioService.listarTodos()).thenReturn(List.of());

        mockMvc.perform(get("/api/v1/admin/usuarios"))
            .andExpect(status().isOk());
    }
}
```

`@WebMvcTest(UsuarioController.class)` carga solo el controlador y la infraestructura web.

`@MockBean` `UsuarioService` mockea el servicio.

`@WithMockUser(roles = "USER")` simula un usuario con rol USER.

`@WithMockUser(roles = "ADMIN")` simula un usuario con rol ADMIN.

Pregunta: ¿Por qué el test sin anotación devuelve 401?

Implementación acumulativa M6 — actualizada

`UsuarioControllerSecurityTest` usa contexto completo de test para validar la configuración real y cubre:

- anónimo → 401 + NO_AUTENTICADO;
- USER → 403 + ACCESO_DENEGADO;
- ADMIN → 200.

M6 evita `@MockBean`. Cuando se necesita sustituir un bean Spring, el contrato exige `@MockitoBean`.

Paso 6.9.3 — Ejecutar los tests

Fuente: p. 1532.

Base de la fuente — preservada

Ejecuta los tests:

```
./mvnw test -Dtest=UsuarioControllerSecurityTest
```

Verás que los tres tests pasan. Si alguno falla, revisa la configuración de seguridad.

Pregunta: ¿Qué pasa si el `SecurityFilterChain` no está bien configurado? ¿Los tests lo detectarían?

Implementación acumulativa M6 — actualizada

Ejecutamos la suite **acumulativa completa**, no sólo los tests nuevos:

```
./mvnw -B clean verify
```

El objetivo final es **58 tests**: los 32 heredados de M5 más 26 tests M6, sin fallos, errores ni skipped, y con Checkstyle a cero.

Cinco `@WebMvcTest` heredados se mantienen como tests de controlador/error/CORS, no como tests de seguridad. Al entrar Spring Security en el classpath de M6 se evolucionan con `@AutoConfigureMockMvc(addFilters = false)` para que sigan verificando su contrato M5 y, además, con `@MockitoBean` para los dos colaboradores del `JwtAuthenticationFilter` (`JwtService` y `TokenRevocationService`). Esto último es necesario porque desactivar la aplicación de filtros en `MockMvc` **no desactiva la creación del bean filtro** durante el arranque del slice. La seguridad se prueba por separado en los tests dedicados de este módulo y en el runtime gate real.

Los cinco ficheros heredados que reciben esa modificación mínima y exclusivamente de aislamiento del slice son:

- `AlumnoControllerTest.java`;
- `ErrorHandlingTest.java`;

- CorsConfigTest.java;
- ExpedienteControllerTest.java;
- FicheroExceptionHandlerTest.java.

En cada uno de esos cinco slices heredados, el patrón de aislamiento queda explícito:

```
@AutoConfigureMockMvc(addFilters = false)
```

```
@MockitoBean
private JwtService jwtService;
```

```
@MockitoBean
private TokenRevocationService tokenRevocationService;
```

La trazabilidad inversa exige que estos cinco ficheros aparezcan como M5_EVOLVED, vinculados a 6.9.3, y que los otros tests heredados permanezcan byte-a-byte preservados.

Si cambia una regla y abre /admin, los escenarios USER/anónimo de la suite M6 deben fallar inmediatamente.

Paso 6.9.4 — Testear endpoint de gestor

Fuente: p. 1532.

Base de la fuente — preservada

Añade tests para el endpoint de gestor:

```
@Test
@WithMockUser(roles = "GESTOR")
void gestor_debeDevolver200_cuandoRolGestor() throws Exception {
    mockMvc.perform(get("/api/v1/gestor/documentos"))
        .andExpect(status().isOk());
}

@Test
@WithMockUser(roles = "ADMIN")
void gestor_debeDevolver200_cuandoRolAdmin() throws Exception {
    mockMvc.perform(get("/api/v1/gestor/documentos"))
        .andExpect(status().isOk());
}

@Test
@WithMockUser(roles = "USER")
void gestor_debeDevolver403_cuandoRolUser() throws Exception {
    mockMvc.perform(get("/api/v1/gestor/documentos"))
        .andExpect(status().isForbidden());
}
```

El endpoint de gestor permite GESTOR y ADMIN. Los tres tests verifican cada caso.

Pregunta: ¿Por qué el usuario con rol USER recibe 403 y no 401?

Implementación acumulativa M6 — actualizada

GestorSecurityTest verifica:

```
GESTOR → 200
ADMIN  → 200
```

USER → 403

Pregunta: USER está autenticado, por eso el resultado correcto es 403 y no 401.

Paso 6.9.5 — Testear endpoint público

Fuente: p. 1533.

Base de la fuente — preservada

Crea PublicControllerSecurityTest:

```
package es.mecd.demo.miproyecto.common.controller;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.test.web.servlet.MockMvc;

import static
    org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static
    org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

@WebMvcTest(PublicController.class)
class PublicControllerSecurityTest {

    @Autowired
    private MockMvc mockMvc;

    @Test
    void info_debeDevolver200_sinAutenticacion() throws Exception {
        mockMvc.perform(get("/api/v1/public/info"))
            .andExpect(status().isOk());
    }
}
```

Si el endpoint es público, el test pasa sin autenticación. Si no, devuelve 401.

Pregunta: ¿Qué pasaría si el endpoint público requiriera autenticación por error? ¿El test lo detectaría?

Implementación acumulativa M6 — actualizada

PublicControllerSecurityTest llama /api/v1/public/info sin autenticación y exige 200.

Este test protege contra una regresión frecuente: que un cambio en la cadena convierta accidentalmente un endpoint público en privado.

Paso 6.9.6 — Testear login

Fuente: p. 1535.

Base de la fuente — preservada

Crea AuthControllerSecurityTest:

```

package es.mecd.demo.miproyecto.auth;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.boot.test.mock.mockito.MockBean;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;

import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.when;
import static
    ↪ org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static
    ↪ org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static
    ↪ org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

@WebMvcTest(AuthController.class)
class AuthControllerSecurityTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private AuthService authService;

    @Test
    void login_debeDevolver200ConTokens_cuandoCredencialesValidas() throws Exception {
        LoginResponseDTO response = new LoginResponseDTO("token", "refresh", 3600);
        when(authService.login(any())).thenReturn(response);

        mockMvc.perform(post("/api/v1/auth/login")
            .contentType(MediaType.APPLICATION_JSON)
            .content("""
                {"username":"ana","password":"ana123"}
                """))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.access_token").value("token"))
            .andExpect(jsonPath("$.token_type").value("Bearer"));
    }

    @Test
    void login_debeDevolverError_cuandoCredencialesInvalidas() throws Exception {
        when(authService.login(any()))
            .thenThrow(new NegocioException("Credenciales incorrectas"));

        mockMvc.perform(post("/api/v1/auth/login")
            .contentType(MediaType.APPLICATION_JSON)
            .content("""
                {"username":"ana","password":"incorrecta"}
                """))
            .andExpect(status().isConflict());
    }
}

```

Pregunta: ¿Por qué el login no necesita @WithMockUser?

Implementación acumulativa M6 — actualizada

La suite end-to-end AuthFlowIntegrationTest ejecuta login real con ana/ana123 y exige token_type=Bearer.

La documentación conserva también el enfoque fuente de controller-slice con servicio mockeado, pero el proyecto final prioriza una prueba real del flujo de seguridad.

Credenciales inválidas deben producir 401 CREDENCIALES_INVALIDAS, no 409 del NegocioException histórico.

Paso 6.9.7 — Testear refresh

Fuente: p. 1537.

Base de la fuente — preservada

Añade tests para el refresh:

```
@Test
void refresh_debeDevolver200ConNuevosTokens_cuandoRefreshTokenValido() throws
↳ Exception {
    LoginResponseDTO response = new LoginResponseDTO("nuevoToken",
↳ "nuevoRefresh", 3600);
    when(authService.refresh("refreshValido")).thenReturn(response);

    mockMvc.perform(post("/api/v1/auth/refresh")

        .contentType(MediaType.APPLICATION_JSON)
        .content("""
            {"refresh_token":"refreshValido"}
            """))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.access_token").value("nuevoToken"))
        .andExpect(jsonPath("$.refresh_token").value("nuevoRefresh"));
}

@Test
void refresh_debeDevolverError_cuandoRefreshTokenInvalido() throws Exception {
    when(authService.refresh("refreshInvalido"))
        .thenThrow(new NegocioException("Refresh token inválido o expirado"));

    mockMvc.perform(post("/api/v1/auth/refresh")
        .contentType(MediaType.APPLICATION_JSON)
        .content("""
            {"refresh_token":"refreshInvalido"}
            """))
        .andExpect(status().isConflict());
}
```

Pregunta: ¿Qué cubren los dos tests?

Implementación acumulativa M6 — actualizada

Tras login, el test usa el refresh válido → 200. Vuelve a usar **el mismo** → 401 TOKEN_INVALIDO.

Así se verifica firma/tipo y la rotación, no sólo el controlador.

Paso 6.9.8 — Testear logout

Fuente: p. 1539.

Base de la fuente — preservada

Añade el test de logout:

```
@Test
void logout_debeDevolver204_cuandoTokenValido() throws Exception {
    mockMvc.perform(post("/api/v1/auth/logout")
        .header("Authorization", "Bearer tokenValido"))
        .andExpect(status().isNoContent());
}
```

Pregunta: ¿Por qué el logout espera 204 y no 200?

Implementación acumulativa M6 — actualizada

El mismo flujo hace logout con Bearer válido → 204 y luego intenta reutilizar access → 401.

Pregunta: 204 es apropiado porque la operación se completó y no necesita body.

Paso 6.9.9 — Testear con post-processors

Fuente: p. 1539.

Base de la fuente — preservada

Añade tests usando user() en lugar de @WithMockUser:

```
import static org.springframework.security.test.web.servlet.request.
    SecurityMockMvcRequestPostProcessors.user;

@Test
void listar_debeDevolver200_cuandoRolAdminConPostProcessor() throws Exception {

    when(usuarioService.listarTodos()).thenReturn(List.of());

    mockMvc.perform(get("/api/v1/admin/usuarios")
        .with(user("admin").roles("ADMIN")))
        .andExpect(status().isOk());
}

@Test
void listar_debeDevolver403_cuandoRolUserConPostProcessor() throws Exception {
    mockMvc.perform(get("/api/v1/admin/usuarios")
        .with(user("ana").roles("USER")))
        .andExpect(status().isForbidden());
}
```

.with(user("admin").roles("ADMIN")) aplica el usuario a esa petición concreta.

Pregunta: ¿Qué ventaja tiene usar user() en lugar de @WithMockUser?

Implementación acumulativa M6 — actualizada

UsuarioControllerSecurityTest usa:

```
.with(user("admin").roles("ADMIN"))
```

para aplicar una identidad a una petición concreta, además de `@WithMockUser`.

Ventaja: permite cambiar identidad por request sin modificar el contexto del método completo.

Paso 6.9.10 — Testear con `@WithUserDetails`

Fuente: p. 1540.

Base de la fuente — preservada

Para usar `@WithUserDetails`, necesitas un `UserDetailsService` configurado. En un test de integración con `@SpringBootTest`, puedes hacerlo:

```
@SpringBootTest
@AutoConfigureMockMvc
class UsuarioControllerIntegrationTest {

    @Autowired
    private MockMvc mockMvc;

    @Autowired
    private UsuarioRepository usuarioRepository;

    @Autowired
    private RolRepository rolRepository;

    @Autowired
    private PasswordEncoder passwordEncoder;

    @BeforeEach
    void setUp() {
        if (usuarioRepository.findByUsername("test").isEmpty()) {
            Rol rol = rolRepository.findByNombre("ADMIN")
                .orElseGet(() -> rolRepository.save(new Rol("ADMIN",
                    ↪ "Admin")));

            Usuario usuario = new Usuario();
            usuario.setUsername("test");
            usuario.setPassword(passwordEncoder.encode("test123"));

            usuario.setEmail("test@educacion.gob.es");
            usuario.setActivo(true);
            usuario.getRoles().add(rol);
            usuarioRepository.save(usuario);
        }
    }

    @Test
    @WithUserDetails("test")
    void listar_debeDevolver200_conUsuarioDeBD() throws Exception {
        mockMvc.perform(get("/api/v1/admin/usuarios"))
            .andExpect(status().isOk());
    }
}
```

`@WithUserDetails("test")` carga el usuario test desde el `UserDetailsService` configurado. Si el usuario tiene el rol ADMIN, el

endpoint devuelve 200.

`@WithUserDetails` requiere `@SpringBootTest` porque necesita un `UserDetailsService` en el contexto. `@WebMvcTest` solo carga la

capa web y no carga el `UserDetailsService`. Por eso no vale con `@WebMvcTest`.

Pregunta: ¿Qué ventaja tiene `@WithUserDetails` frente a `@WithMockUser` en este caso?

Implementación acumulativa M6 — actualizada

`UsuarioControllerIntegrationTest` usa:

```
@WithUserDetails("admin")
```

con `@SpringBootTest`. El usuario existe por el inicializador idempotente; Spring lo carga mediante nuestro `UsuarioDetailsService` real.

Observable: el test cubre la transformación DB → `UserDetails` → roles.

Paso 6.9.11 — Testear el filtro JWT

Fuente: p. 1543.

Base de la fuente — preservada

Para testear el filtro JWT, necesitas un test de integración que envíe un token real:

```
@SpringBootTest
@AutoConfigureMockMvc
class JwtFilterIntegrationTest {

    @Autowired
    private MockMvc mockMvc;

    @Autowired
    private JwtService jwtService;

    @Autowired
    private UsuarioRepository usuarioRepository;

    @Test
    void perfil_debeDevolver200_conTokenValido() throws Exception {
        Usuario usuario = usuarioRepository.findByUsername("ana").orElseThrow();
        String token = jwtService.generarToken(usuario);

        mockMvc.perform(get("/api/v1/perfil")
            .header("Authorization", "Bearer " + token))
            .andExpect(status().isOk());
    }

    @Test
    void perfil_debeDevolver401_conTokenInvalido() throws Exception {
        mockMvc.perform(get("/api/v1/perfil")
            .header("Authorization", "Bearer tokenInvalido"))
            .andExpect(status().isUnauthorized());
    }
}
```

```

@Test
void perfil_debeDevolver401_sinToken() throws Exception {
    mockMvc.perform(get("/api/v1/perfil"))
        .andExpect(status().isUnauthorized());
}
}

```

Estos tests son de integración: arrancan toda la aplicación, incluyendo el filtro JWT. Verifican que el filtro funciona con tokens reales.

Pregunta: ¿Por qué estos tests necesitan @SpringBootTest y no @WebMvcTest?

Implementación acumulativa M6 — actualizada

JwtServiceIntegrationTest verifica además, contra el contexto Spring real, que access y refresh son tipos mutuamente excluyentes, que el access contiene roles y que exp > iat. Es la evidencia directa de que el contrato criptográfico de 6.6 no es sólo sintáctico.

JwtFilterIntegrationTest genera un access token real con JwtService:

- token válido → perfil 200;
- token inválido → 401;
- sin token → 401.

Por qué no jwt() como sustituto: nuestro sistema no es el resource-server OAuth2 de Spring; necesitamos demostrar que **nuestro filtro** procesa una cabecera firmada por **nuestro JwtService**.

Paso 6.9.12 — Errores comunes del ejercicio

Fuente: p. 1426.

Base de la fuente — preservada

Error Causa Solución

NoClassDefFoundError Falta spring-security-test Añadirlo

Los tests devuelven 401 El filtro JWT no se ejecuta Usar @WithMockUser o @SpringBootTest siempre en @WebMvcTest

@WithMockUser no se aplica Falta la dependencia Añadirlo

El test espera 403 pero recibe El usuario no está autenticado Verificar el @WithMockUser 401

El test espera 200 pero recibe El usuario no tiene el rol Verificar el rol 403

El JSON del error no coincide Falta exceptionHandling Configurarlo

@WithUserDetails falla El usuario no existe Crearlo en @BeforeEach

Los tests de integración son Se usa @SpringBootTest Usar @WebMvcTest donde sea posible lentos

Implementación acumulativa M6 — actualizada

Error	Causa	Corrección
clases de security test no existen	falta dependencia	spring-security-test

Error	Causa	Corrección
esperaba 403, recibe 401	no hay identidad	preparar usuario antes de roles
esperaba 200, recibe 403 error JSON no coincide @WithUserDetails falla filtro no se prueba	rol incorrecto handlers no activos servicio/usuario ausente se simuló SecurityContext	revisar ROLE_/matcher verificar cadena completa @SpringBootTest + datos enviar JWT real en integración
tests lentos innecesarios	todo es @SpringBootTest	usar slice cuando no se necesite mecanismo real
tests dependen de orden	datos compartidos mutables	escenarios independientes

Paso 6.9.13 — Reto resuelto: test completo de autorización

Fuente: p. 1546.

Base de la fuente — preservada

Reto: Escribir un test completo que verifique los cinco escenarios para el endpoint /api/v1/admin/usuarios: 401 sin autenticar, 403 con USER, 403 con GESTOR, 200 con ADMIN, y estructura del error 403.

Solución paso a paso:

```
@WebMvcTest(UsuarioController.class)
class UsuarioControllerCompleteSecurityTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private UsuarioService usuarioService;

    // Escenario 1: sin autenticación → 401
    @Test
    void listar_debeDevolver401_cuandoNoAutenticado() throws Exception {
        mockMvc.perform(get("/api/v1/admin/usuarios"))
            .andExpect(status().isUnauthorized())
            .andExpect(jsonPath("$.codigo").value("NO_AUTENTICADO"))
            .andExpect(jsonPath("$.mensaje").exists())
            .andExpect(jsonPath("$.path").value("/api/v1/admin/usuarios"));
    }

    // Escenario 2: rol USER → 403
    @Test
    @WithMockUser(roles = "USER")
    void listar_debeDevolver403_cuandoRolUser() throws Exception {
        mockMvc.perform(get("/api/v1/admin/usuarios"))
            .andExpect(status().isForbidden())
            .andExpect(jsonPath("$.codigo").value("ACCESO_DENEGADO"));
    }

    // Escenario 3: rol GESTOR → 403
    @Test
    @WithMockUser(roles = "GESTOR")
```

```

void listar_debeDevolver403_cuandoRolGestor() throws Exception {
    mockMvc.perform(get("/api/v1/admin/usuarios"))
        .andExpect(status().isForbidden())
        .andExpect(jsonPath("$.codigo").value("ACCESO_DENEGADO"));
}

// Escenario 4: rol ADMIN → 200
@Test
@WithMockUser(roles = "ADMIN")
void listar_debeDevolver200_cuandoRolAdmin() throws Exception {
    when(usuarioService.listarTodos()).thenReturn(List.of());

    mockMvc.perform(get("/api/v1/admin/usuarios"))
        .andExpect(status().isOk());
}

// Escenario 5: verificar datos devueltos con ADMIN
@Test
@WithMockUser(roles = "ADMIN")
void listar_debeDevolverListaDeUsuarios_cuandoRolAdmin() throws Exception {
    UsuarioResponseDTO usuario = new UsuarioResponseDTO();
    usuario.setIdentificador("1");
    usuario.setUsername("ana");
    usuario.setEmail("ana@educacion.gob.es");
    usuario.setRoles(List.of("USER"));

    when(usuarioService.listarTodos()).thenReturn(List.of(usuario));

    mockMvc.perform(get("/api/v1/admin/usuarios"))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$[0].username").value("ana"))
        .andExpect(jsonPath("$[0].email").value("ana@educacion.gob.es"));
}
}

```

Los cinco tests cubren todos los escenarios. Si alguno falla, sabes exactamente qué regla de seguridad está mal.

Pregunta: ¿Por qué el test del escenario 3 (GESTOR) espera 403 y no 401?

Resultado esperado global

Al final del ejercicio, deberías tener:

- Un UsuarioControllerSecurityTest con tests de 401, 403 y 200.
- Un PublicControllerSecurityTest con test de acceso sin autenticación.
- Un AuthControllerSecurityTest con tests de login, refresh y logout.
- Tests con @WithMockUser y con user().
- (Opcional) Un test de integración con @WithUserDetails.
- (Opcional) Un test del filtro JWT con tokens reales.
- Todos los tests pasando.

Resumen técnico del ejercicio El ejercicio ha demostrado:

- spring-security-test: dependencia para tests de seguridad.
- @WithMockUser: simula usuario con roles.

- @WithUserDetails: carga usuario del UserDetailsService. Requiere @SpringBootTest.
- user() y jwt(): post-processors para peticiones.
- SecurityMockMvcResultMatchers: verifica el estado de seguridad.
- Tests de autorización: 401, 403, 200.
- Tests de autenticación: login, refresh, logout.
- Tests de endpoints públicos.
- Tests de integración con JWT.

Implementación acumulativa M6 — actualizada

UsuarioControllerCompleteSecurityTest cubre cinco escenarios:

1. anónimo → 401 + estructura NO_AUTENTICADO;
2. USER → 403 + ACCESO_DENEGADO;
3. GESTOR → 403;
4. ADMIN → 200;
5. ADMIN → lista con campos de usuario.

```
./mvnw -B -Dtest=UsuarioControllerCompleteSecurityTest test
```

Observable: si se abre accidentalmente el endpoint admin o cambia el formato de errores, el test falla.

Resultado esperado global del módulo

- 112/112 pasos trazados;
- 45/45 bloques y 135/135 subpuntos preservados;
- usuarios persistentes;
- Spring Security moderno;
- JWT JJWT 0.13.0;
- STATELESS;
- HTTP Basic eliminado;
- roles y método;
- refresh de un solo uso en el proceso;
- logout con revocación de access por jti;
- 401/403 en formato M5;
- CORS/OpenAPI preservados;
- suite nueva + suite heredada sin eliminar tests.